Home Lab Manager

ספר פרויקט מקצועי

פרט	ערך
שם בעברית	מנהל מעבדת הבית
שם באנגלית	Home Lab Manager
שם המוצר המופיע ב־UI	CyberRange
מחבר	Nave
תאריך יצירה	5 באוגוסט 2026
גרסת המסמך	1.1
ענף Git	`main`
Commit	`dc4f66b10bfcd43e2e47f47a8f7f83986c49ef92`

> הספר מתעד את מצב הקוד ב־commit לעיל. הקוד, המיגרציות וקובצי התצורה הם מקור האמת. מסמכי הריפו שימשו רק כרקע ונבדקו מולם.

תוכן עניינים

Home Lab Manager 1

ספר פרויקט מקצועי 1

תוכן עניינים 2

1. תקציר מנהלים 6

1.1 מקרא סטטוסים 6

2. רקע והגדרת הבעיה 7

2.1 מטרות 7

2.2 דרישות פונקציונליות מאומתות 8

2.3 דרישות לא־פונקציונליות והערכתן 8

3. קהלי יעד ותפקידי משתמשים 8

3.1 משתמש רגיל 8

3.2 מנהל מערכת 9

3.3 מגבלות משותפות 9

4. מקרי שימוש 9

4.1 מפת מקרי שימוש 9

4.2 UC-01 — הרשמה 10

4.3 UC-02 — התחברות וכניסה למסלול מוגן 11

4.4 UC-04 — פתיחת קורס והתחלת משימה 11

4.5 UC-05 — השלמת תוכן ופתיחת המשימה הבאה 12

4.6 UC-06 — יצירת Lab 12

4.7 UC-07 ו־UC-08 — טרמינל והגשת דגל 13

4.8 UC-09 — מחיקת Lab 13

4.9 UC-10 — ניהול פרופיל וסיסמה 14

4.10 UC-11 עד UC-13 — פעולות מנהל 14

5. סקירת הטכנולוגיות 14

6. ארכיטקטורת־על 15

7. מבנה הריפו 18

8. ארכיטקטורת ה־Backend 18

9. ארכיטקטורת ה־Frontend 21

10. מסד הנתונים 22

10.1 Data Dictionary 24

10.2 קשרים, indexes וכללים 26

11. API 26

11.1 Authentication, user, dashboard ו־profile 26

11.2 Academy ו־progress 27

11.3 Labs, CTF ו־terminal 28

11.4 Admin Academy 28

11.5 Admin operations 29

12. הרשמה, התחברות ואבטחה 30

12.1 מקור האמת להרשאות 32

12.2 חוזה ההרשמה וההתחברות 32

12.3 מסלול בקשה מוגנת 33

12.4 מפת איומים ובקרות נוכחיות 33

12.5 מאפייני אבטחה שאינם קיימים עדיין 34

13. מערכת האקדמיה והתקדמות המשתמש 34

13.1 אלגוריתם הגישה למשימות 36

13.2 נקודות, אחוזים ומקורות אמת 36

13.3 פרסום, עריכה ווידאו 37

13.4 ביצועים ועקביות ב־Dashboard 37

13.5 מקרי קצה שיש לבדוק 37

14. מערכת המעבדות 37

14.1 תרחישי הצלחה וכישלון 40

14.2 אינווריאנטים של מחזור החיים 40

14.3 Provisioning לפי ציר זמן 40

14.4 הקצאת פורט ובידוד 41

14.5 Cleanup, תחרותיות וריבוי replicas 41

15. WebSocket, SSH והטרמינל 42

15.1 Handshake וגבולות אמון 44

15.2 Backpressure וסיום session....44
15.3 PTY, resize וסודות תפעוליים....45
16. CTF, דגלים והשלמת משימות....45
16.1 סדר הבדיקות בהגשה....47
16.2 אטומיות ו־idempotency....47
16.3 הגנת ערכי דגל....48
17. פרופיל משתמש....48
17.1 חוזי validation מדויקים....48
17.2 עקביות client ו־session....49
18. ממשק הניהול....49
18.1 שכבות הגנה וגבולות פעולה....50
18.2 Pagination, filters ויציבות תוצאות....50
18.3 Auditability....51
19. תהליכים מלאים מקצה לקצה....51
19.1 הרשמה....51
19.2 התחברות....52
19.3 כניסה למסלול מוגן....52
19.4 פתיחת קטלוג וקורס....52
19.5 התחלת משימה....53
19.6 השלמת LESSON או PRACTICE והענקת נקודות....53
19.7 יצירת שיעור וידאו דרך Admin....54
19.8 הצגת סרטון למשתמש....54
19.9 יצירת Lab....55
19.10 התחברות לטרמינל....55
19.11 הגשת דגל....56
19.12 מחיקת Lab....56
19.13 עריכת פרופיל....56
19.14 שינוי סיסמה....57
19.15 שינוי role דרך Admin....57
19.16 סיום מעבדה דרך Admin....57
19.17 Dashboard לאחר שינוי מצב....58

20. ממשק משתמש....58
21. קטעי קוד מוסברים....74
21.1 הרכבת Router....74
21.2 JWT ומצב חשבון עדכני....75
21.3 נעילת משימות....75
21.4 יצירת Lab....76
21.5 VirtualBox....76
21.6 Bridge WebSocket–SSH....77
21.7 טרנזקציית דגל....77
21.8 AuthContext....77
21.9 API service....78
21.10 component מרכזי....78
22. התקנה והרצה על Ubuntu....78
22.1 דרישות מוקדמות....78
22.2 משתני סביבה....79
22.3 הכנת DB והרצה....79
22.4 בדיקות smoke מוצעות לאחר התקנה....80
22.5 תקלות נפוצות....80
23. בדיקות ואימות....80
24. אבטחה, מגבלות וחוב טכני....81
25. מצב המימוש הנוכחי....83
26. תוכנית המשך....84
26.1 דחוף — אבטחה ושלמות....84
26.2 אמינות ובדיקות....85
26.3 מוצר ותפעול....85
27. סיכום ומסקנות....85
28. נספחים....86
נספח א — מילון מונחים....86
נספח ב — משתני סביבה....86
נספח ג — רשימת migrations....86
נספח ד — מפת קבצים מרכזיים....87

נספח ה — מגבלות האימות.....87
נספח ו — מקורות.....88
נספח ז — קטלוג API מלא.....89
ז.1 Auth, Users, Dashboard ו־Profile.....89
ז.2 Academy ו־Progress.....90
ז.3 Labs, Terminal ו־CTF.....91
ז.4 Academy Admin.....92
ז.5 Admin Operations.....94

1. תקציר מנהלים

Home Lab Manager היא פלטפורמת לימוד סייבר בהתקנה עצמית, המחברת בין תוכן לימודי מדורג לבין סביבות תרגול אמיתיות ב־VirtualBox. משתמש נרשם, מתחבר, בוחר קורס ומשימה, מתקדם לפי סדר, מפעיל מעבדת VM במשימת `LAB`, עובד בטרמינל דפדפן ומגיש דגל CTF. מנהל מערכת מתחזק קורסים, פרקים, משימות, תרחישים ודגלים, ומנהל משתמשים ומעבדות פעילות.

המערכת מיועדת בעיקר ללומדי סייבר ולמפעיל יחיד או צוות קטן. היא פותרת את הפיצול בין Home Lab מערכת Backend מתזמן תוכן, מעקב התקדמות, יצירה ידנית של מכונות וירטואליות וטרמינל חיצוני. ה־Frontend מספק חוויית React אחידה. ה־SSH ו־`VBoxManage`, PostgreSQL.

מצב המוצר הוא **MVP רחב שמומש ומאומת בקוד**, אך עדיין אינו production-ready. Authentication, Academy, progress, Labs, CTF, Profile ורוב Admin ממומשים קיימים. מנגד, אין restart, reconciliation מלא לאחר deployment מתועד, אין בדיקות אינטגרציה מקצה לקצה, אין SSH ופרטי CORS פתוח, לפיתוח קשיחים בקוד JWT נשמר ב־`localStorage`, ה־Leaderboard הוא placeholder בלבד. ראיות מרכזיות: `backend/src/routes/mod.rs:12`, `backend/src/services/instance_service.rs:37`, `frontend/src/App.jsx:26`, וכל migrations תחת `backend/migrations/`.

1.1 מקרא סטטוסים

סטטוס	משמעות
ממומש ומאומת בקוד	נמצאה זרימה מלאה ב־route, handler/service/repository או

סטטוס	משמעות
	UI/API רלוונטיים
ממומש חלקית	קיים קוד שימושי, אך חסרה זרימה, הגנה, בדיקה או השלמה מהותית
מתוכנן בלבד	מופיע ב־roadmap או placeholder, ללא מימוש פעיל
מוזכר בתיעוד אך לא נמצא בקוד	טענה במסמך שאינה נתמכת במימוש הנוכחי
לא ניתן לאמת	חסרים נתוני runtime או שירות חיצוני הדרושים להוכחה

2. רקע והגדרת הבעיה

לימוד סייבר אפקטיבי מחייב מעבר מקריאה לתרגול. בסביבת Home Lab ידנית יש ליצור clones, להגדיר NAT ו־port forwarding, להפעיל מכונה, להמתין ל־SSH, לזכור לאיזה משתמש שייכת כל מכונה ולנקות משאבים. במקביל צריך לנהל סדר לימוד, נעילת תרגילים, ניקוד ותשובות CTF. ללא מערכת מרכזית, מצב ה־VM ומצב הלמידה מתפצלים בין כלים שאינם מכירים זה את זה.

הפתרון שבקוד הוא יישום Web בעל שלושה צירים: קטלוג Academy היררכי (`courses → sections → tasks`), מנוע Labs היוצר `environment` ו־`instance` וממפה אותם ל־VirtualBox, ומנוע התקדמות פר־משתמש. משימת `LAB` מקושרת ל־`scenario`; תרחיש מצביע בשם אל template קיים. ראיות: migrations `20260706012517_create_academy_tables.sql` ו־`20260706013954_create_lab_engine_tables.sql`, וכן `instance_service::create_user_lab`.

2.1 מטרות

- לאחד לימוד עיוני, תרגול ו־Labs במסלול אחד.
- לאכוף סדר משימות ולשמור התקדמות וניקוד.
- להקצות סביבת VM מבודדת למשתמש ולחשוף אליה SSH בדפדפן.
- למנוע צריכת משאבים בלתי מבוקרת באמצעות מעבדה פעילה אחת ו־idle expiry.
- לספק למנהל CRUD תוכן ותפעול משתמשים, דגלים ומעבדות.

2.2 דרישות פונקציונליות מאומתות

הרשמה והתחברות; JWT ל־24 שעות; תפקידי `User` ו־`Admin`; פרופיל ושינוי סיסמה; Academy וקורסים מפורסמים; משימות `LESSON`, `PRACTICE`, `LAB`; סרטון YouTube אופציונלי ל־LESSON; progress ונעילה; Dashboard; יצירה/מחיקה/expiry של Lab; WebSocket–SSH; דגלים וניקוד; Admin CRUD, pagination, filters ו־activity logs.

2.3 דרישות לא־פונקציונליות והערכתן

דרישה	מצב	הערכה מבוססת קוד
אבטחת סיסמאות	ממומש	bcrypt ב־`auth_service.rs:11` ו־`profile_service.rs`
עקביות נתונים	ממומש חלקית	constraints וטרנזקציה בהגשת דגל; provisioning חוצה DB/VirtualBox ללא טרנזקציה מבוזרת
זמינות	ממומש חלקית	cleanup מחזורי; אין health endpoint, retries מערכתיים או reconciliation
ביצועים	ממומש חלקית	pagination ב־Admin ואינדקסים; Frontend bundle גדול מ־500KB
ניתנות לתחזוקה	ממומש	שכבות ו־feature modules ברורות; מספר handlers עדיין ממפים שגיאות במחרוזות
בדיקות	ממומש חלקית	12 unit tests; אין integration/E2E/Frontend tests

3. קהלי יעד ותפקידי משתמשים

3.1 משתמש רגיל

משתמש רשום יכול להתחבר, לראות Dashboard, Academy וקורסים מפורסמים, להתחיל ולהשלים משימות לפי הסדר, ליצור Lab פעיל אחד, לפתוח terminal, למחוק Lab, להגיש דגל ולנהל את הפרופיל שלו. הוא אינו יכול לקרוא או לשנות Admin APIs. מקור ההרשאה הוא role עדכני שנקרא מהטבלה בכל REST request מוגן (`auth_middleware.rs:31-54`), ולא role ישן מתוך ה־JWT בלבד.

3.2 מנהל מערכת

`/admin` למנהל כל יכולות המשתמש ובנוסף מסכי: סקירה, Academy, scenarios, users, labs, flags ו־activity. ה־Frontend מסנן לפי role ב־`RequireAdmin.jsx`, אבל ההגנה הקובעת היא Backend: `auth_middleware` ולאחריו `admin_middleware`. Admin יכול להשבית חשבון, לשנות role, לאפס סיסמה, לסיים Lab של החשבון הפעיל ולנהל תוכן. קיימות מגבלות עסקיות המונעות השבתה/הורדת role עצמו ומונעות מחיקת ישויות מקושרות; ראיות ב־`admin_service.rs`.

3.3 מגבלות משותפות

עד refresh בעת token ולכן הוא אינו מאמת, שנשמר ב־`localStorage` session מסתמך על ה־UI. `users` endpoint או הרשאות ברמת קורס, tenants נוספים. אין קבוצות role הבאה. אין API לקריאת `/api/admin/users` הישן נשמר לצד; שניהם מוגנים למנהל (`user_routes.rs`).

4. מקרי שימוש

4.1 מפת מקרי שימוש

מזהה	מקרה שימוש	שחקן	תוצאה עסקית	מצב מימוש
UC-01	הרשמה	אורח	נוצר חשבון `user` ללא חשיפת hash	ממומש ומאומת בקוד
UC-02	התחברות	משתמש	מתקבל JWT ל־24 שעות ונפתח session מקומי	ממומש ומאומת בקוד
UC-03	כניסה למסלול מוגן	משתמש	ה־UI מוצג וה־API מאמת חשבון עדכני	ממומש ומאומת בקוד
UC-04	פתיחת קורס והתחלת משימה	משתמש	תוכן, progress ומשימה זמינה מוצגים	ממומש ומאומת בקוד
UC-05	השלמת LESSON/PRACTICE	משתמש	task מושלמת, נקודות נרשמות והבאה נפתחת	ממומש ומאומת בקוד

מזהה	מקרה שימוש	שחקן	תוצאה עסקית	מצב מימוש
UC-06	יצירת Lab	משתמש	VM משוכפלת, מופעלת ונגישה ב־SSH	ממומש בקוד; runtime לא אומת
UC-07	עבודה בטרמינל	משתמש	WebSocket מגושר ל־shell ב־VM שבבעלותו	ממומש בקוד; runtime לא אומת
UC-08	הגשת דגל	משתמש	task מושלמת, flag נרשם, מוענקים `tasks.points`־ו	ממומש ומאומת בקוד
UC-09	מחיקת Lab	משתמש	VM נמחקת והרשומות מסומנות `Destroyed`	ממומש בקוד; runtime לא אומת
UC-10	ניהול פרופיל	משתמש	פרטים, סיסמה או תמונת data URL מתעדכנים	ממומש ומאומת בקוד
UC-11	ניהול תוכן Academy	מנהל	קורסים, פרקים ומשימות מתוחזקים ונרשמים ב־audit	ממומש ומאומת בקוד
UC-12	ניהול חשבונות	מנהל	status, role וסיסמה מנוהלים עם הגנת self-action	ממומש ומאומת בקוד
UC-13	ניהול Labs פעילים	מנהל	רשימה מסוננת ויכולת termination	ממומש ומאומת בקוד
UC-14	Leaderboard	משתמש	אין תוצאה פעילה; מוצג placeholder מתוכנן בלבד	

4.2 UC-01 — הרשמה

שחקן ומטרה. אורח מבקש ליצור חשבון כדי להשתמש במסלולים המוגנים. התנאים המקדימים הם חיבור ל־PostgreSQL, Backend זמין וכתובת email שאינה קיימת. ה־trigger הוא שליחת טופס Register.

מהלך עיקרי. (1) `Register.jsx` אוסף `user_name`, `email` ו־`password`. (2) `authService.register` שולח `POST /api/auth/register` כ־JSON. (3) `auth_handler::register` אין middleware מחלץ את הגוף; (4) מפני שזה נתיב ציבורי. `auth_service::register` מפעיל bcrypt עם `DEFAULT_COST`. (5) `user_repo::create_user` מכניס row ל־`users`, עם role קשיח `'user'`. (6) מסד הנתונים יוצר UUID ו־timestamps ומאפך `UserResponse` מומר ל־Entity הייחודי. (7) ה־email שאינו מכיל `password_hash`. (8) השרת מחזיר `201 Created`; ה־Frontend מציג הצלחה ומפנה להתחברות. אין JWT ואין auto-login בתשובת ההרשמה.

עם גוף `Bad Request` ממופים כיום ל־`400` כפול repository או כל שגיאת email. **חלופות ושגיאות.** לבין תקלה פנימית. לנתיב אין validation טקסטואלי, בלי הבחנה יציבה בין password policy, normalization של email, rate limit או email verification. Postcondition מוצלח: row חדש ופעיל עם `total_score=0`; Postcondition כושל: לא נוצר session. ראיות. ציבורי. הרשאה: `routes/auth.rs`, `handlers/auth_handler.rs:7-14`, `services/auth_service.rs:11-19`, `repositories/user_repo.rs:4-40` והמיגרציות של `users`.

4.3 UC-02 — התחברות וכניסה למסלול מוגן

שחקן ומטרה. משתמש רשום ופעיל מבקש Login. וסיסמה ב־ email הוא שליחת trigger־ session. `authService.login` שולח `POST /api/auth/login`; השירות מאתר את ה־User לפי email, משווה bcrypt, קורא שוב `is_active` ויוצר JWT שכולל `sub`, `exp` ו־`role`. `exp` הוא זמן נוכחי ועוד 86,400 שניות. התשובה היא `200 OK` עם `{token,user}`.

`AuthContext.login` שומר את שני הערכים ב־state וב־`localStorage`. ניווט ל־`/dashboard` עובר דרך `RequireAuth`, שבודק רק אם קיים token; זהו guard של UI, לא אימות קריפטוגרפי. הקריאה הראשונה ל־API עוברת דרך `auth_middleware`, שמאמת חתימה ותוקף, מפענח את `sub`, קורא `role,is_active` מה־DB ודורס את role שב־token. handler מקבל `Claims` רק לאחר מכן. token חסר, פג או פגום מקבל `401`; חשבון מושבת מקבל `403` מעודכנים.

גם היא disabled לא קיים וסיסמה שגויה מוחזרים שניהם כ־`401`, אך הודעת email מסלולים חלופיים: מחזיר `403` לחשבון שהושבת לאחר שכבר login, בעוד middleware מוחזרת כ־`401` בזמן handler דרך קיבל token. Postcondition: token נשמר בדפדפן וה־Backend עדיין נשאר מקור האמת להרשאות. ראיות: `auth_service.rs:21-78`, `AuthContext.jsx:20-58`, `RequireAuth.jsx`, `auth_middleware.rs`.

4.4 UC-04 — פתיחת קורס והתחלת משימה

המשתמש בוחר קורס ב־`CoursesPage`. `CoursePage` מקבל `courseId` מה־router, מפעיל את `useCourse` ואת `useCourseProgress` לתוכן המלא ולמצב האישי. הקריאה הראשונה היא ציבורית מבחינת Backend: `GET /api/academy/courses/:id/full`; השנייה מוגנת: `GET /api/task-progress/courses/:course_id`. ה־Academy service קורא course, sections ו־tasks, ואילו progress repository מבצע `LEFT JOIN` ל־`user_task_progress` וממלא `NOT_STARTED` כאשר אין row.

השירות גוזר AVAILABLE, tasks.order_index ואז sections.order_index לאחר מיון לפי ומונע task ID לפי map בונה Frontendלכל הבאות. ה־ LOCKEDלמשימה הראשונה שטרם הושלמה ו־ click על locked. בחירה במשימה NOT_STARTED שולחת POST /tasks/:task_id/start. השרת אינו עוקף את המדיניות DOM חוזר על בדיקת הנעילה; לכן שינוי repository מבצע upsert למצב קודם. לאחר completed שכבר task ואינו מחזיר started_at, שומר IN_PROGRESSל־ מתעדכן Sidebarמחדש וה־ progress טוען hookה־ response.

שגיאות: course או task 404 — חסרים; JWT 401 — חסר; task כשל 403 — נעולה DB — 500. Postcondition: יש row progress או שהמצב הקודם נשמר. ראיות: CoursePage.jsx:13-137, academyService.js, task_progress_service.rs, task_progress_repo.rs.

4.5 UC-05 — השלמת תוכן ופתיחת המשימה הבאה

המשתמש לוחץ Complete. ה־layout מפעיל completeContentTask, בLESSON או PRACTICE ששולח POST /api/task-progress/tasks/:task_id/complete. middleware קובע user ID; handler אינו מקבל user ID מהלקוח. service טוען את הקורס, מאתר progress, דוחה task מסוג LAB, בודק access ורק אז מבצע upsert לCOMPLETED.

פעם אחת ומעניק completed_at אם כבר היה קיים, קובע started_at שומר SQLה־ tasks.points. אם task כבר completed, earned_points הקיים נשמר — פעולה חוזרת אינה מוסיפה נקודות. response הוא snapshot מלא של progress. CoursePage.handleTaskCompleted מרענן, מאתר את המשימה הזמינה הבאה, מסמן אותה ובוחר אותה. אם הקורס הושלם, המשימה האחרונה נשארת מוצגת. שגיאות: ניסיון להשלים IN_PROGRESS LAB 404 — מזהה חסר; 403 — נעילה; 400 — בנתיב זה.

4.6 UC-06 — יצירת Lab

התנאים: JWT תקף, task LAB נגישה, scenario פעיל, template רשום, ואין למשתמש environment פעיל באחד הסטטוסים Building, Running או Stopping. ה־UI שולח POST /api/lab/create עם scenario_id. handler מפיק user ID מה־Claims וקורא create_user_lab.

הוא מנסה למחוק אותו; אחרת מוחזרת expired, פעיל גלובלי. אם הוא environment השירות בודק שגיאת policy. לאחר אימות scenario הוא מכניס environment=Building, בוחר שם VM ייחודי ופורט ephemeral, ומכניס instance=Starting. ב־spawn_blocking מתבצעים clone, הגדרת NAT,

ואז start headless והמתנה עד 120 שניות ל-TCP. רק לאחר הצלחה instance ו-environment מסומנים כ-`Running`. response `201` מכיל `env_id`, `ssh_port`, expiry והודעה.

מסלולי כשל מפורטים בפרק 14: template חסר, unique conflict, port forwarding כושל, timeout SSH, כשל עדכון DB אחרי start וכשל cleanup. הרשאה ובעלות נאכפות בשרת; index חלקי ייחודי הוא קו הגנה שני נגד מעבדות כפולות. היכולת ממומשת בקוד אך לא הופעלה בביקורת זו.

4.7 UC-07 ו-UC-08 — טרמינל והגשת דגל

לאחר Lab Running, `TerminalWrapper` בונה URL מסוג `ws://…/terminal/:environment_id?token=…`. לפני upgrade השרת מאמת token, חשבון פעיל, ownership, environment/instance Running ופורט תקין. לאחר `101 Switching Protocols` שני ערוצי mpsc מגשרים בין WebSocket async ל-ssh2 blocking. input מהדפדפן נכתב ל-PTY; output מה-SSH מוחזר כ-Text frames. disconnect של אחד הצדדים מבטל את tasks האחרים.

הגשת דגל היא בקשת REST נפרדת. `verify_and_submit_flag` בודק בעלות environment ומוודא שה-task היא `LAB` ומקושרת לאותו scenario, מרענן expiry, Running ובמצב המשתמש שאינה locked. רק אז מתבצעת השוואה מדויקת של flag trimmed. דגל נכון מפעיל transaction: `user_flags` נכתב עם `ON CONFLICT DO NOTHING`, ו-progress מושלם עם `tasks.points`. `flags.points` אינו משמש להענקה בפועל. דגל שגוי מוחזר כ-`200` עם הודעה; שגיאות policy מוחזרות `400`.

4.8 UC-09 — מחיקת Lab

המשתמש מפעיל Delete/Stop, וה-Frontend שולח `POST /api/lab/delete` עם `env_id`. השירות מבצע query המשלב environment ID ו-user ID, ולכן משתמש אחר מקבל `404` ולא יכול למחוק. environment שכבר `Destroyed` נחשב הצלחה idempotent. environment אחרת מסומן `Stopping`; instance מסומנת גם היא `Stopping` אם קיימת. פעולת `VBoxManage unregistervm --delete` רצה ב-`spawn_blocking`. בהצלחה instance ו-environment עוברים ל-`Destroyed`, timestamps ו-expiry מתעדכנים. בכשל הם עוברים ל-`Failed` מנוקה ככל האפשר, ו-`500` חוזר ללקוח.

4.9 UC-10 — ניהול פרופיל וסיסמה

`ProfilePage` טוען `GET /api/profile/`. כל endpoints משתמשים ב־`sub` מה־Claims; אין user ID ב־URL, ולכן משתמש אינו בוחר יעד. עדכון פרטים חותך whitespace, ממיר email ל־lowercase, מאמת שם באורך 3–50 ופורמט email, ובודק בעלות email לפני UPDATE. unique constraint הוא קו הגנה נוסף. ה־hook מעדכן את AuthContext ואת ה־user ב־localStorage כך שה־Navbar משתנה מיד.

שינוי סיסמה בודק התאמת new/confirm, מינימום שמונה תווים, סיסמה נוכחית נכונה ושוני מהסיסמה הישנה. רק אז נוצר bcrypt hash חדש. current password שגוי מחזירה `401`; validation מחזיר `400`. Avatar הוא data URL מסוג PNG/JPEG/WebP, לא URL חיצוני ולא upload קבצים לשרת; השירות מגביל את המחרוזת ל־2.8 מיליון תווים ומציג הודעת “קטן מ־2 MB”.

4.10 UC-11 עד UC-13 — פעולות מנהל

כל בקשת Admin עוברת תחילה Authentication ואז Authorization. `auth_middleware` טוען role עדכני; `admin_middleware` דורש `Role::Admin`. CRUD Academy משתמש ב־DTOs ייעודיים ומפעיל `record_activity` לאחר mutation. ניהול משתמשים מוסיף הגנות: מנהל אינו יכול להשבית את עצמו או להוריד לעצמו role, וערכי role מוגבלים. איפוס סיסמה מבצע validation ו־bcrypt.

ניהול Labs קורא רשימה paginated ומסוננת. Termination מאתר את owner של environment ומעביר אותו ל־`delete_user_lab`, ולכן אין עקיפה של ownership בתוך service. ניהול flags מציג list ממוסך; endpoint details מחזיר ערך מלא רק למנהל. פעילות נכתבת ל־`admin_activity_logs` עם actor, action, entity ו־JSON details. כשל mutation אינו אמור להירשם כהצלחה. ראיות מפורטות: `admin_routes.rs`, `admin_handler.rs`, `admin_service.rs`, `admin_repo.rs` ו־`features/admin`.

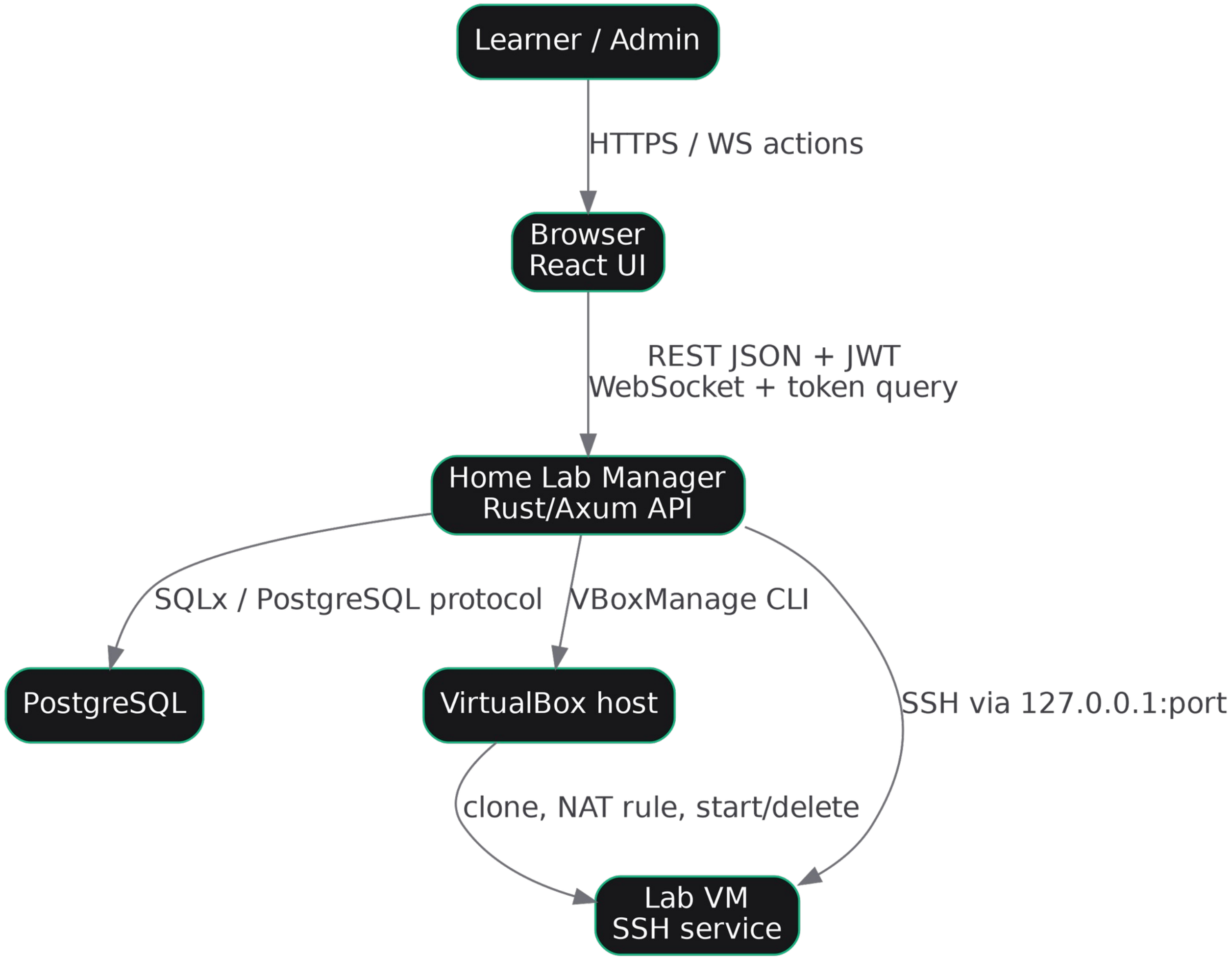
5. סקירת הטכנולוגיות

תחום	טכנולוגיה וגרסה בקבצים	תפקיד והתאמה
Backend	Rust edition 2024; package `0.1.0`	טיפוסים חזקים ובטיחות זיכרון בתהליך תשתיתי ארוך־חיים
Web	Axum `0.7` עם `ws`, `macros`	Router, extractors, middleware ו־WebSocket באותו runtime

תחום	טכנולוגיה וגרסה בקבצים	תפקיד והתאמה
Async	Tokio `1.52.3` full	שרת, cleanup ticker ־tasks; blocking קוד מבודד ב־`spawn_blocking`
DB	PostgreSQL + SQLx `0.8.6`	constraints יחסיים, migrations ומאקרו־queries מאומת מול schema
Auth	bcrypt `0.19.1`; jsonwebtoken `10.4`	password hashing ו־JWT חתום עם expiry
SSH	ssh2 `0.9.4`	session, password auth, PTY ו־shell אינטראקטיבי
Infra	`VBoxManage` CLI	clone/start/NAT/delete של VMs בלי SDK נוסף
Frontend	React/React DOM `^19.2.6`	UI רכיבי ו־state hooks
Routing	react-router-dom `^7.18.0`	routes, layouts מוגנים ו־Admin nested routes
Build	Vite `^8.0.12`	dev proxy ו־production bundling
Styling	Tailwind CSS `^4.3.1`	utility classes; palette בפועל בעיקר Zinc/Red, ובתרשימי סטטוס גם Emerald
Terminal UI	xterm `^5.3.0`, fit addon `^0.11.0`	terminal emulation והתאמת dimensions
Quality	Cargo fmt/test; ESLint `^10.3.0`; Vite build	בדיקות קומפילציה, lint ו־bundle

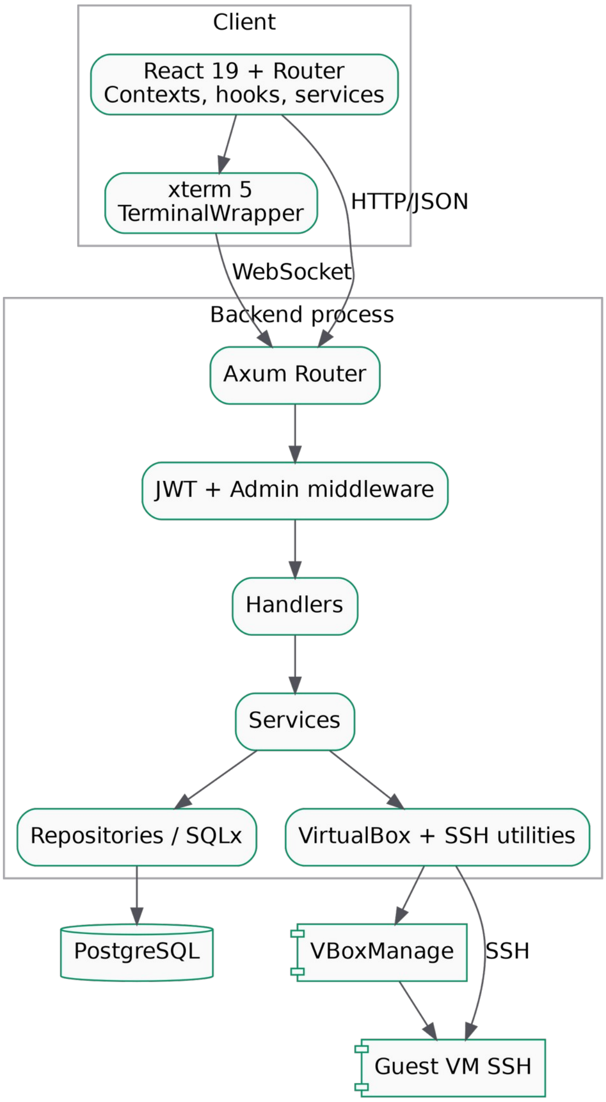
הגרסאות לעיל נלקחו רק מ־`backend/Cargo.toml` ומ־`frontend/package.json`. `Cargo.lock` ו־`package-lock.json` מקבעים בפועל גרסאות טרנזיטיביות, אך הספר אינו מנחש מהן API behavior.

6. ארכיטקטורת־על



System Context — תרשים 1

הדפדפן מתקשר ב־REST JSON עם `/api/*`. ב־REST מוגן ה־JWT נשלח ב־`Authorization: Bearer`. עבור terminal הוא נשלח ב־query parameter משום שממשק WebSocket בדפדפן אינו מאפשר header מותאם בעת handshake. ה־Backend מתקשר עם PostgreSQL בפרוטוקול DB דרך SQLx, עם VirtualBox דרך תהליך `VBoxManage`, ועם ה־guest דרך SSH אל `127.0.0.1:<allocated-port>`.



תרשים 2 — ארכיטקטורת Containers

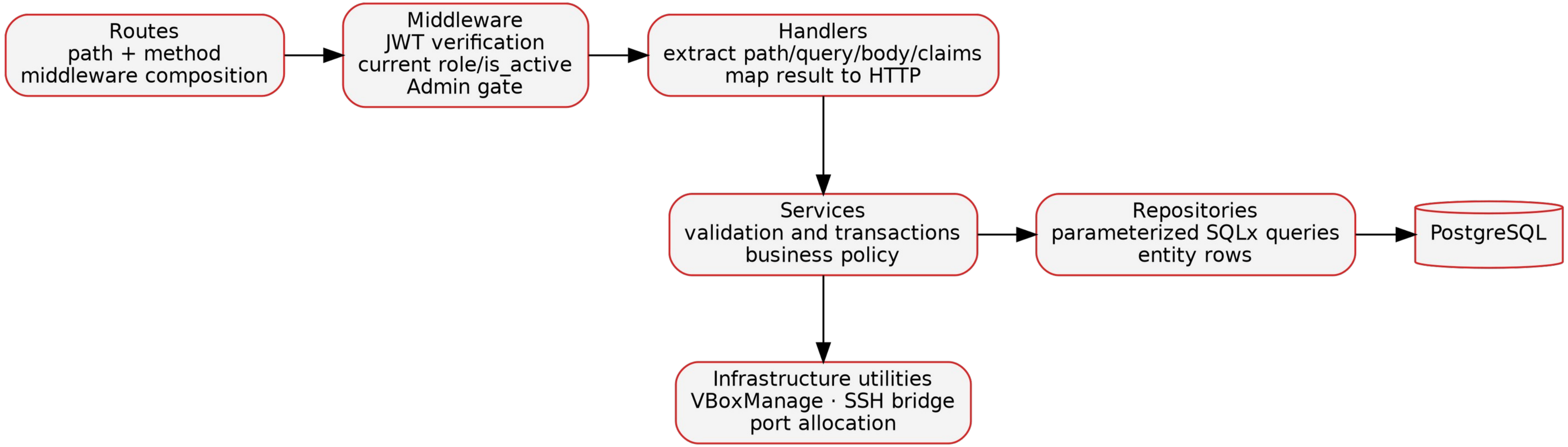
גבול האחריות של Frontend הוא הצגה, ניווט ו־client state. כל החלטת אבטחה, בעלות, נעילה וניקוד מתקבלת ב־Backend. PostgreSQL הוא מקור האמת למשתמש, התקדמות ומצב לוגי של Labs; VirtualBox הוא מקור המצב הפיזי של ה־VM. אין כיום reconciliation מלא בין השניים לאחר restart, ולכן עלולה להיווצר סטייה.

7. מבנה הריפו

```
home-lab-manager/
├── backend/
│   ├── Cargo.toml, Cargo.lock
│   ├── migrations/                # כרונולוגיות migrations עשר
│   └── src/
│       ├── main.rs                # bootstrap, DB pool, cleanup, CORS, listener
│       ├── routes/                # method/path ומידלוור
│       ├── handlers/              # Axum extraction ו־HTTP mapping
│       ├── services/              # כללים עסקיים ותזמור
│       ├── repositories/          # SQLx queries
│       ├── models/                # entities, DTOs, statuses, auth models
│       ├── middleware/            # JWT/current account ו־Admin
│       ├── utils/                 # VirtualBox, SSH, YouTube, port, monitor
│       └── errors/                # AppError → HTTP/JSON
├── frontend/
│   ├── package.json, package-lock.json
│   ├── vite.config.js
│   └── src/
│       ├── App.jsx                # route tree
│       ├── context/               # AuthContext
│       ├── routes/                # RequireAuth/RequireAdmin
│       ├── layouts/               # public/app shells
│       ├── features/              # academy/admin/auth/ctf/dashboard/labs/profile
│       ├── shared/ui/             # Button, Card, Input
│       └── config/api.js          # REST/WS origins
└── docs/
    ├── *.md                       # תיעוד קיים, משני לקוד
    └── project-book/              # תוצרי ספר זה
```

תחת המאגר `AGENTS.md` הושמטו במכוון. לא נמצא build artifacts ו־`node_modules`, `target`, `dist` או ההורה שלו בזמן הבדיקה.

8. ארכיטקטורת ה־Backend



תרשים 3

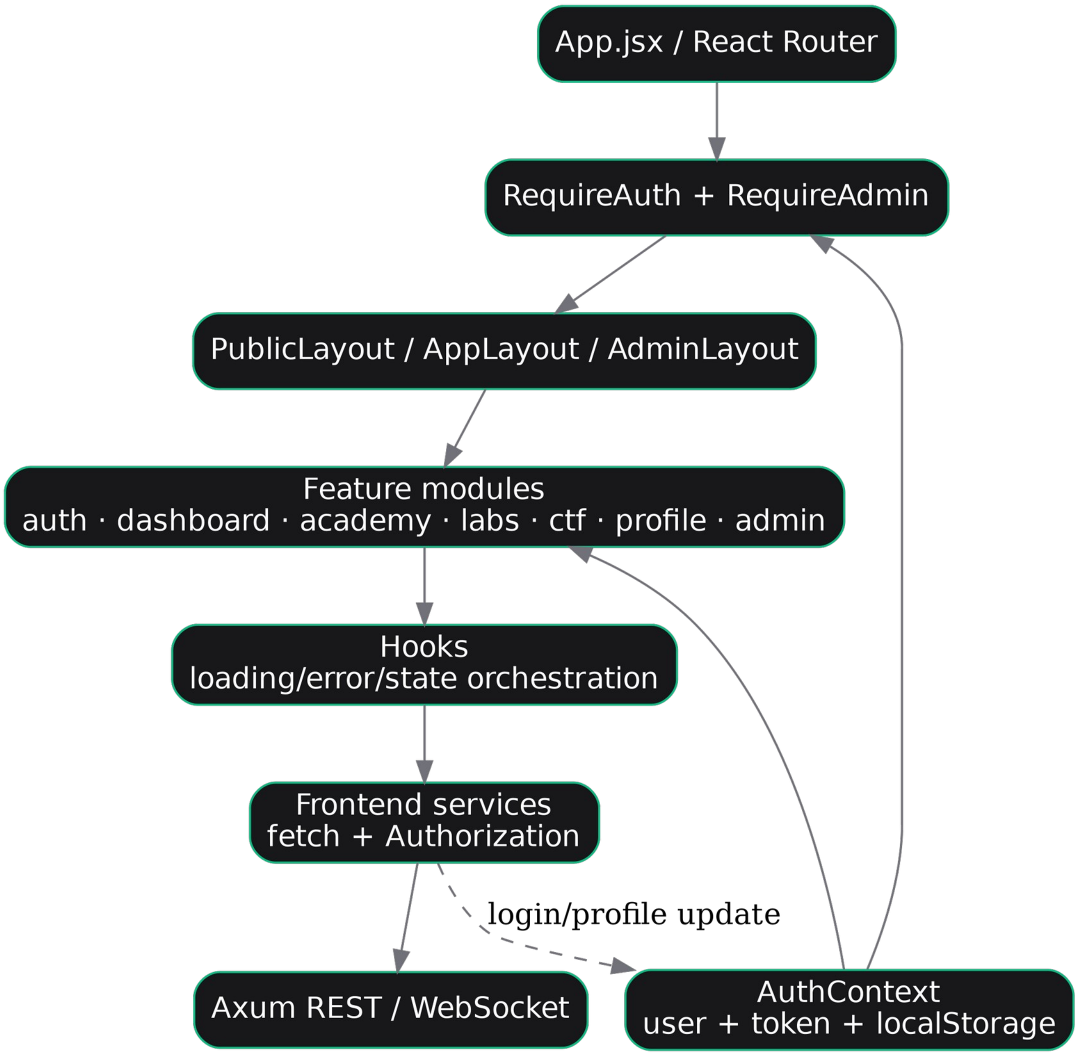
— שכבות Backend

בקשה נכנסת ל־Router. בנתיב מוגן, `auth_middleware` מאמת חתימה ו־expiry, מפענח `sub` ל־UUID, קורא `role,is_active` עדכניים ומוסיף `Claims` ל־request extensions. בנתיב Admin, `admin_middleware` בודק את ה־role המעודכן. Handler מחלץ `State`, `Path`, `Query`, `Json` ו־`Extension<Claims>`, ואז קורא service. Service מבצע validation, policy ותזמור. Repository מריץ SQL פרמטרי. הזרימה ניכרת למשל ב־`POST /api/lab/create`: `lab_routes.rs:14` → `lab_handler::handle_create_lab` → `instance_service::create_user_lab` → environment/instance/scenario repositories ו־VirtualBox utilities.

`AppError` מרכז תשובות 400/401/403/404/500 ומסתיר פרטי SQL מהלקוח. חלק ישן יותר של Lab/Auth עדיין משתמש ב־`Result<_, String>` וממפה שגיאות לפי תוכן מחרוזת; זה פחות יציב ואחיד. Entities מייצגים rows מלאים; DTOs מונעים בדרך כלל חשיפת `password_hash`. `UserResponse` אינו כולל hash, timestamps או `is_active`; Profile/Admin משתמשים ב־DTOs ייעודיים.

גישה ל־DB נעשית ברובה דרך repositories. החריג הבולט הוא query ישיר ב־`auth_middleware.rs`, שנועד לקבל מצב חשבון עדכני בכל request. מאקרו־SQLx דורשים schema נגיש בזמן compile; זו הסיבה ש־`cargo test` נזקק ל־PostgreSQL המקומי.

9. ארכיטקטורת ה־Frontend



תרשים 4 —

מבנה Frontend

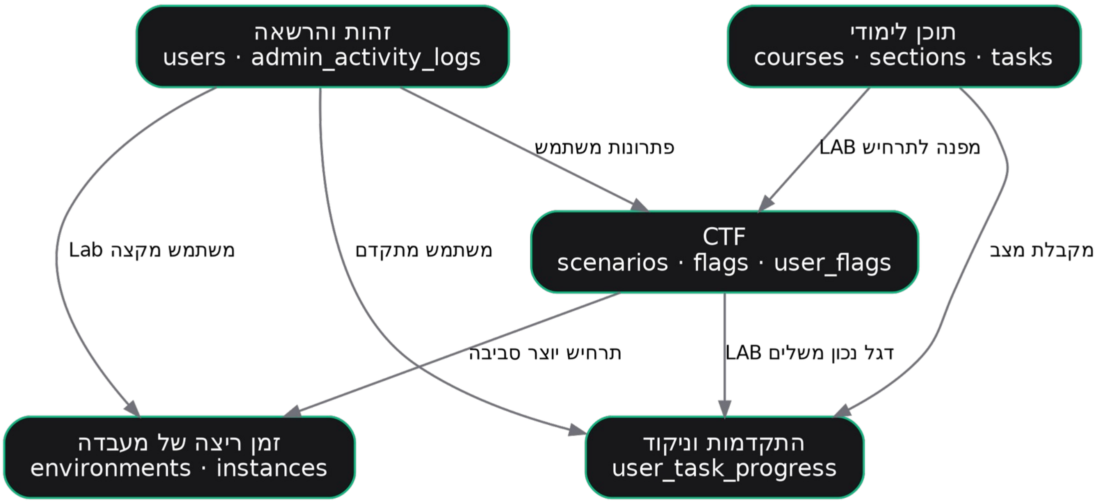
App.jsx מגדיר שלושה מסלולים ציבוריים, layout מוגן למשתמש ומסלול Admin מקונן. RequireAuth בודק קיום token בלבד; RequireAdmin בודק role מתוך context/localStorage. זוהי הגנת UX בלבד — ה־Backend הוא גבול האבטחה.

כל feature כולל שילוב של pages, components, hooks ו־services. Services קוראים API_BASE_URL, מוסיפים Bearer token וממפים response errors. Hooks מחזיקים loading, error, data ופעולות mutation. AuthContext טוען session מ־localStorage פעם אחת, מספק

login/logout ו־`updateStoredUser` לסנכרון profile. אין state manager חיצוני, query cache או test harness.

Academy משתמש ב־`CoursePage`, `SectionSidebar`, `TaskRenderer` ו־layouts לפי סוג משימה. `VideoWidget` מציג YouTube ID שכבר נורמל בשרת. Labs משתמש ב־`useLabs`, `LabWorkspace` ו־`TerminalWrapper`; Admin בנוי סביב `AdminLayout`, hooks לפי ישות, טבלאות, forms ו־pagination controls. Loading/Error/Empty מיושמים ברמת pages/hooks, אך אינם אחידים לחלוטין בין features.

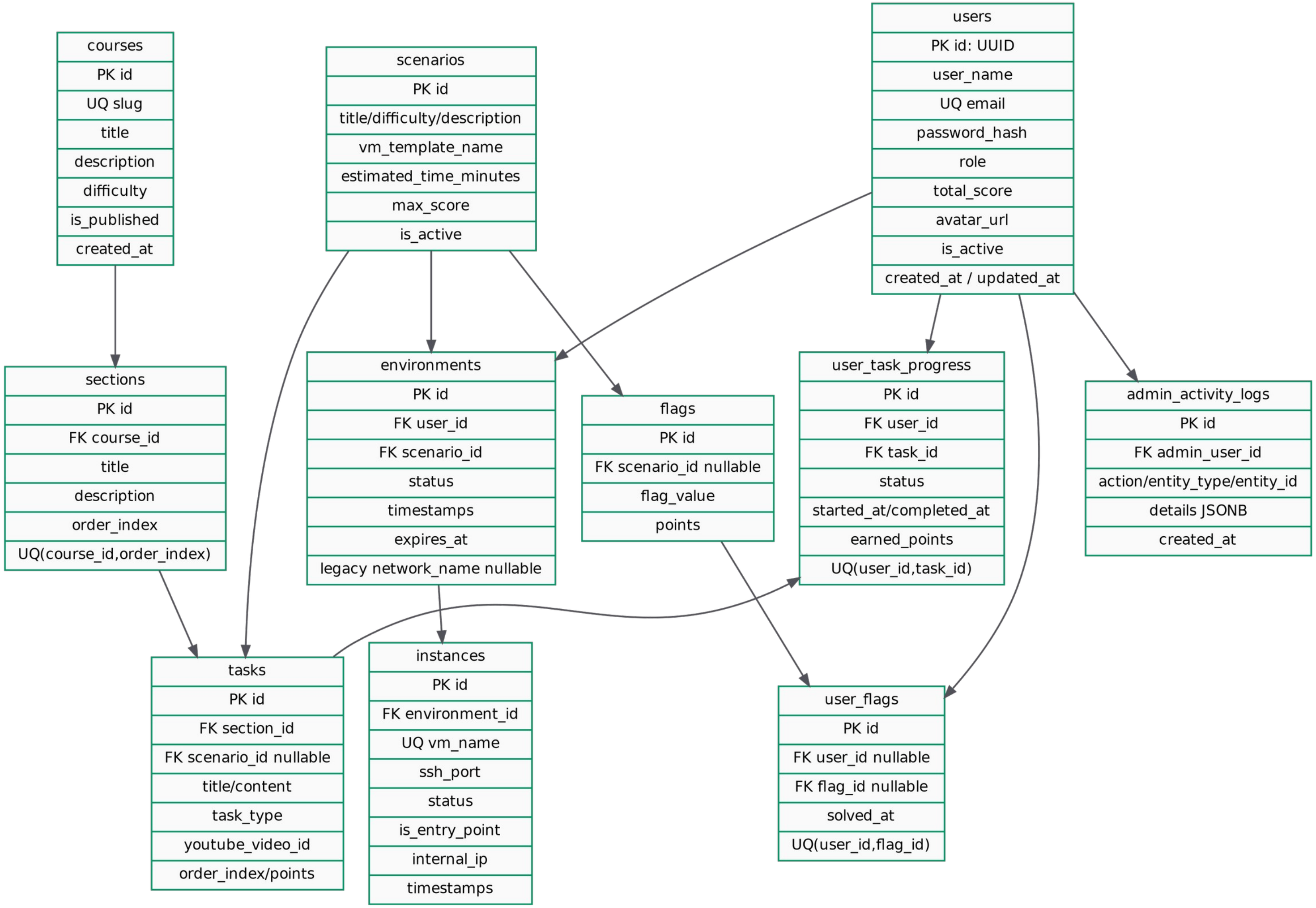
10. מסד הנתונים



תרשים 5 — ERD ברמת תחומים

התרשים הבא הוא גרסת עומק. הוא מוצג בעמוד רוחבי כדי לשמור על שמות הטבלאות והשדות בגודל קריא; מילון הנתונים שאחריו משלים טיפוסים, אילוצים ומשמעות עסקית.

Home Lab Manager — ספר פרויקט



תרשים 6 — ERD מפורט

הקוד אינו מפעיל migrations. הסכמה הסופית מתקבלת מהחלה סדרתית של עשר migrations ב־PostgreSQL או דרך כלי SQLx CLI ההתקנה חייבת להריץ אותן מראש באמצעות אוטומטיתstartup; כל המפתחות הראשיים הם UUID מתאים עם `gen_random_uuid()`.

10.1 Data Dictionary

טבלה	שדות עיקריים, טיפוסים ואילוצים	משמעות עסקית
users	`id UUID PK`; `user_name varchar(225) NOT NULL`; `email varchar(255) NOT NULL UNIQUE`; `password_hash varchar(255) NOT NULL`; `role varchar(50) NOT NULL default 'user'`; `total_score int NOT NULL default 0`; `avatar_url text NULL`; `is_active bool NOT NULL default true`; timestamps NOT NULL	חשבון, תפקיד, מצב ו־score denormalized
courses	`id UUID PK`; `title NOT NULL`; `slug UNIQUE NOT NULL`; description/difficulty nullable; `is_published bool NOT NULL`; `created_at`	יחידת לימוד עליונה
sections	`id UUID PK`; `course_id FK NOT NULL ON DELETE CASCADE`; title; description nullable; `order_index int`; `UNIQUE(course_id,order_index)`	פרק מסודר בקורס
tasks	`id UUID PK`; `section_id FK CASCADE`; `scenario_id FK NULL`; title/content; `task_type`; `youtube_video_id varchar(11) NULL`; order/points; `UNIQUE(section_id,order_index)`; checks על type/video	משימת LESSON/PRACTICE/LAB
user_task_progress	`id UUID PK`; user/task FKs CASCADE; `status`; timestamps nullable; `earned_points int NOT NULL`; `UNIQUE(user_id,task_id)`	מצב שמור; LOCKED אינו נשמר אלא נגזר
scenarios	`id UUID PK`; title; difficulty/description nullable; template, time, score, active NOT NULL לאחר backfill	הגדרת Lab לוגית המצביעה

טבלה	שדות עיקריים, טיפוסים ואילוצים	משמעות עסקית
		ל-VM template
`environments`	`id UUID PK`; user/scenario FKs; status; timestamps; `expires_at`; `network_name` legacy nullable	הקצאת Lab למשתמש; index חלקי אוכף active אחד
`instances`	`id UUID PK`; environment FK CASCADE; `vm_name`; entry flag; internal IP; ssh port/status/timestamps; unique vm_name; unique active ssh_port	מכונה פיזית בתוך environment; הקוד הנוכחי יוצר אחת
`flags`	`id UUID PK`; `scenario_id UUID NULL FK CASCADE`; `flag_value NOT NULL`; `points NOT NULL`	תשובת CTF; בפועל service Admin דורש scenario
`user_flags`	`id UUID PK`; user/flag FKs nullable לפי migration; solved_at; unique pair	היסטוריית פתרון דגל; nullable הוא פער schema
`admin_activity_log`	`id UUID PK`; `admin_user_id FK NOT NULL`; action/entity; entity_id NULL; details JSONB NULL; created_at	audit של פעולות Admin

טבלה s	שדות עיקריים, טיפוסים ואילוצים	משמעות עסקית

10.2 קשרים, indexes וכללים

מחיקת course גוררת sections/tasks/progress; מחיקת scenario גוררת environments ו־flags, אך משימות עם FK ברירת מחדל חוסמות מחיקה אם הן עדיין מפנות לתרגיל — והשירות מתרגם זאת ל־validation. אינדקסים נוספים קיימים על activity time/admin/entity, `users.is_active`, environment user/scenario/status/expiry, instance environment/status. `idx_environments_one_active_lab_per_user` הוא unique partial index לסטטוסים `Building`, `Running`, `Stopping`.

ה־migration הראשון היה NOT NULL `network_name`: יש אי־התאמה היסטורית מתועדת בקוד עצמו של Labs, אך migration `complete_runtime_schema` הופך אותו nullable כי repository הנוכחי אינו כותב אותו. גם ה־migration האחרון, אך `QUIZ/READING/VIDEO` מוקדמים מציינים סוגי comments המצב האחרון הוא מקור האמת; מגביל בפועל ל־`LESSON/PRACTICE/LAB`.

11. API

כל הנתיבים מורכבים תחת `/api` ב־`routes/mod.rs`. `JWT` פירושו Bearer token; `Admin` פירושו JWT עדכני role `admin`. תשובות validation חדשות משתמשות בדרך כלל ב־`{message}`, בעוד handlers ישנים של Auth מחזירים `{error}`; לכן ה־Frontend בודק את שניהם.

11.1 Authentication, user, dashboard ו־profile

Method + path	גישה	מטרה	תוצאה מרכזית
POST `/auth/register`	ציבורי	יצירת חשבון user	`201 UserResponse`; קלט/service שגוי `400`
POST `/auth/login`	ציבורי	אימות והנפקת JWT	`200 {token,user}`; כשל או disabled הם `401`
GET `/users/`	Admin	רשימת משתמשים legacy	`200 UserResponse[]`

Method + path	גישה	מטרה	תוצאה מרכזית
GET /dashboard/	JWT	תמונת מצב ללומד	200 DashboardResponseDto
GET /profile/	JWT	קריאת פרופיל עצמי	200 ProfileResponseDto
PUT /profile/	JWT	שינוי שם ו־email	profile; 400/409 validation/conflict־ב
PUT /profile/password	JWT	החלפת סיסמה	message; 400/401
PUT /profile/avatar	JWT	data URL או הסרה	profile; 400/413

שרשרת השכבות היא handler→service→repository בתחום המתאים. שמות הפונקציות, DTOs וכל statuses מופיעים בנספח ז הרוחבי.

11.2 Academy ו־progress

Method + path	גישה	מטרה	תוצאה מרכזית
GET /academy/courses/	ציבורי ב־Backend	קטלוג מפורסם	200 courses
GET /academy/courses/:id/full	ציבורי ב־Backend	עץ course/sections/tasks	200; 404 חסר
GET /task-progress/courses/:course_id	JWT	progress וגישה נגזרת	200; 404 חסר
POST /task-progress/tasks/:task_id/start	JWT	התחלת המשימה הזמינה	200; 403 נעולה
POST /task-progress/tasks/:task_id/complete	JWT	השלמת תוכן	200; LAB ;400 הוא 403 נעולה

Academy route מגן על ה־Frontend, אך שני endpoints ציבוריים עצמם של תוכן מפורסם (academyService.js:15-42), אך השרת אם קיים Authorization הוא בכל זאת שולח ב־Backend. אינו צורך אותו.

11.3 Labs, CTF ו־terminal

Method + path	גישה	מטרה	תוצאה מרכזית
POST `/lab/create`	JWT	יצירת Lab מתרחיש	201 עם env/port/expiry; 400 policy
POST `/lab/delete`	JWT + owner	סיום Lab עצמי	200; 404 חסר
POST `/lab/submit`	JWT + owner	אימות דגל והשלמת LAB	200 גם לדגל שגוי; 400 validation
GET `/lab/status/:environment_id`	JWT + owner	מצב environment/instance	status DTO או null
GET `/lab/active`	JWT	Lab פעיל כלשהו	Lab או null
GET `/lab/active/:scenario_id`	JWT	Lab פעיל לתרחיש	Lab או null
GET→WS `/lab/terminal/:environment_id?token`	query JWT + owner	bridge ל־SSH	101; ייתכנו upgrade לפני 401/403/404/409/500

11.4 Admin Academy

כל הנתיבים הבאים הם תחת `/academy/admin` ודורשים Admin. DTOs נמצאים ב־`models/dto/{course,section,task}.rs`; handlers ב־`academy_admin_handler.rs`; service/repository ב־`academy_service.rs` ו־`academy_repo.rs`.

Method + path	תוצאה/Body	Status מרכזי
GET, POST `/courses`	list all / `CreateCourseRequest` → course	200/201; 400/409/500
PUT, DELETE `/courses/:id`	`UpdateCourseRequest` / no body	200/204; 404/409
POST `/sections`	`CreateSectionRequest` → section	201; 400/409
GET, PUT, DELETE	section / patch / no body	200/204; 404/409

Method + path	תוצאה/Body	Status מרכזי
`/sections/:id`		
GET `/courses/:course_id/sections`	sections ordered	200
POST `/tasks`	כולל `CreateTaskRequest` `video_url`	201; 400 validation
GET, PUT, DELETE `/tasks/:id`	task / patch / no body	200/204; 404/409
GET `/sections/:section_id/tasks`	tasks ordered	200

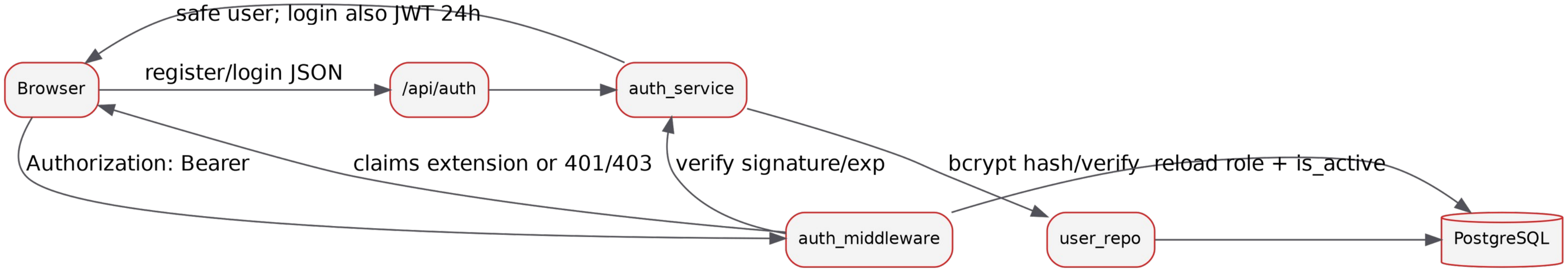
11.5 Admin operations

כל הנתיבים הבאים תחת `/admin`, עם `admin_handler→admin_service→admin_repo` אלא אם מצוין אחרת. list endpoints מקבלים `page`, `page_size` ועד subset של `search,status,scenario_id,admin_user_id,action,entity_type,order`. page size מנורמל ומוגבל בשירות.

Method + path	קלט/פלט	תפקיד וסטטוסים
GET `/dashboard`	statistics + recent activity	overview; 200/500
GET `/users`	query → `PaginatedResponse<AdminUserDto>`	filter/page/חיפוש
GET `/users/:user_id`	details + summary + recent labs	200/404
PUT `/users/:user_id/status`	`{is_active}`	השבתה/הפעלה; self-disable חסום
PUT `/users/:user_id/role`	`{role}`	user/admin בלבד; self-demotion חסום
PUT	new/confirm	bcrypt reset; validation

Method + path	קלט/פלט	תפקיד וסטטוסים
`/users/:user_id/password`		
GET `/labs`	query → paginated labs	filter status/scenario/search
POST `/labs/:environment_id/terminate`	message	קורא `instance_service::delete_user_lab` בשם owner
GET, POST `/flags`	paginated masked list / create	list אינו חושף value; create validates
GET, PUT, DELETE `/flags/:id`	details / update / delete	details חושף value רק ל־Admin
GET `/activity`	query → paginated logs	filters/order
GET, POST `/scenarios`	list / create request	CRUD scenario
PUT, DELETE `/scenarios/:id`	update / delete	deletion נחסמת על references

12. הרשמה, התחברות ואבטחה



תרשים 7

Authentication —

ברישום, השרת מחשב bcrypt עם `DEFAULT_COST`, ואז מכניס role קבוע `'user'`. `UserResponse` נבנה מ־Entity ואינו כולל hash. לא נמצא password complexity validator במסלול registration; זו חולשה. email הוא unique ברמת DB, אך handler Auth מחזיר טקסט שגיאה פנימי יחסית במקום `AppError` אחיד.

בהתחברות, המשתמש מאותר לפי email, bcrypt מאומת ומצב `is_active` נקרא. JWT מכיל `sub`, `exp` ו־`role`, נחתם עם `JWT_SECRET`, ותוקפו 86,400 שניות. ב־REST, ה־role ב־token נדרס ב־role עדכני מה־DB. לפיכך שינוי role והשבתת חשבון נכנסים לתוקף מיד. ב־terminal נבדק `is_active`, אך role אינו רלוונטי.

בצד הלקוח token ו־user נשמרים ב־`localStorage`. יתרון: session שורד refresh; חסרון: XSS יכול לקרוא token. אין refresh token, revoke list, cookie `HttpOnly`, CSRF token או Content Security Policy הנראים בקוד. token נשלח ב־URL של WebSocket, ולכן עלול להופיע בלוגי reverse proxy/history telemetry; יש לצמצם ל־one-time ticket.

12.1 מקור האמת להרשאות

- זהות: `sub` חתום ב־JWT.
- תקפות: חתימה ו־`exp` ב־jsonwebtoken `Validation::default()`.
- role ומצב חשבון: row עדכני ב־`users` בכל REST request.
- בעלות Lab: `WHERE environments.id=$1 AND user_id=$2`.
- נעילת משימה: progress נגזר בסדר section/task.
- Admin: middleware שרץ לאחר Auth ומקבל Claims מעודכנים.

12.2 חוזה ההרשמה וההתחברות

הרשמה מקבלת `user_name`, `email` ו־`password`. השירות בודק שאין משתמש בעל אותו email, מחשב hash, יוצר חשבון פעיל בתפקיד `user` ומחזיר DTO. הצלחה היא `201 Created` בטוח — לא `200` — ו־`400 Bad Request` הוותיק ל־handler ממופות ב־service ובקשת קלט או שגיאה כטקסט. אין במסלול case מפורש, מדיניות חוזק סיסמה או אימות דוא״ל. לכן שני כתיבים שונים מבחינת email זה נרמול עלולים להתנהג לפי collation של והאילוץ PostgreSQL, והמערכת אינה מוכיחה בעלות על הכתובת.

לרבות סיסמה, `auth_service` מאתרת משתמש לפי email ומאמתת bcrypt. כל שגיאת התחברות מאתרת account מבחינת אבטחה, איחוד התשובה מצמצם `{error}` שגויה וחשבון מושבת, הופכת ל־`401` עם

enumeration. מבחינת תפעול, ה־UI אינו יכול להבחין בוודאות בין credentials שגויים להשבתת החשבון. בהצלחה מוחזרים token ופרטי משתמש, והלקוח מעדכן את AuthContext ואת `localStorage` לפני ניווט ל־Dashboard.

12.3 מסלול בקשה מוגנת

בקשת REST מוגנת עוברת שרשרת קבועה: חילוץ כותרת `Authorization`; אימות הסכמה `Bearer`; פענוח JWT; parsing של `sub` ל־UUID; קריאת `role` ו־`is_active` מה־DB; דחיית חשבון שאינו פעיל; והכנסת Claims מעודכנים ל־request extensions. בנתיב Admin רץ אחר כך middleware נוסף, שקורא את ה־Claims המעודכנים ודורש `role == "admin"`. לסדר השכבות יש משמעות: בדיקת Admin אינה סומכת על role שהיה נכון בזמן הנפקת token.

החלטה זו נותנת revocation מעשי ברמת חשבון ללא רשימת ביטול tokens: השבתת משתמש משפיעה בבקשה הבאה. גם קידום או הורדת role משפיעים מיד. המחיר הוא query נוסף לטבלת `users` בכל בקשת REST מוגנת. אם המערכת תגדל, אפשר לשקול cache קצר עם invalidation, אך אין להמהיר את הבדיקה לאמון מלא ב־role שב־JWT בלי לקבל במפורש חלון הרשאות ישנות של עד 24 שעות.

12.4 מפת איומים ובקרות נוכחיות

איום	בקרה קיימת	פער נותר	המלצה
גניבת סיסמה מה־DB	bcrypt עם `DEFAULT_COST`	אין rate limit או policy	rate limiting, policy מדודה וניטור failures
שינוי role בתוך token	חתימת JWT	secret יחיד וללא rotation מתועד	rotation ו־key identifiers
שימוש ב־token של חשבון מושבת	קריאת `is_active` בכל REST request	token עצמו נשאר תקף מחוץ לשרת	revoke/session model לפי צורך
XSS וגישה ל־token	React escaping מסייע	`localStorage` קריא ל־JavaScript	cookie `HttpOnly` או ticket architecture
token בלוגי URL	אימות לפני WebSocket upgrade	query string עשוי להילכד בפרוקסי	one-time terminal ticket קצר־חיים
privilege escalation ב־UI	`RequireAdmin` משפר UX	localStorage ניתן לעריכה	Backend middleware הוא הבקרה הקובעת
שימוש לרעה ב־CORS	אין credentials cookies כרגע	CORS פתוח מרחבי surface	allowlist מפורש לפי סביבת deployment

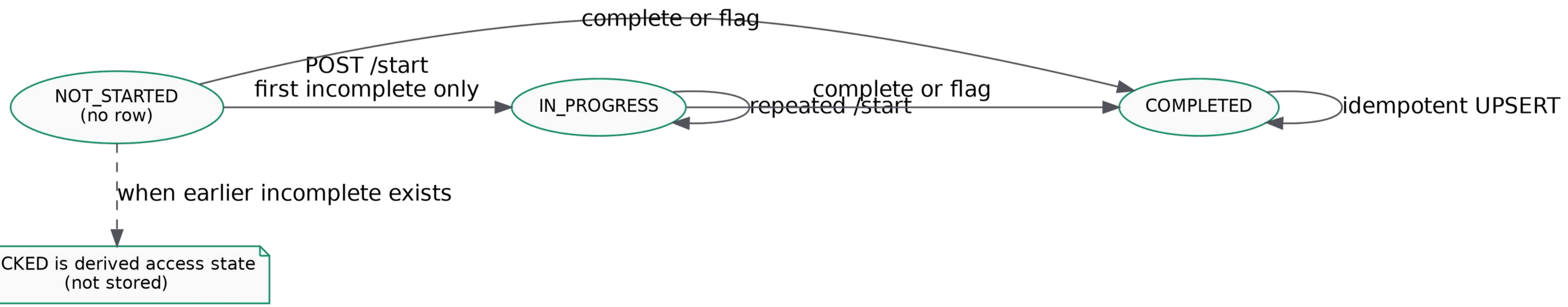
12.5 מאפייני אבטחה שאינם קיימים עדיין

rate ,בדוא״ל, אימות דוא״ל password reset, MFA, שרת logout, refresh tokens לא נמצאו limiting, נעילת חשבון לאחר ניסיונות, CSP מוגדרת, secrets manager או security headers ייעודיים. מקומי ולא לפריסה MVP אין להסיק מכך שהמערכת ״לא מאובטחת״; כן יש להסיק שגבול ההגנה מתאים ל־Backend לפי סביבת ההפעלה, במיוחד משום שה־threat model ציבורית. לפני חשיפה לאינטרנט נדרש למכונות SSH ולפתוח `VBoxManage` יכול להפעיל.

13. מערכת האקדמיה והתקדמות המשתמש

ומגדיר את הסדר parent ייחודי בתוך `order_index` .task־ו course, section היררכיית התוכן היא חייב להיות מקושר `LAB` .`LAB` ,`PRACTICE` ,`LESSON`ומוגבל ל־uppercase מנורמל ל־task type .העסקי והגשת דגל דורשת במפורש התאמה ,validation מטפל ב־service ;כדי להיות שימושי scenario־ל.

LESSON יכול להכיל `video_url` בבקשת Admin. השרת מקבל רק HTTPS YouTube/YouTu.be ושימוש רק format אוכף DB .`youtube_video_id` ושומר רק character ID-נתמכים, מחלץ 11 paths ממוקדות unit בדיקות .ID בטוח מתוך embed בונה Frontend־ה .LESSON־ב עבור YouTube/validation־ב.



תרשים 8

— state machine של התקדמות

NOT_STARTED הוא fallback מחושב כאשר אין row. IN_PROGRESS ו־COMPLETED נשמרים. LOCKED/AVAILABLE אינם ערכי DB בפועל: השירות מסדר את כל משימות הקורס, מסמן completed כזמינות, את הראשונה שאינה completed כזמינה, ואת כל הבאות כנעולות. לכן נעילה חוצה גבולות sections. percentage הוא completed/total ומעוגל לשתי ספרות. total/earned points מחושבים מהמשימות ומה־earned_points, לא רק מ־users.total_score.

השלמת LESSON/PRACTICE מעניקה tasks.points באמצעות upsert. LAB אינה ניתנת להשלמה ב־content endpoint; רק דגל נכון משלים אותה. נקודות הן idempotent: update של task שכבר הושלמה משמר earned_points קיימים.

13.1 אלגוריתם הגישה למשימות

השירות טוען את כל משימות הקורס לפי section.order_index ואז task.order_index, ומצרף את rows של המשתמש. הוא עובר על הרשימה בסדר: משימה שהושלמה מוצגת כנגישה; המשימה הראשונה שטרם הושלמה היא AVAILABLE; וכל משימה אחריה היא LOCKED. אם המשתמש כבר התחיל את המשימה הראשונה, מצבה השמור IN_PROGRESS מוצג לצד הגישה. כך אין צורך לשמור LOCKED בטבלה, ואין סכנה ששינוי סדר ישאיר rows נעולים שאינם תואמים למבנה החדש.

היתרון הוא מודל פשוט ודטרמיניסטי. החיסרון הוא ששינוי סדר של תוכן שפורסם משנה בדיעבד מה זמין למשתמש. מחיקה או הוספה בתחילת קורס עשויה להזיז את נקודת ההמשך. אין versioning של תוכנית לימוד ואין snapshot פר־cohort. לכן עריכת סדר בקורס פעיל היא פעולה עסקית בעלת השפעה, אף שה־API מציג אותה כ־CRUD רגיל.

13.2 נקודות, אחוזים ומקורות אמת

לכל משימה יש points; בעת השלמה הערך מועתק ל־user_task_progress.earned_points. ההעתקה היא בחירה נכונה לשימור היסטוריה: אם מנהל משנה בעתיד את ניקוד המשימה, הניקוד שכבר הוענק אינו משתנה אוטומטית. users.total_score הוא שדה denormalized, אך מסכי progress ו־Dashboard נסמכים על סכימת earned_points במסלולים שנבדקו. יש להגדיר במפורש מי מתחזק את total_score, או להסיר אותו כדי למנוע שני מקורות אמת.

אחוז ההשלמה מחושב ממספר המשימות שהושלמו חלקי מספר המשימות הכולל, ולא לפי משקל נקודות. קורס שבו משימת LAB אחת שווה 100 נקודות ועשר משימות קצרות שוות נקודה עדיין מציג כל משימה כיחידה אחת באחוז. זו התנהגות קוהרנטית למדד “כיסוי תוכן”, אך אינה מדד mastery משוקלל.

13.3 פרסום, עריכה ווידאו

של קורס מלא אינו עטוף endpoint. רשימת הקורסים הציבורית מחזירה רק קורסים מפורסמים ולא על מסלול הrepository, לכן פרטיות התוכן נשענת על שאילתת השירות וה־middleware של Auth בנפרד. האילוצים הייחודיים על tasks ו־sections, course ניתן לערוך Admin בממשק הFrontend. מגינים מכפילויות, אך שינוי מקום בין שני פריטים דורש תכנון כדי לא להתנגש זמנית; לא `order_index` אטומית ייעודית reorder נמצאה פעולה.

מלאות וקצרות YouTube נוח למשתמש. השירות מכיר כתובות URL שולח Adminביצירת שיעור וידאו ה־ HTML אינו מטמיע Frontend מחלץ מזהה בן 11 תווים ושומר רק אותו. בכך ה־ HTTPS, נתמכות, דורש `youtube-nocookie.com`משתמש ב־ player ה־. URLs שהוזן בידי מנהל ואינו צריך לפרש מגוון שמונע צירוף וידאו לסוג אחר check מוסיף DB; השדה מנוקה, `LESSON` במשימה שאינה.

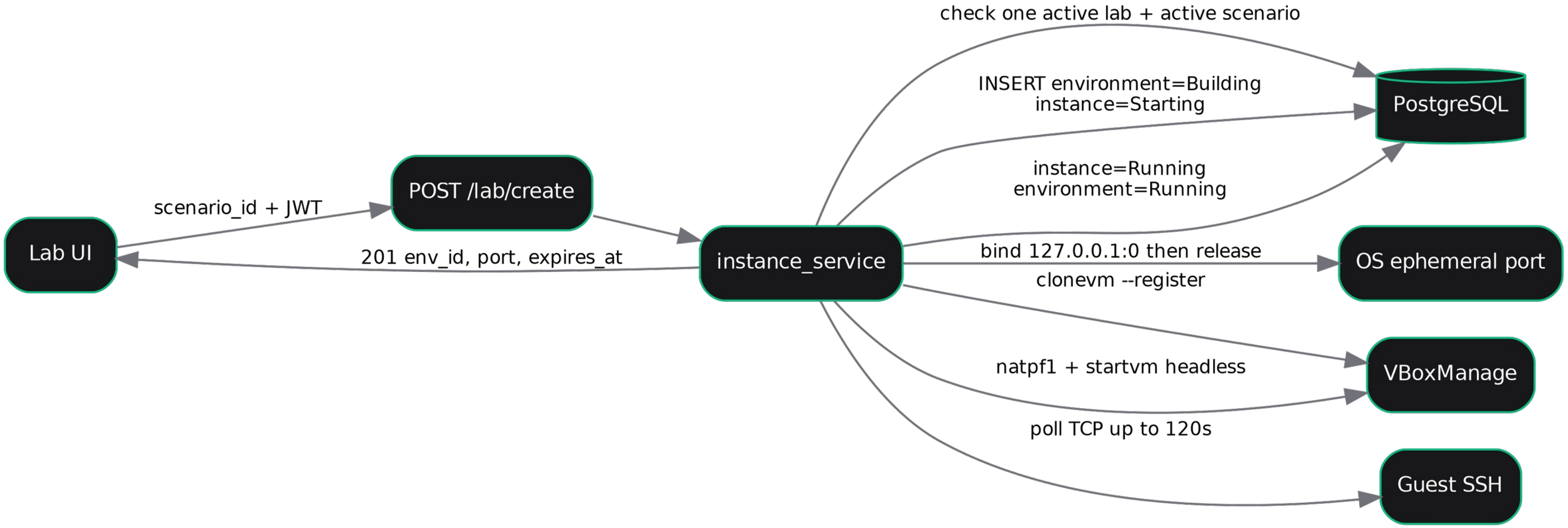
13.4 ביצועים ועקביות ב־Dashboard

טוען פרטי משתמש, קורסים שהתחילו, קורסים זמינים ומצב למידה. עבור כל קורס שהתחיל Dashboard בקנה. N+1 גדל עם מספר הקורסים — דפוס queriesנפרדת, ולכן מספר ה־ progress מתבצעת קריאת או לטעון repository־ב aggregation קטן הוא סביר; בקטלוג גדול כדאי לאחד Home Lab מידה של הראשונה. כך AVAILABLEואם אין — את ה־ ,`IN_PROGRESS` בחירת "המשימה הנוכחית" מעדיפה batch. נוסף בדפדפן state מחזיר את המשתמש לנקודת עבודה סבירה בלי refresh.

13.5 מקרי קצה שיש לבדוק

מקרה	התנהגות מצופה מן הקוד	חשיבות
קורס ללא משימות	0 משימות ואחוז בטוח, ללא חלוקה באפס	מסך empty תקין
התחלת משימה נעולה	`403` progress וללא יצירת	אכיפת סדר בצד שרת
השלמה חוזרת	אין הענקת נקודות כפולה	idempotency
השלמת LAB דרך content API	`400`	דגל הוא שער ההשלמה היחיד
שינוי סדר לאחר progress	הגישה נגזרת מחדש לפי הסדר החדש	סיכון עריכת תוכן חי
מחיקת task	progress נמחק ב־cascade	היסטוריית למידה אובדת

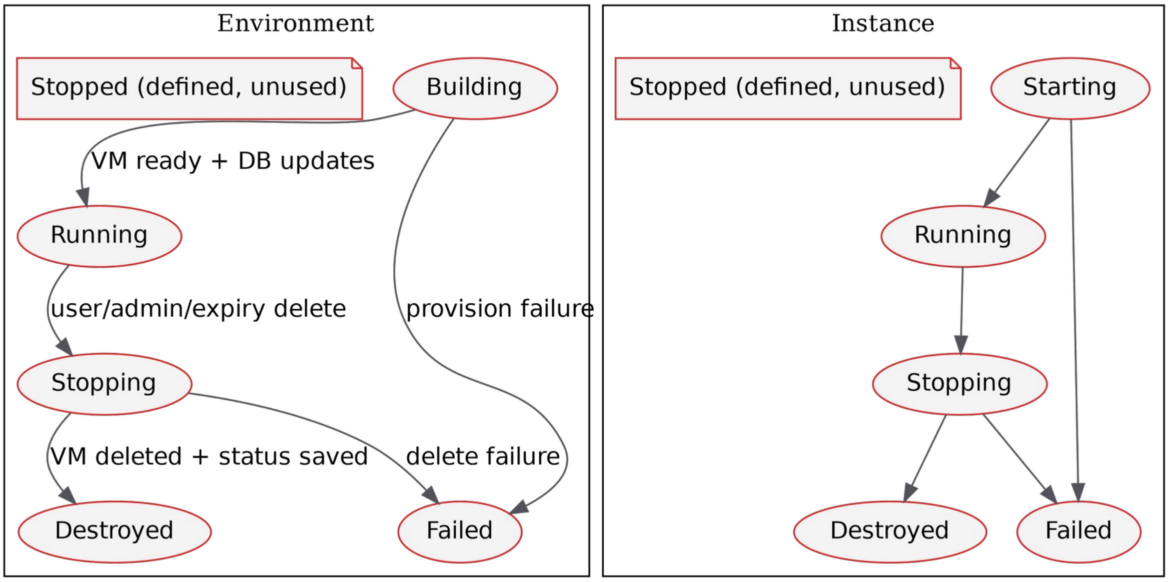
14. מערכת המעבדות



תרשים 9 — יצירת מעבדה

ראשית נבדקת מעבדה פעילה: “instance ואז clone” הסדר האמיתי שונה מעט מתיאור נפוץ של־scenario; לאחר מכן נוצר `environment=Building`; מוקצים שם VM ופורט; נוצר `instance=Starting`; ורק אז מבוצעים clone/start/wait. הקצאת פורט נעשית על ידי bind ל־`127.0.0.1:0` ושחרור מיידי, לכן קיים חלון race עד ש־VirtualBox מגדיר את הפורט. unique partial index מונע שימוש כפול בפורט בין instances במצבי Starting/Running, אך אינו מונע מתהליך חיצוני לתפוס אותו.

VirtualBox manager מוודא template קיים, מריץ `clonevm --mode all --name … --register`, מגדיר `natpf1` מ־host port ל־guest 22, מפעיל headless וממתין לפתיחת TCP עד 120 שניות. לאחר הצלחה ה־environment וה־instance מסומנים Running. בכישלון שניהם מסומנים Failed ונעשה ניסיון למחוק VM.



תרשים 10 — מצבי environment ו־instance

מחיקה היא idempotent אם environment כבר `Destroyed`. אחרת environment ו־instance עוברים `Stopping`, ה־VM נמחק עם `unregistervm --delete`, ואז הרשומות מסומנות `Destroyed` ואינן נמחקות מה־DB. environment ללא instance יתום נסגר ישירות. כשל delete מסומן `Failed`.

מרענן terminal חיובי. קלט `LAB_IDLE_TIMEOUT_MINUTES` ברירת המחדל הוא 20 דקות או expiry. כל 60 שניות שולף background loop עד 10 Labs ומפעיל delete. flag submission מרענן פעילות לכל היותר פעם בדקה; גם אין locking של batch, reconciliation לאחר crash או distributed coordination, ולכן כמה Backend replicas אינם נתמכים בבטחה.

14.1 תרחישי הצלחה וכישלון

שלב	הצלחה	כשל והתנהגות
policy	אין Lab פעיל	conflict אם יש; expired נמחק תחילה
scenario	active scenario קיים	400 אם חסר/לא פעיל
DB creation	environment + instance	environment Failed אם instance insert נכשל
clone/start/SSH	VM מוכנה עד 120s	statuses Failed + best-effort VM delete
status finalization	שני הסטטוסים Running	cleanup אם עדכון DB נכשל אחרי start
deletion	VM וסטטוסים Destroyed	Failed אם VBox delete נכשל; DB may lag אם final update נכשל

14.2 אינווריאנטים של מחזור החיים

לכל environment המערכת מנסה לשמור חמישה כללים: למשתמש לכל היותר instance פעיל אחד; לכל environment פעיל port ייחודי; environment Running צריך instance Running; environment שייך למשתמש. שני הכללים הראשונים מקבלים סיוע. `vm_name` ניתנת לאיתור לפי VirtualBox אחד; ומכונת scenario־ו־DB החמישי חוצה את גבול ה־ .FKs מאינדקסים חלקיים. השלישי והרביעי נאכפים בעיקר בשירות וב־ constraint אינו יכול לקבל אמיתי.

הפער בין DB לבין VirtualBox הוא מרכז הסיכון. לדוגמה, VM יכולה לעלות בהצלחה ותהליך השרת לקרוס לפני סימון Running. במקרה כזה ה־DB נשאר Starting בעוד משאב פיזי פעיל. בכיוון ההפוך, משתמש יכול למחוק VM ידנית ב־VirtualBox וה־DB ימשיך להציג Running. cleanup לפי expiry מטפל בחלק מהמקרים בלבד; הוא אינו inventory reconciliation המשווה את שתי המערכות.

14.3 Provisioning לפי ציר זמן

1. ה־handler מפענח Claims ו־`scenario_id` ומעבירם לשירות.
2. השירות בודק environment פעיל. אם הוא expired, הוא מנסה למחוק אותו לפני המשך; אם הוא עדיין תקף, הבקשה נדחית.
3. scenario נטען ונדרש להיות active. `vm_template_name` למשאב FK הוא שם חיצוני ולא VirtualBox.

4. נוצרת רשומת environment ב־Building עם `expires_at`; מוקצים שם VM ופורט host.
5. נוצרת instance ב־Starting. מכאן כבר קיימת זהות DB לכל משאב עתידי.
6. `VBoxManage clonevm`, הגדרת NAT, הפעלה headless והמתנת TCP מתבצעים באופן סינכרוני מבחינת בקשת HTTP, עד 120 שניות.
7. בהצלחה instance ו־environment הופכים Running והתשובה היא `201`. בכשל הם מסומנים Failed ומתבצע cleanup מיטבי.

פירוש שהטרמינל אמור `201` זמין מפשטת את ה־Frontend: העובדה שהבקשה ממתינה עד שה־SSH ארוך, רגיש ל־timeouts של reverse proxy ומוגבל ביכולת scale. המחיר הוא request להיות בר־חיבור. מודל עתידי יציב יותר יחזיר `202 Accepted`, יפעיל job ויאפשר מתמשך polling או server events עד Running.

הקצאת פורט ובידוד 14.4

ה־Backend מבקש ממערכת ההפעלה port באמצעות bind פנוי ל־`127.0.0.1:0`, קורא את המספר ומשחרר את socket. אין reservation לאחר השחרור עד ש־VirtualBox מתקין NAT forwarding. אך אינו מונע תהליך אחר במארח. האינדקס הייחודי מונע שתי instances פעילות באותו port בתוך DB, מקומי שמחזיק proxy ונסיונות חוזרים, או reservations עם טבלת allocator פתרון חזק יותר הוא listeners ומנתב לפי מזהה.

המיפוי הוא מ־loopback host אל port 22 של כל interfaces, באורח. זו בחירה טובה יותר מחשיפה על כל host/network רץ באותו Backend הצפויה וה־host מכאן מקבל את כתובת ה־`VBoxManage` עוד namespace. אין בקוד יצירת host-only network או firewall פר־Lab; השדה `network_name` הוא legacy nullable. לכן טענות על בידוד רשת מלא אינן נתמכות.

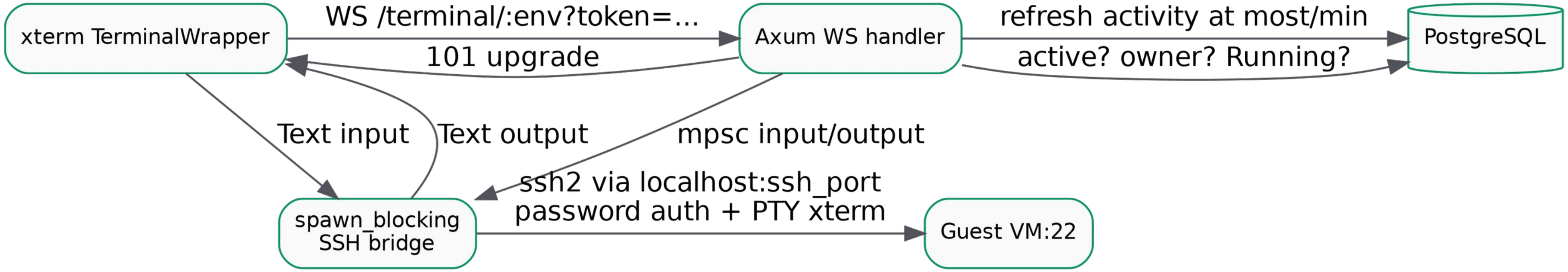
14.5 Cleanup, תחרותיות וריבוי replicas

שפג תוקפם. קלט environments לולאת הרקע מתעוררת כל 60 שניות ולוקחת לכל היותר עשרה terminal והגשת flag מעדכנים פעילות/expiry, אך refresh על כל API read. אין `FOR UPDATE SKIP LOCKED` או lease worker, ולכן שני replicas יכולים לבחור אותו batch ולנסות למחוק אותה VM. פעולת delete מתוכננת להיות סבילה לחזרה בחלק מהמצבים, אך פקודות חיצוניות ועדכוני status עדיין יכולים להתחרות.

לפני horizontal scaling דרושים job ownership, לפריסה יחידה זו מגבלה ידועה ולא תקלה מיידית. reconciliation בעת startup, timeout לכל פקודת `VBoxManage`, telemetry של שלבים ומדיניות

לא יפגע בבקשות VM כדי שכשל או עומס בתשתית API מן provisioning worker רצוי גם להפריד retry. תוכן ופרופיל.

15. WebSocket, SSH והטרמינל



תרשים

11 — WebSocket אל SSH

`TerminalWrapper` יוצר xterm, FitAddon ו־WebSocket אל URL שמייצר `labService.getTerminalUrl`. לפני upgrade השרת מאמת token, UUID subject, `is_active`, בעלות environment, environment Running, instance קיים ו־Running ו־ssh_port חוקי. רק אז הוא מחזיר 101.

אחרי upgrade ה־socket מתפצל ל־sender/receiver. שני ערוצי mpsc bounded בגודל 100 מחברים בין async WebSocket לבין `spawn_blocking` של ssh2. SSH מתחבר ל־localhost:port, מבצע password auth, מבקש PTY בשם `xterm` בגודל 80×24 ומפעיל shell. Text frames מועברים כקלט; bytes שנקראו ב־chunks של 1024 מומרים lossily ל־UTF-8 ונשלחים כ־Text. binary frames נזרקים. אין protocol ל־resize PTY, ולכן FitAddon משנה רק את תצוגת הדפדפן.

`tokio::select!` מסיים את session כאשר צד send, receive או SSH מסתיים, ומבטל את האחרים. אין explicit close reason ואין ללקוח reconnect/resume. ב־DEBUG הקוד מדפיס כל קלט SSH — סיכון דליפת פקודות/סודות ללוג.

15.1 Handshake וגבולות אמון

WebSocket בדפדפן אינו מאפשר להגדיר בחופשיות כותרת Authorization, ולכן הלקוח מצרף token ל־query string. ה־handler מאמת אותו בעצמו לפני `on_upgrade`; המסלול אינו משתמש ב־REST auth middleware. הוא בודק שהחשבון פעיל, שה־environment שייך ל־subject, ושני הסטטוסים Running. סדר זה מונע פתיחת socket ורק אחר כך גילוי שאין הרשאה.

עם זאת, URL הוא ערוץ רגיש: access logs של proxy, כלי telemetry או browser diagnostics עשויים לשמור query. ההמלצה היא endpoint שמנפיק ticket חד־פעמי, עם environment ID, subject ו־expiry של שניות. ה־WebSocket ישתמש ב־ticket והשרת יסמן אותו consumed. כך JWT ארוך־חיים אינו מופיע ב־URL.

15.2 Backpressure וסיום session

שני ערוצים bounded בגודל 100 מונעים גידול זיכרון בלתי מוגבל אם צד אחד איטי. כאשר הערוץ מלא, producer ממתין במקום לצבור frames. זהו מנגנון backpressure בסיסי וטוב. עם זאת, כל output נהפך ל־Text אחרי UTF-8 lossy; יישום terminal שמפיק bytes שאינם UTF-8 עשוי להציג תווי החלפה. input Binary WebSocket נזרק, ולכן אין תמיכה בפרוטוקול עשיר יותר.

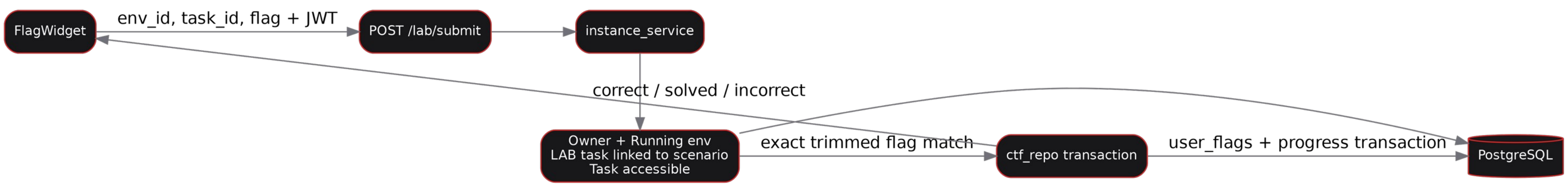
אין heartbeat ייעודי, timeout של session או הודעת close מובנית המתארת ”Lab expired“. cleanup עשוי להפסיק VM בעוד socket פתוח, ואז הצד של SSH יסתיים וה־select יפיל את session. UX טוב יותר ישלח close code והודעה ברורה, ויבדיל בין disconnect רשת, expiry, deletion ו־SSH authentication failure.

15.3 PTY, resize וסודות תפעוליים

מסוג frame בדפדפן, אך אין xterm מודד מחדש את חלון `FitAddon`. נוצר תמיד כ־80×24 PTY־ה או עורכי טקסט עשויות לחשוב `top` תוכניות כמו; `request_pty_size` שמגיע לשרת ואין קריאת resize payload, לצד JSON control שהמסך נשאר בגודל המקורי. פרוטוקול עתידי יכול להשתמש בהודעות לדוגמה `{type:"resize",cols:120,rows:36}`.

פרטי SSH קשיחים בקוד הם חוב אבטחה ותפעול: כל template חייב להתאים להם, ורוטציה דורשת build. הספר בכוונה אינו משכפל את הערכים. יש להעבירם ל־secret store או להזריק key קצר־חיים לכל clone. בנוסף יש להסיר logging של input או לכל הפחות להשביתו כברירת מחדל, מפני שמשתמש יכול להקליד credentials בתוך shell.

16. CTF, דגלים והשלמת משימות



תרשים 12 — הגשת דגל

המקושרת לאותו LAB לכן אותו דגל תקף לכל משימת task. אין לא ישירות ל-scenario, אלא דרך scenario מקושר ל-task.scenario_id = environment.scenario_id ו-task_type='LAB' השירות מצמצם סיכון בכך שהוא דורש scenario; למשימה מסוימת מעבר ל-flag הוא אינו קושר scenario מסוים.

נשמרים flag values ללא hashing וללא constant-time compare. ההשוואה מדויקת לאחר `trim`, מחזיר masked value, אך endpoint details מחזיר את הערך המלא למנהל. list Admin plaintext. מפרש הודעה ולא Frontend ולכן ה-HTTP 200 עם הודעת "Incorrect", בתשובה לדגל שגוי מוחזר status.

טרנזקציית `submit_flag_and_complete_task` מכניסה `user_flags` עם `ON CONFLICT DO NOTHING`, ומשלימה task עם `tasks.points`. שדה `flags.points` נקרא אך אינו משמש לניקוד; task points הם מקור הניקוד. זוהי אי־בהירות במודל שדורשת הכרעה עתידית בפועל.

16.1 סדר הבדיקות בהגשה

דורש סוג task, טוען את environment ושהוא Running, לפני השוואת הדגל השירות מאמת בעלות על LAB, ומוודא התאמת `task.scenario_id` ל-`environment.scenario_id`. רק אחר כך הוא מחפש flag באמצעות LAB הסדר מונע ממשתמש להשלים משימת `trim` עבור התרחיש ומשווה ערך לאחר environment מתרחיש אחר, גם אם הוא מנחש דגל תקף.

פשוט אך UI זה מאפשר business-negative. ולכן מתקבל `200` עם תשובת exception דגל שגוי אינו לא יבחין הצלחה בכישלון. אפשר לשמור `200` ולהחזיר status פוגע בסמנטיקה ובמדדים: ניטור לפי של מחרוזת matching או לבחור `422`; העיקר לנסח חוזה מפורש ולא להסתמך על `{correct:false}`, הודעה.

16.2 אטומיות ו-idempotency

על זוג `ON CONFLICT DO NOTHING` .DB הכנסת פתרון והשלמת המשימה מתבצעות באותה טרנזקציית user/flag ו- ,upsert של progress ומהיסטוריית פתרון כפולה. אם התהליך נופל שומר ניקוד שהוענק. שתי הפעולות מתבטלות; אם הלקוח לא קיבל תשובה וחוזר על הבקשה, אין הענקה כפולה. ,commit לפני זוהי אחת הזרימות העקביות ביותר בפרויקט.

ייחודי לפי `user_flags` בעוד ,scenario יכולות להפנות לאותו tasks המודל עדיין מאפשר עמימות: כמה flag ולא לפי task. נוספת; progress עשוי LAB משתמש שכבר פתר את הדגל יכול להגיע אותו למשימה יחיד או סביבת challenge מייצג scenario להסתיים, אך לא תיווצר רשומת פתרון נוספת. יש להחליט אם

עבודה שיכולה לשרת כמה challenges. במקרה השני עדיף קשר flag→task או טבלת submission מפורטת.

16.3 הגנת ערכי דגל

ערכי flags נשמרים plaintext ונחשפים ב־details למנהל. במערכת לימוד מקומית הדבר מקל על תפעול, אך DB dump או log לא מבוקר חושף תשובות. אם אין צורך להציג את הערך לאחר יצירה, ניתן לשמור hash ולבצע השוואה. אם צריך reveal, יש להצפין במנוחה עם key מחוץ ל־DB ולתעד audit לכל צפייה. constant-time comparison היא הקשחה משנית; rate limiting וניטור ניסיונות חשובים יותר נגד brute force.

17. פרופיל משתמש

כל endpoints של profile מפיקים user ID מ־Claims, בלי path לכן; IDOR של משתמש אחר בסיסי. update profile אינו אפשרי. email מטפל ב־unique violation מינימלי, מאמת שם לא ריק ופורמט new/confirm נכון, התאמת current password דורש password שינוי. `updated_at` ומעדכן hash bcrypt חדש שמוגדר בשירות; לאחר מכן נשמר.

אף MiB 3 של body limit מגדיר route. `avatar_url` אופציונלי בינארי: זהו upload אינו Avatar טוען נתונים, מחזיק `useProfile` בסיסי validation השירות מבצע. JSON URL; שבפועל מועבר navbar־שה, `updateStoredUser` קורא update ואחרי loading/error/action states, profile אוטומטי של re-fetch השמור; אין user־ה טוען refresh מתעדכנים. לאחר AuthContext־וה AuthProvider־ב.

17.1 חוזי validation מדויקים

ונבדק ככתובת לפני עדכון. email עובר trim, הופך lowercase. שם משתמש נדרש לאורך 3–50 תווים וסיסמה חדשה השונה מן confirm, שינוי סיסמה דורש סיסמה נוכחית נכונה, מינימום שמונה תווים, התאמת קיימת מגבלת data URL מסוג PNG, JPEG או WebP; `null` להסרה או avatar מקבל הנוכחית. עדכון Base64, זהו פער בין קידוד MB אורך של כ־2.8 מיליון תווים, אף שהודעת השגיאה מדברת על פחות מ־2 JSON מגבלת והמסר למשתמש.

17.2 עקביות client ו־session

לאחר שינוי שם, email או avatar, ה־hook מחזיר profile ומעדכן את המשתמש השמור. בכך תפריט הניווט אינו נשאר עם avatar ישן. שינוי password אינו מבטל tokens קיימים ואינו מכריח login מחדש; מי שמחזיק token גנוב יכול להמשיך עד expiry או השבתת החשבון. אם שינוי סיסמה אמור להיות אירוע אבטחה, נדרש session version בתוך DB/JWT או רשימת sessions שניתנת לביטול.

אין endpoint למחיקת חשבון עצמי. מנהל יכול להשבית משתמש, וה־middleware יחסום אותו מיידית. הנתונים המקושרים נשארים לצורכי audit והיסטוריה. מדיניות export, retention ו־privacy אינה קיימת בקוד ויש להגדירה לפי סביבת שימוש.

18. ממשק הניהול

מודול	מסך	פעולות ו־API	הרשאה	מצב
Overview	`AdminOverviewPage`	GET `/admin/dashboard`	RequireAdmin + שני middleware	ממומש
Courses	`AdminCoursesPage`	list/create/edit/delete courses	כנ״ל	ממומש; page עצמו wrapper קצר ל־hook/editor
Course editor	`AdminCourseEditorPage`	sections/tasks CRUD, publish, video URL	כנ״ל	ממומש
Scenarios	`AdminScenariosPage`	CRUD scenario/template/time/score/active	כנ״ל	ממומש
Users	`AdminUsersPage`	search, status filter, pagination	כנ״ל	ממומש
User details	`AdminUserDetailsPage`	status/role/password + activity summary	כנ״ל	ממומש
Labs	`AdminLabsPage`	search/filter/page/terminate	כנ״ל	ממומש
Flags	`AdminFlags`	masked list, details,	כנ״ל	ממומש

מודול	מסך	פעולות ו־API	הרשאה	מצב
	Page	CRUD		
Activity	AdminActivityPage	action/entity/admin filters, order/page	כנ״ל	ממומש

גם פעולות mutation נרשמות ב־admin_activity_logs עם actor, action, entity ו־JSON details. CRUD Academy קורא record_activity דרך admin handler/service. list queries מנורמלות ומוגבלות; filters נבנים באמצעות parameters ולא string interpolation. Activity logs הם audit תפעולי, לא security log מלא: login failures, user actions ו־terminal sessions אינם נרשמים לטבלה זו.

18.1 שכבות הגנה וגבולות פעולה

Admin ולאחר מכן Auth מקבל תחילה route תחת /academy/admin וכל route תחת /admin כל middleware. השרת, ואו URL מנווט ישירות ל־localStorage ו־role גם אם משתמש משנה ידנית את מזהה מנהל אינו מתקבל מגוף הבקשה. כך ה־Claims מן ה־actor מפיקים את handlers הדוחה אותו. ה־audit אינו יכול להיות מיוחס ביוזמת הלקוח למנהל אחר.

בלבד: מנהל אינו יכול להשבית את עצמו או להוריד RBAC מוסיף כללים שאינם נובעים מ־service ה־ וסיסמה חדשה חייבת לעמוד באימות. מחיקת admin או user חדש חייב להיות role, role לעצמו הופכים אותה למסוכנת. כללים אלה מגינים מפני נעילה runtime נחסמת כאשר קשרי תוכן או scenario מנהל יחיד עדיין יכול; two-person approval של workflow עצמית ומפני שבירת קשרים, אבל אינם לשנות משתמשים ותוכן בהיקף רחב.

18.2 Pagination, filters ויציבות תוצאות

מסכי users, labs, flags ו־activity משתמשים ב־pagination שרת. ה־service מנרמל page/page_size ומגביל גודל כדי למנוע שאילתות בלתי חסומות. החיפוש והמסננים מועברים כ־bind parameters, ולכן ערכי משתמש אינם נבנים לתוך SQL. order מוגבל לערכים שהקוד מכיר. אלה בקרות חשובות הן לביצועים והן למניעת injection.

בעת שינוי או מחיקה ה־hooks מרעננים את הרשימה או את פרטי הישות ומציגים error state. לא נמצא optimistic update מורכב; זו בחירה סבירה ל־Admin מפני שמקור האמת נשאר תגובת השרת. מנגד, אין ETag, version column או optimistic concurrency control: שני מנהלים העורכים אותה ישות יכולים לבצע last-write-wins בלי התראה.

18.3 Auditability

רשומת activity כוללת actor, פעולה, סוג ישות, מזהה אופציונלי, details JSONB וזמן. זה מספיק לשחזור פעולות CRUD עיקריות ולמסך פעילות מסונן. אין חתימה קריפטוגרפית, append-only privilege ברמת DB או יצוא immutable, ולכן מנהל DB יכול לשנות את היומן. גם פעולות קריאה רגישות, כגון צפייה בערך flag מלא, אינן מתועדות בהכרח.

לסביבה מבוקרת מומלץ להגדיר taxonomy קבוע לפעולות, להימנע מהכנסת סודות ל־details, ולשלוח עותק ל־sink חיצוני. יש להוסיף request/correlation ID כדי לחבר בין error log, access log ו־activity row. אירועי authentication, provisioning, terminal connect ו־attempts flag צריכים ערוץ security telemetry נפרד או הרחבה מפורשת של המודל הקיים.

19. תהליכים מלאים מקצה לקצה

פרק זה עוקב אחר גבולות המערכת בסדר כרונולוגי. בכל תהליך מצוינים רכיב ה־UI, ה־Hook או service בצד הלקוח, התקשורת, שרשרת ה־Backend, הנתונים המשתנים, התשובה, עדכון ה־UI והבקרות. הוא משלים את מקרי השימוש בכך שהוא מדגיש את מעבר המידע בין שכבות.

19.1 הרשמה

1. `Register.jsx` האורח ממלא את; component־ה מחזיק את ערכי הטופס ואת מצב השגיאה.
2. Submit קורא `AuthContext.register`, שמעביר ל־`authService.register`.
3. נשלחת בקשת `POST /api/auth/register` עם JSON; אין token ואין middleware.
4. `auth_handler::register` מחלץ `RegisterRequest` וקורא `auth_service::register`.
5. השירות מחשב bcrypt hash. אין validation של חוזק, lowercase email או בדיקת שדות ריקים בשירות.
6. `user_repo::create_user` מבצע INSERT ל־`users` עם role `'user'`; DB מאכף email unique וממלא defaults.
7. row מלא כולל hash מומר ל־`UserResponse` בטוח.
8. `201 Created` השרת מחזיר. handler בכל שגיאה מהשירות, גם אם עם טקסט, `400` מחזיר מקורה DB.
9. ה־Frontend אינו שומר session; הוא מציג הצלחה ומוביל ל־Login. בקרות האבטחה הן bcrypt, role קשיח ו־DTO שאינו חושף hash.

19.2 התחברות

1. Login.jsx קורא AuthContext.login(email,password).
2. authService.login שולח POST /api/auth/login ללא Authorization.
3. handler מפענח LoginRequest; service קורא user_repo::get_user_by_email.
4. repository מחזיר User, ובמקביל מחשב total_score מסכום earned_points ולא בהכרח מהעמודה denormalized.
5. bcrypt משווה את הסיסמה. email חסר וסיסמה שגויה מתכנסים ל־Invalid credentials.
6. is_user_active מבצע query נוסף. חשבון לא פעיל נדחה לפני יצירת token.
7. generate_token יוצר claims: UUID ב־sub, role כפי שנקרא ו־24 שעות תוקף.
8. 200 OK מחזיר token ו־user. AuthContext מעדכן state ו־localStorage.
9. ה־router עובר ל־Dashboard. אין refresh token או revalidation מיידית בצד הלקוח; האימות הבא מתרחש בבקשת API.

19.3 כניסה למסלול מוגן

1. React Router עוטף את /dashboard, /academy, /machines ו־/profile ב־RequireAuth.
2. guard רק קורא localStorage.getItem("token"); token פג עדיין מאפשר render ראשוני.
3. page hook שולח API עם Authorization: Bearer <token>.
4. auth_middleware מפצל את header, מאמת JWT ו־expiry וממיר sub ל־UUID.
5. middleware קורא role,is_active מ־users; משתמש חסר מקבל 401, מושבת מקבל 403.
6. role העדכני נכתב חזרה ל־Claims ומתווסף ל־request extensions.
7. handler משתמש ב־Claims ולא ב־user ID מהלקוח. כשל מגיע ל־service של ה־Frontend, שמציג error; אין interceptor מרכזי שמבצע logout אוטומטי על 401.

19.4 פתיחת קטלוג וקורס

1. CoursesPage מפעיל useCourses; academyService.getCourses שולח GET ל־/api/academy/courses.
2. route זה ציבורי ב־Backend, אף שהעמוד עצמו מוגן ב־Frontend. service מחזיר רק courses שפורסמו.

3. בחירת כרטיס מנווטת ל־`/academy/:courseId`; `CoursePage` מפעיל `useCourse` ו־`useCourseProgress`.
4. GET full מחזיר course עם sections/tasks; GET progress עובר Auth ומזהה את המשתמש מה־JWT.
5. repository ממיין section/task order. service מחשב totals, אחוזים, points ו־access.
6. ה־Frontend מאחד את שני ה־responses לפי task ID, מציג progress bar, Sidebar ו־workspace.
7. 404 תוכן או progress מוצג כטקסט error. אם course קיים אך אין tasks, percentage הוא 0 וה־workspace נשאר ללא selection.

19.5 התחלת משימה

1. `SectionSidebar` מעביר task ל־`CoursePage.handleSelectTask`.
2. ה־component בודק את access state וחוסם click האחרון על `LOCKED`.
3. task נבחרת מיד לתצוגה; רק אם היא `NOT_STARTED` נשלח POST start.
4. Auth middleware פועל; handler מפיק user ID מ־Claims.
5. `start_task` קורא `ensure_task_accessible`, שמחשב מחדש progress מה־DB. זה מונע TOCTOU מול state פשוט של ה־UI ישן.
6. repository מבצע UPSERT ל־`IN_PROGRESS`; completed נשאר completed.
7. השירות מחזיר progress מלא, אך ה־Frontend מתעלם מ־response הנוכחי וקורא ל־`reloadProgress` שוב.
8. הצלחה מעדכנת Sidebar. כשל נכתב ל־console בלבד בבחירה הראשונית; זהו פער UX.

19.6 השלמת LESSON או PRACTICE והענקת נקודות

1. `LessonLayout` או `PracticeLayout` מפעילים `completeContentTask` ומציגים busy state.
2. POST complete נשלח ללא task ID ב־body; ב־path והמשתמש ב־JWT.
3. service מאתר course, טוען progress, דוחה `LAB`, ומחשב access מחדש.
4. `mark_task_completed` מבצע INSERT...SELECT מתוך `tasks`, ולכן points מגיעים מ־`tasks.points`.
5. ON CONFLICT שומר `earned_points` אם הסטטוס כבר completed; אין צבירה כפולה.

6. progress response כולל task הבאה כ־AVAILABLE. ה־component קורא callback, מרענן שוב, בוחר את הבאה ואף מסמן אותה start.
7. Dashboard אינו מקבל push; בכניסה/refresh הבא הוא מחשב מחדש את course category ואת score מסכום progress.

19.7 יצירת שיעור וידאו דרך Admin

1. מנהל פותח `AdminCourseEditorPage` ו־`TaskForm`.
2. בחירת `LESSON` מציגה שדה `video_url`; מעבר לסוג אחר מנקה אותו ואת `scenario_id` בהתאם.
3. validation בצד הלקוח מקבל HTTPS YouTube watch, youtu.be, embed או shorts.
4. `adminService` שולח POST `/api/academy/admin/tasks`; שני middleware מאמתים user ו־Admin.
5. handler מפיק admin ID וקורא `academy_service::create_task`.
6. service מנרמל type ל־`LESSON`; `video_id_from_url` קורא parser שמאמת scheme, port, credentials, domain, path ו־ID באורך 11.
7. repository שומר `youtube_video_id`, לא את URL המקורי. DB חוזר ומאכף format, lesson-only ו־type enumeration.
8. לאחר insert נכתב audit log. response task מחזיר `youtube_video_id`; ה־hook מרענן את editor.

19.8 הצגת סרטון למשתמש

1. GET course full כולל `youtube_video_id` בתוך `TaskDto`.
2. `TaskRenderer` בוחר `LessonLayout`; ה־layout מציג Markdown ולאחריו `VideoWidget`.
3. widget בודק שוב pattern של ID. null או ID לא חוקי אינו מציג iframe.
4. ID תקין משובץ ב־`https://www.youtube-nocookie.com/embed/<id>` עם lazy loading, strict referrer policy ו־allow list.
5. אין בקשת Backend ל־YouTube בזמן הצגה; הדפדפן מתקשר ישירות ל־domain החיצוני.
6. חסימת רשת/Content Policy משאירה iframe כושל אך אינה מונעת השלמת LESSON. צפייה אינה נמדדת, ואין אירוע "video watched".

19.9 יצירת Lab

1. `LabLayout`/`MachineWidget` קוראים hook של Labs, שמפעיל `labService.createLab(scenarioId)`.
2. create POST נשלח עם JWT ו־handler מפיק scenario ID. user ID מה־Claims.
3. service מחפש כל environment פעיל. expired נמחק; פעיל אחר גורם `400`.
4. scenario repository דורש `is_active=true`. לאחר מכן נוצר environment עם `Building` ו־expires_at.
5. OS בוחר port דרך bind זמני; instance נוצר עם `Starting`, שם VM ייחודי ו־entry point.
6. `spawn_blocking` מריץ `clonevm`, מגדיר NAT SSH, מפעיל headless וממתין עד 120 שניות.
7. instance מסומן Running ואז environment Running. response `201` חוזר רק בסיום; אין job ID ביניים.
8. ה־hook שומר `envId`, port, expiry ומצב. כשל מחזיר message; best-effort cleanup מופעל בצד שרת.

19.10 התחברות לטרמינל

1. `TerminalWrapper` יוצר xterm, FitAddon ומציג “Connecting”.
2. `getTerminalUrl` ממיר HTTP origin ל־WS ומוסיף environment ID ו־token מקודד ל־query.
3. handler מאמת token כמו ה־REST middleware, קורא `is_active`, ובודק environment בבעלות subject.
4. environment ו־instance חייבים להיות Running; port חייב להיכנס ל־u16. שגיאות לפני upgrade הן 401/403/404/409/500.
5. אחרי upgrade נפתחים channels bounded ו־`spawn_blocking` ל־ssh2.
6. ssh2 מתחבר ל־localhost:port, מבצע password auth, מבקש PTY 80×24 ומפעיל shell.
7. Text input עובר browser ל־SSH; output נקרא ב־1KB ומוחזר כ־Text. input מרענן activity לכל היותר פעם בדקה.
8. resize בדפדפן מתאים את canvas בלבד; אין הודעת resize ל־PTY. cleanup סוגר WebSocket/xterm ומבטל tasks.

19.11 הגשת דגל

1. `FlagWidget` אוסף value; `useCTF`/lab service שולחים env ID, task ID ו־flag trimmed.
2. Auth middleware קובע user. service טוען environment בבעלות המשתמש ודורש Running.
3. activity/expiry מתרעננים best-effort. `task_repo` דורש task מסוג LAB עם אותו scenario.
4. progress service דורש task AVAILABLE. כך לא ניתן לדלג על משימה קודמת באמצעות דגל ידוע.
5. `ctf_repo::get_flag` משווה plaintext flag תחת scenario. אין relation ישיר flag-task.
6. דגל שגוי מחזיר `200` והודעת failure; דגל נכון מפעיל transaction.
7. transaction מכניס `user_flags` פעם אחת ומשלים progress עם task points. response מבדיל “newly solved” ו־“already submitted”.
8. ה־Frontend מציג message וקורא callback לרענון progress. דגל נכון פותח את המשימה הבאה.

19.12 מחיקת Lab

1. פעולה ב־MachineWidget קוראת `deleteLab(environmentId)`.
2. POST delete עובר Auth; service מבצע ownership query.
3. environment עובר Stopping. `Destroyed` אחרת מחזיר הצלחה ללא שינוי.
4. instance חסרה גורמת לסגירת environment יתום. instance קיימת עוברת Stopping.
5. VBox delete רץ blocking; אם VM כבר אינה רשומה utility מחזיר הצלחה.
6. לאחר הצלחה instance עוברת Destroyed ואז environment, עם stopped_at ו־expires_at null.
7. ה־hook מנקה active Lab ומפסיק להציג failure terminal. Failed משאיר ומציג הודעה.

19.13 עריכת פרופיל

1. `useProfile` טוען profile ומאכלס `ProfileDetailsForm`.
2. submit שולח PUT `/api/profile/` עם `user_name,email`; אין target ID.
3. middleware מצרף Claims; handler מפיק UUID.
4. service trims, lowercases email, מאמת 3–50 תווים ופורמט email, ובודק duplicate.

5. repository מעדכן רק `WHERE id=$user_id` ומחזיר row בטוח.
6. hook מחליף profile וקורא `updateStoredUser`; AuthContext ו־localStorage מסתנכרנים.
7. validation מוצג בטופס. unique violation מטופל גם לאחר race באמצעות error code של DB.

19.14 שינוי סיסמה

1. `ChangePasswordForm` אוסף current/new/confirm ושולח PUT `/profile/password`.
2. service בודק confirm, שמונה תווים לפחות, וקורא hash נוכחי.
3. bcrypt מאמת current; אחר כך משווה שהחדשה אינה זהה.
4. bcrypt hash חדש נוצר ונשמר; response message מוצג והטופס מתאפס.
5. ה־JWT הקיים אינו מבוטל לאחר שינוי; אין session revocation. current שגוי — 401; user חסר — 404; validation — 400.

19.15 שינוי role דרך Admin

1. `AdminUserDetailsPage` שולח PUT `/api/admin/users/:id/role` עם `{role}`.
2. Auth middleware קורא role actor עדכני; Admin middleware דורש Admin.
3. handler מפיק admin ID מה־Claims. service מנרמל/מאמת role ומונע self-demotion.
4. repository מעדכן target. `record_activity` כותב actor, target ופרטים.
5. response מחזיר user DTO; ה־hook מרענן. בבקשה הבאה של target, role נטען מה־DB ולכן השינוי מיידי אף שה־JWT לא הוחלף.

19.16 סיום מעבדה דרך Admin

1. `AdminLabsPage` שולח POST `/api/admin/labs/:environment_id/terminate`.
2. שני middleware מאמתים Admin. service טוען Lab row כדי לזהות owner וסטטוס.
3. הוא קורא `instance_service::delete_user_lab(pool, owner_id, environment_id)`; אין שימוש ב־admin ID כבעלים.
4. lifecycle זהה למחיקת משתמש: Stopping, VBox delete, Destroyed או Failed.

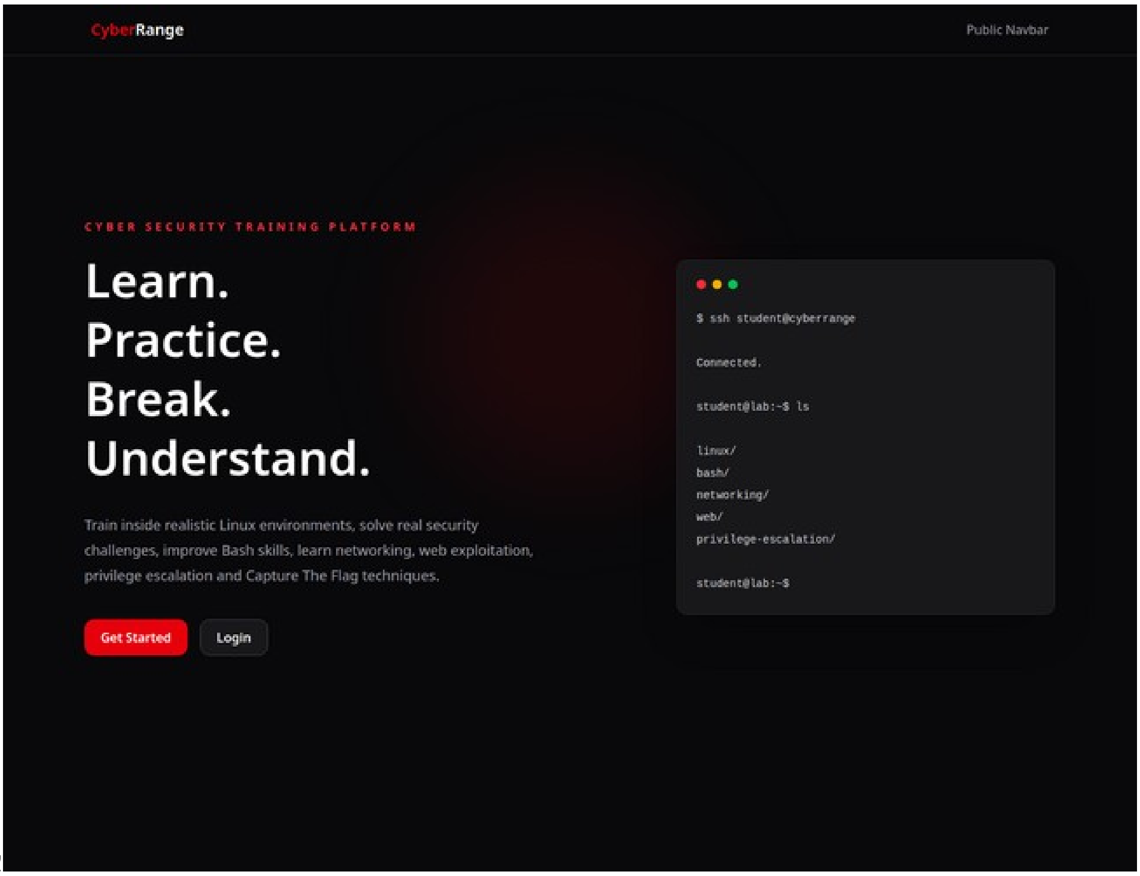
5. רק לאחר הצלחה נכתב response message activity log. מעדכן את hook והרשימה נטענת מחדש.

19.17 Dashboard לאחר שינוי מצב

1. `useDashboard` קורא GET `/api/dashboard/` בעת mount; Auth פועל.
2. repository מחשב user score מסכום `earned_points`, מאתר course IDs עם progress ו־courses מפורסמים ללא progress.
3. לכל קורס שהתחיל, service קורא את מנוע progress המלא. קורס עם כל tasks completed עובר לרשימת completed; השאר ל־continue learning.
4. current task היא task קודם ב־IN_PROGRESS, ואם אין — הראשונה AVAILABLE שאינה completed.
5. statistics נגזרים מהרשימות; available courses כוללים task/point totals.
6. response מוצג בשלושה אזורים. loading/error/empty קיימים ב־page. המחיר הוא N+1 queries לקורסים שהתחילו; זהו יעד אפשרי לאופטימיזציה.

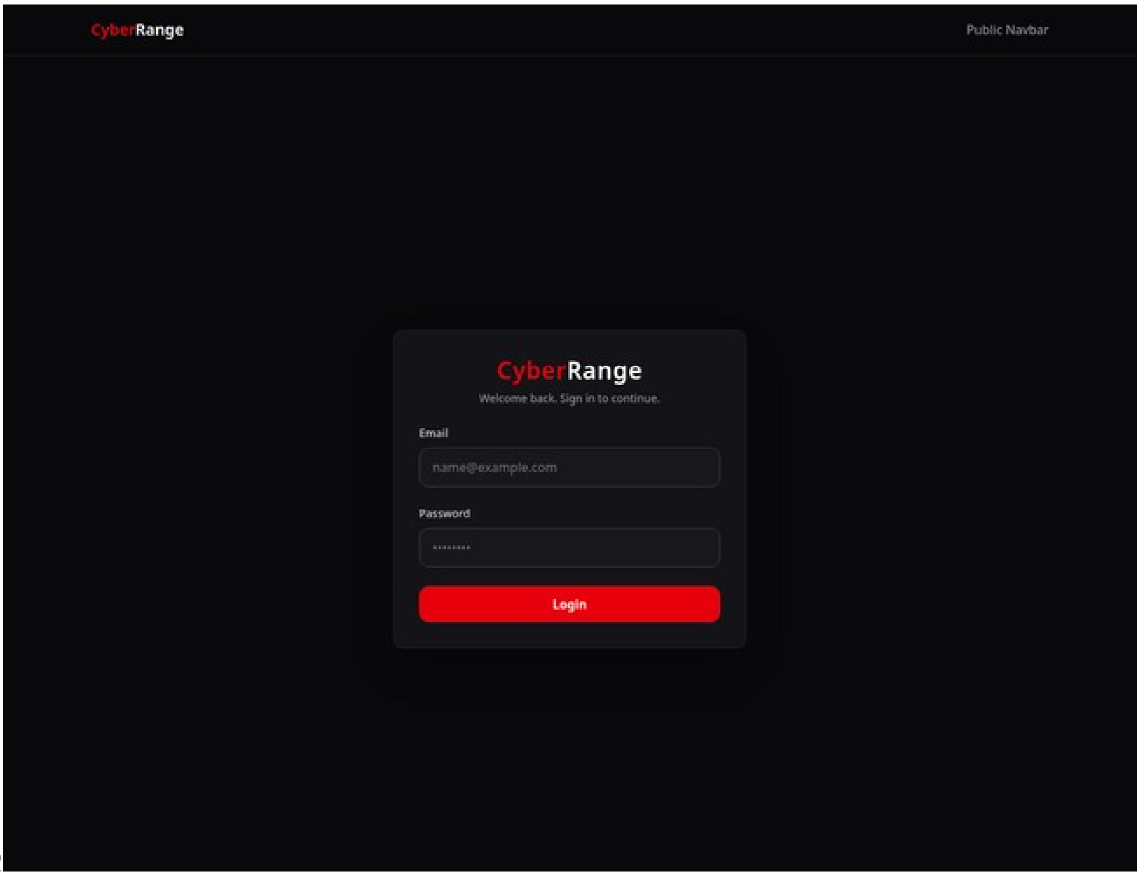
20. ממשק משתמש

ממשק המשתמש בפועל כהה, מבוסס Zinc/black עם אדום כצבע פעולה/מותג. Emerald מופיע בעיקר למצבי הצלחה/השלמה; Red לשגיאה, danger ו־Admin accents. Navbar ציבורי ומוגן, layouts ו־cards משתמשים ב־Tailwind utility classes. אין מערכת design tokens עצמאית מעבר למחלקות ורכיבי shared בסיסיים.

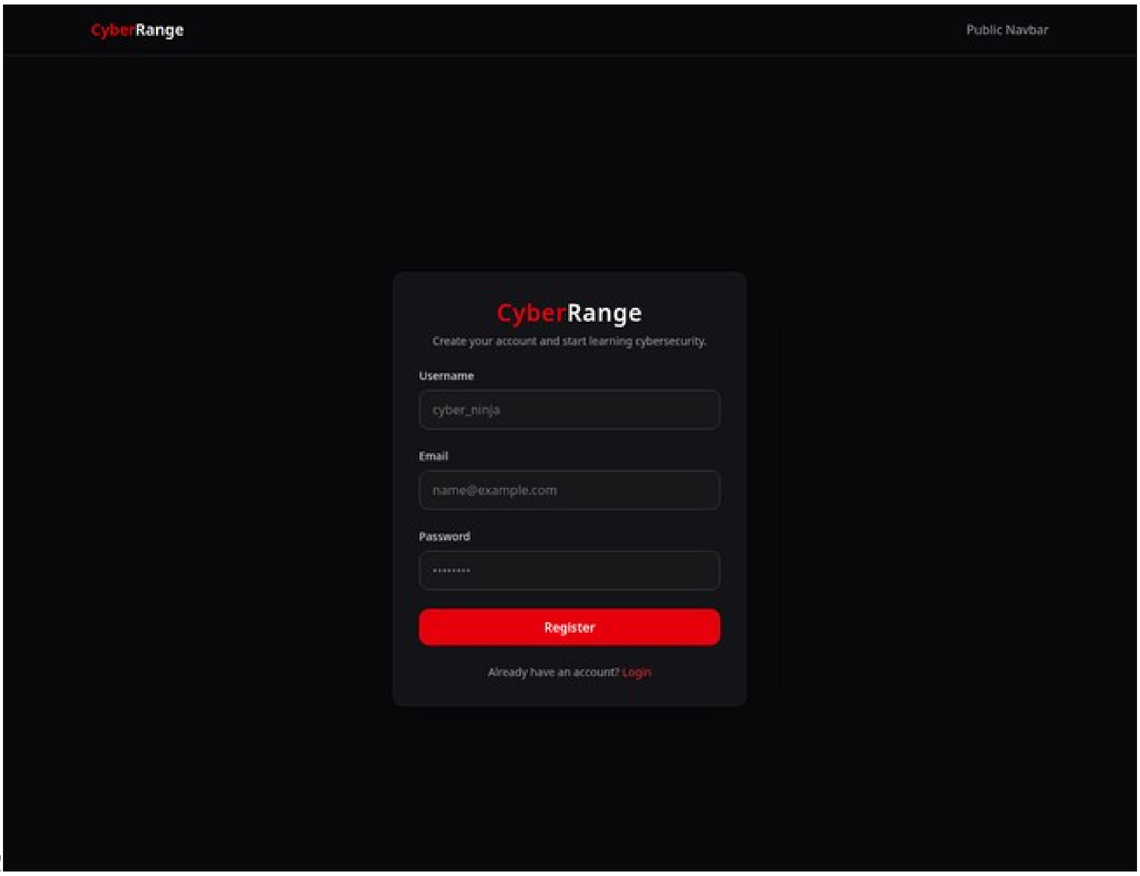


צילום 1 — Landing אמיתי מתוך ה־Frontend

הצילום הופק ב־5 באוגוסט 2026 מ־Vite המקומי, ללא שינוי קוד. הוא מציג את שם ה־mockup ללא UI `CyberRange`, מסר לימודי, terminal illustration ופעולות סטטי Get Started/Login.



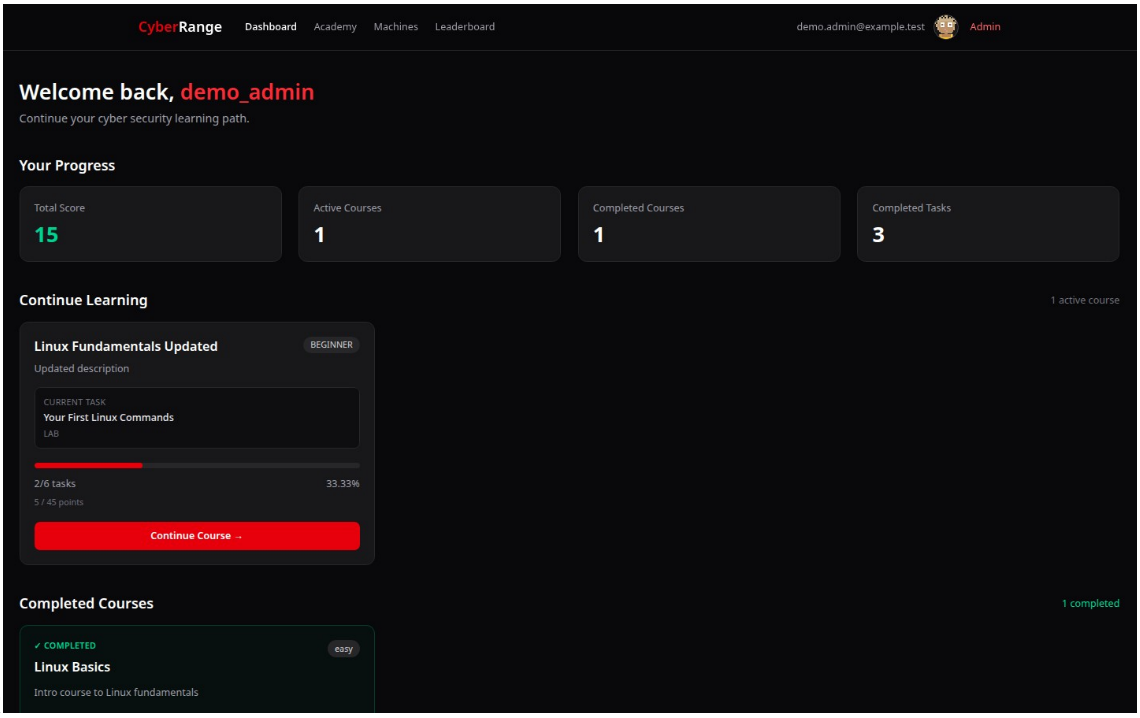
צילום 2 — מסך Login אמיתי



צילום 3 — מסך Register אמיתי

ללא `c28eb07`, קיים של חשבון בדיקה, מגרסת הקוד session המסכים המוגנים הבאים צולמו באמצעות פרטי זיהוי הוחלפו בתצוגה בלבד בערכי VirtualBox. יצירת משתמש, ללא שינוי נתונים וללא הפעלת קיימים React אמיתיים ורכיבי routes הצילומים משקפים .Demo.

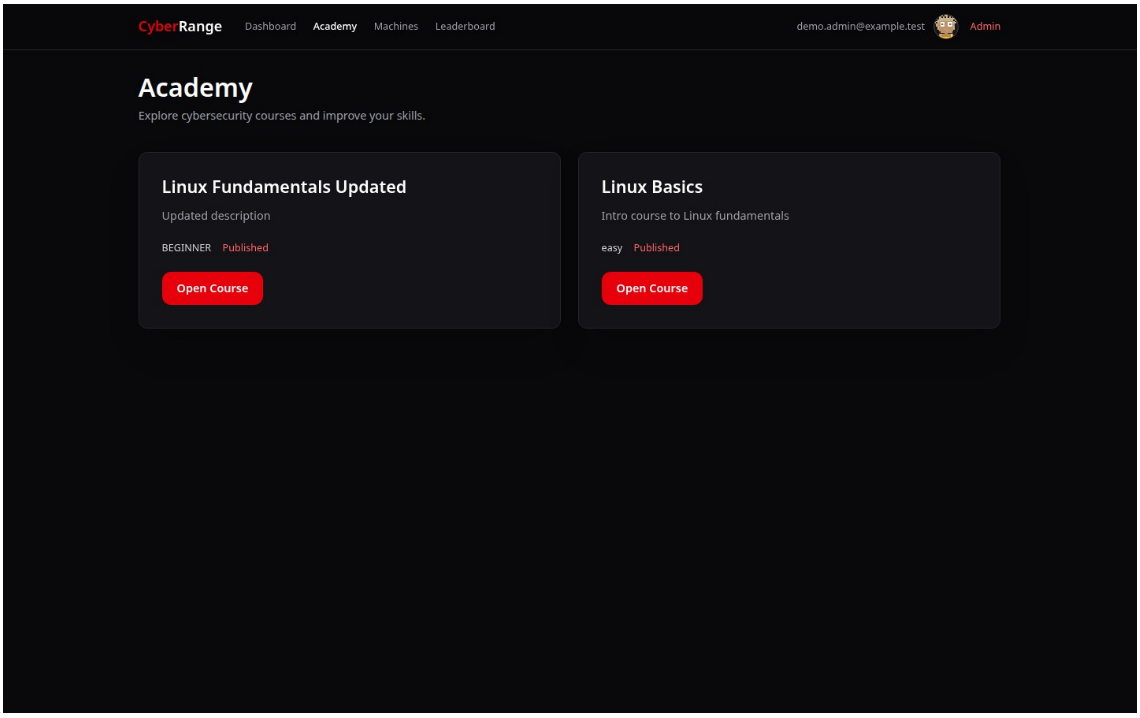
צילום 4 — Dashboard Route: `/dashboard` · הרשאה: User. מטרת המסך היא לרכז score, מחושב מ־`GET /api/dashboard/`, progress קורסים פעילים וקורסים שהושלמו. הכרטיסים מציגים ומאפשרים להמשיך ישירות לקורס הפעיל.



צילום 4 — Dashboard אמיתי של משתמש מחובר

כיתוב: Dashboard עם סטטיסטיקות, Continue Learning וקורס שהושלם.

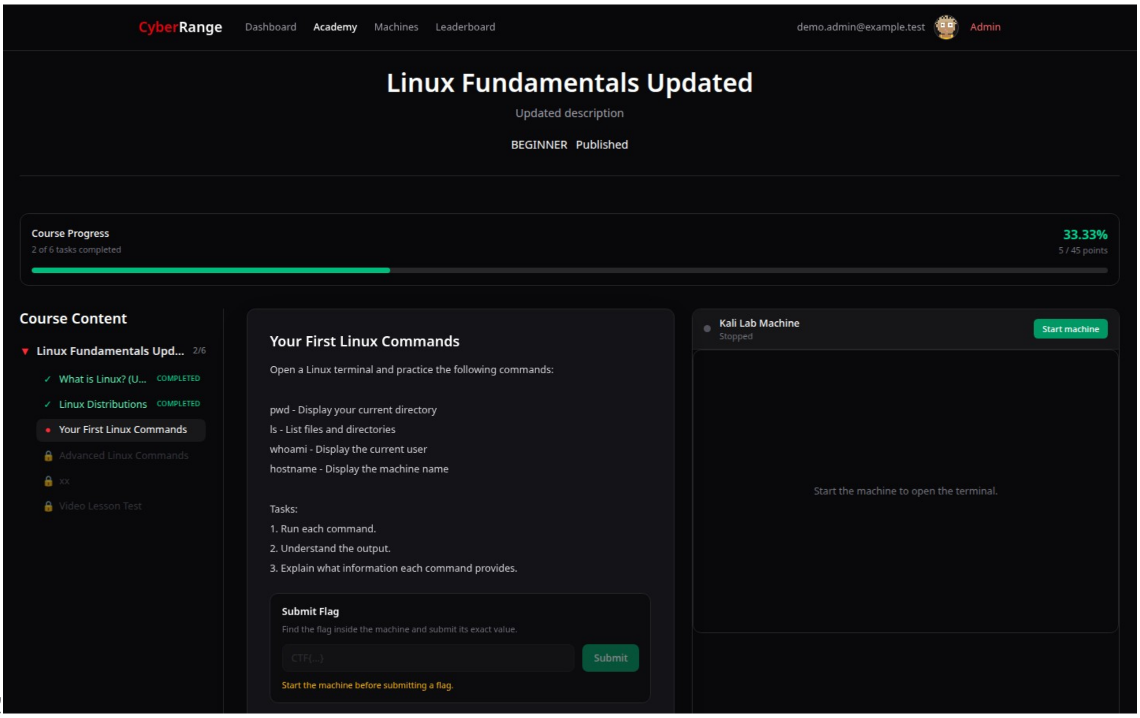
צילום 5 — Academy Route: `/academy` · הרשאה: User. קטלוג הקורסים מציג כותרת, תיאור, difficulty וסטטוס פרסום; הפעולה המרכזית היא פתיחת קורס דרך `/academy/:courseId`.



צילום 5 — קטלוג Academy אמיתי

כיתוב: רשימת הקורסים המפורסמים הזמינים למשתמש.

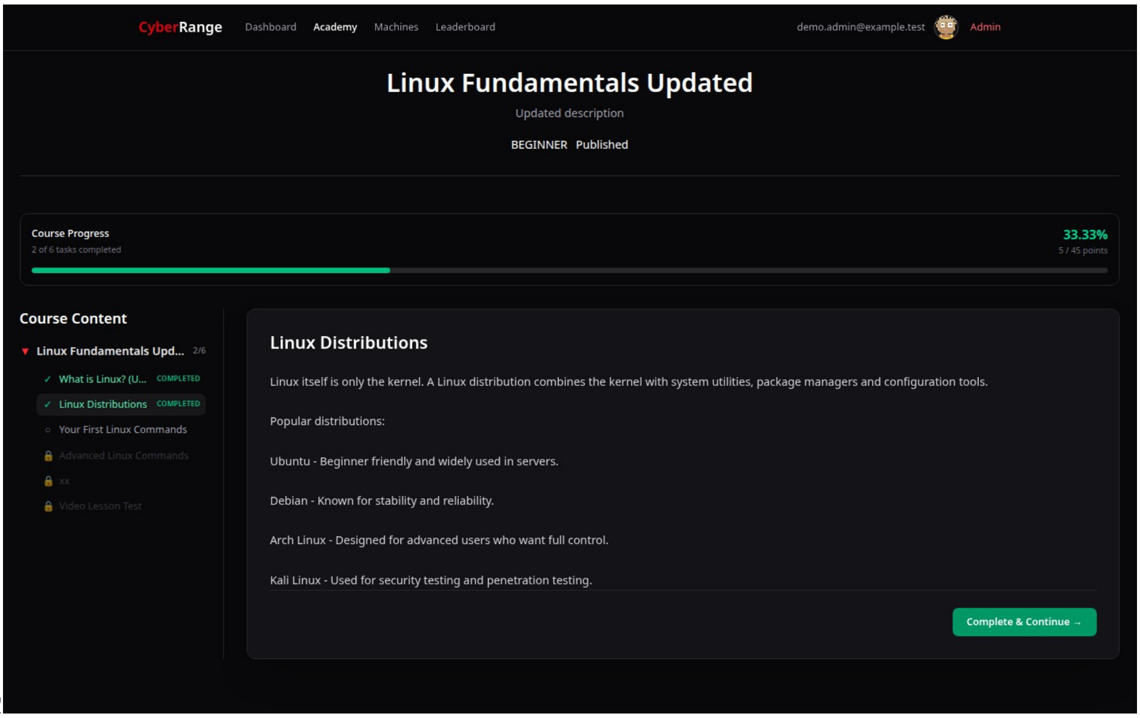
צילום 6 — Course, Sections, Tasks ו־LAB layout Route: `/academy/:courseId` · הרשאה: User. הכותרת והסרגל העליון מציגים התקדמות; הצד השמאלי מציג Sections ו־Tasks עם מצב Completed, Available ו־Locked. משימת LAB פתוחה מציגה הוראות, הגשת flag, מצב מכונה וטרמינל. המכונה נשארה Stopped ולא הופעלה לצורך הצילום.



צילום 6 — Course Page עם משימת LAB בטוחה

כיתוב: פריסת Course מלאה עם sidebar, תוכן LAB ו־Interaction Panel ללא יצירת environment.

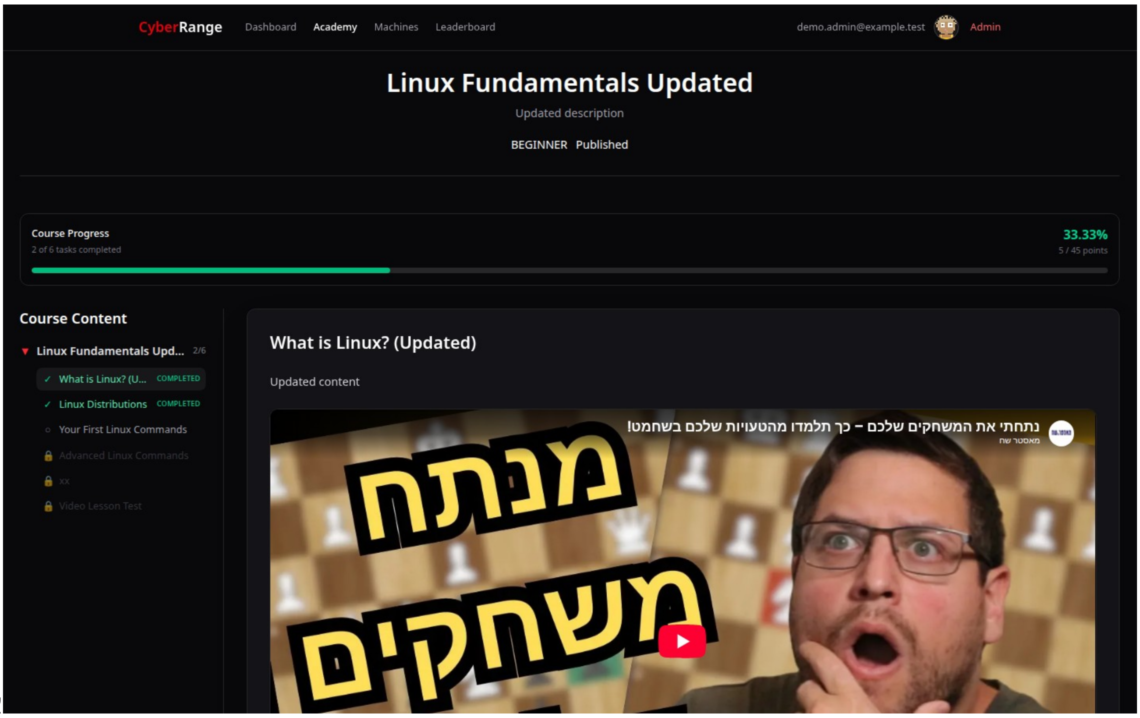
צילום 7 — LESSON רגיל Route: `/academy/:courseId` · הרשאה: User. בחירת משימת LESSON שכבר הושלמה מציגה תוכן לימודי ואת פעולת `Complete & Continue`; הבחירה לצילום לא שינתה progress.



צילום 7 — משימת LESSON רגילה

כיתוב: Learning Panel של שיעור טקסטואלי לצד רשימת המשימות והסטטוסים.

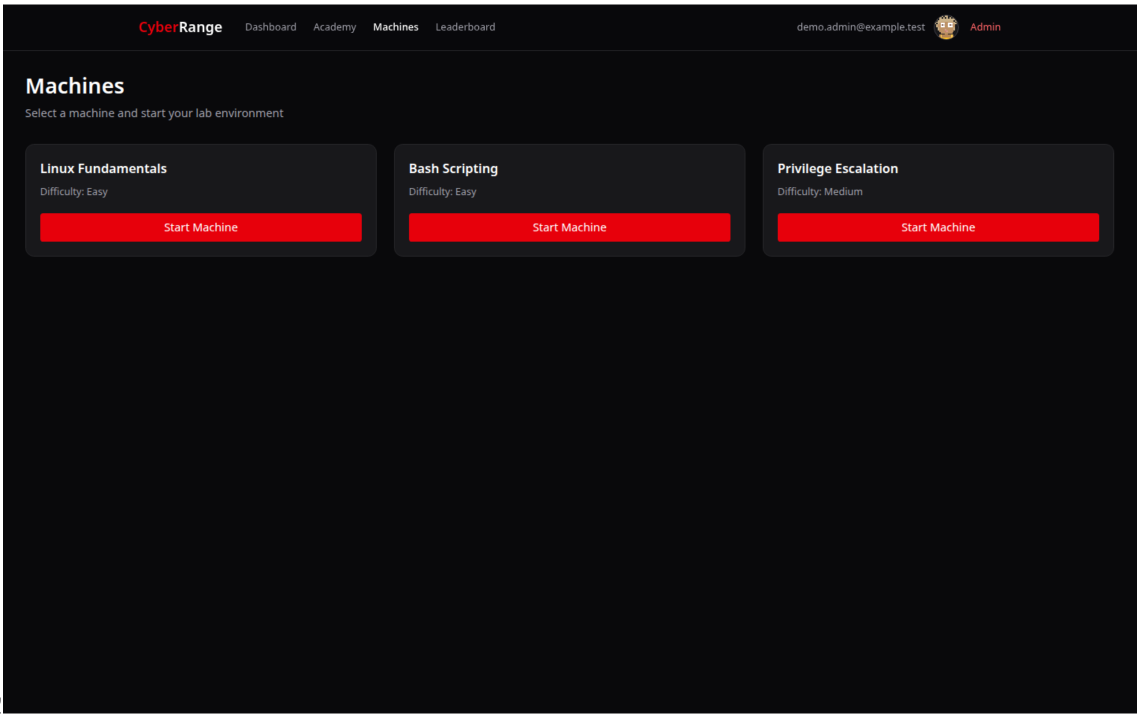
צילום 8 — LESSON עם YouTube Route: `/academy/:courseId` · הרשאה: User. כאשר למשימת LESSON יש `youtube_video_id`, `VideoWidget` משלב נגן YouTube בתוך אותו layout. הווידאו נטען מ־URL שאומת ונשמר על ידי מסלול Academy הקיים.



צילום 8 — משימת LESSON עם נגן YouTube

כיתוב: שיעור וידאו אמיתי בתוך סביבת הקורס, ללא מעבר לעמוד חיצוני.

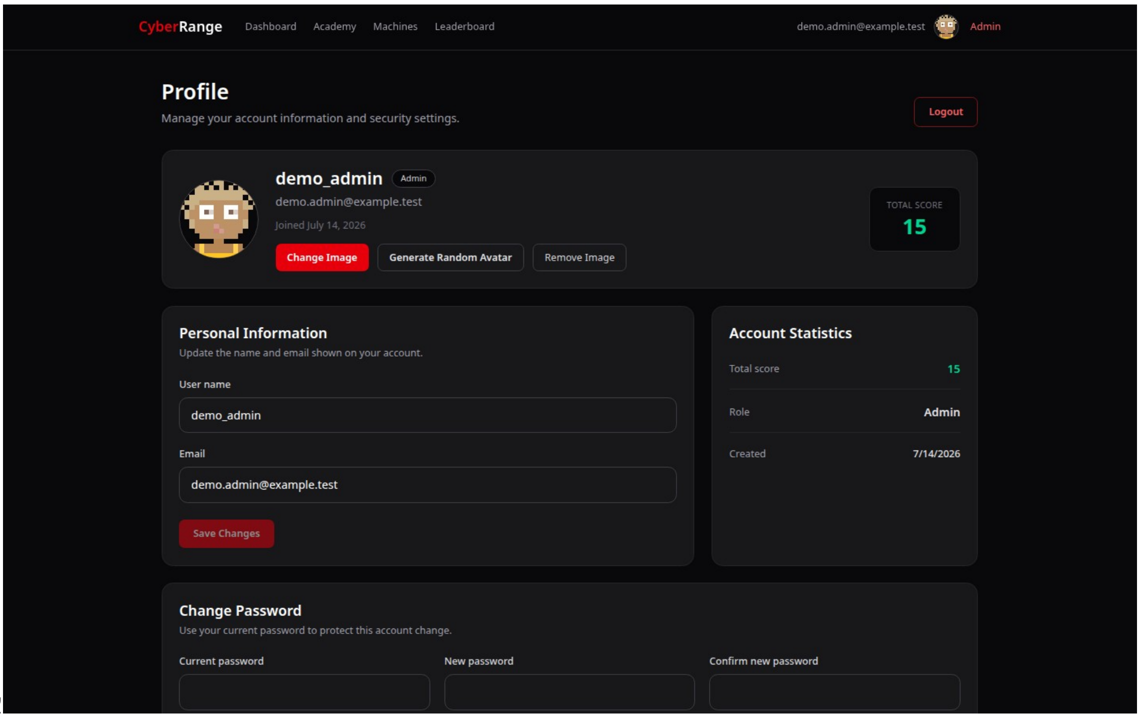
צילום 9 — Machines Route: `/machines` · הרשאה: User. המסך מציג תרחישי מכונה זמינים ופעולות Start Machine. לצורך התיעוד לא נלחץ הכפתור ולא נוצרה מכונה.



צילום 9 — מסך Machines

כיתוב: כרטיסי מכונות עם difficulty ופעולת התחלה שלא הופעלה.

צילום 10 — Profile ו־DiceBear Route: `/profile` · הרשאה: User. מרכז המסך avatar, פרטים אישיים, נתוני חשבון ושינוי סיסמה, ובו מופיע הכפתור `Generate Random Avatar`. זרימת הפיצ'ר היא `Generate → DiceBear HTTP API → Preview → Save Changes → PUT /profile → DB avatar_url → updateStoredUser → AuthContext → Navbar/localStorage`.

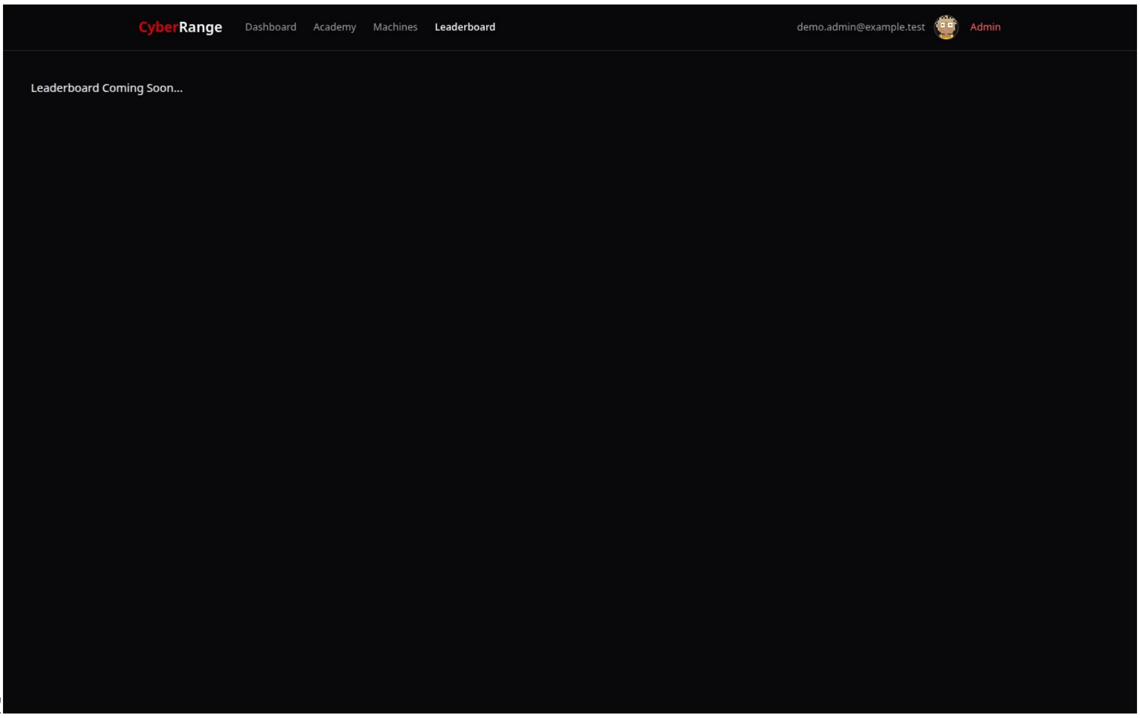


צילום 10 — Profile עם Generate Random Avatar

כיתוב: Profile לאחר מימוש DiceBear, כולל Preview area ופעולת השמירה הקיימת.

האווטאר נבנה מול DiceBear `10.x` בסגנון `pixel-art`; seed חדש נוצר באמצעות `crypto.randomUUID()` וה־URL מקודד באמצעות `encodeURIComponent`. התמונה נטענת מראש ורק לאחר הצלחה מוצג Preview. השמירה מתבצעת רק בלחיצה על `Save Changes`, ונשמר URL בלבד ב־`avatar_url` ולא קובץ, Base64 או bytes. לא נוסף migration. שינוי שם או email אינו שולח מחדש Base64, משום ש־`avatar_url` קיים מצורף ל־payload רק כאשר `hasAvatarChanges` הוא true.

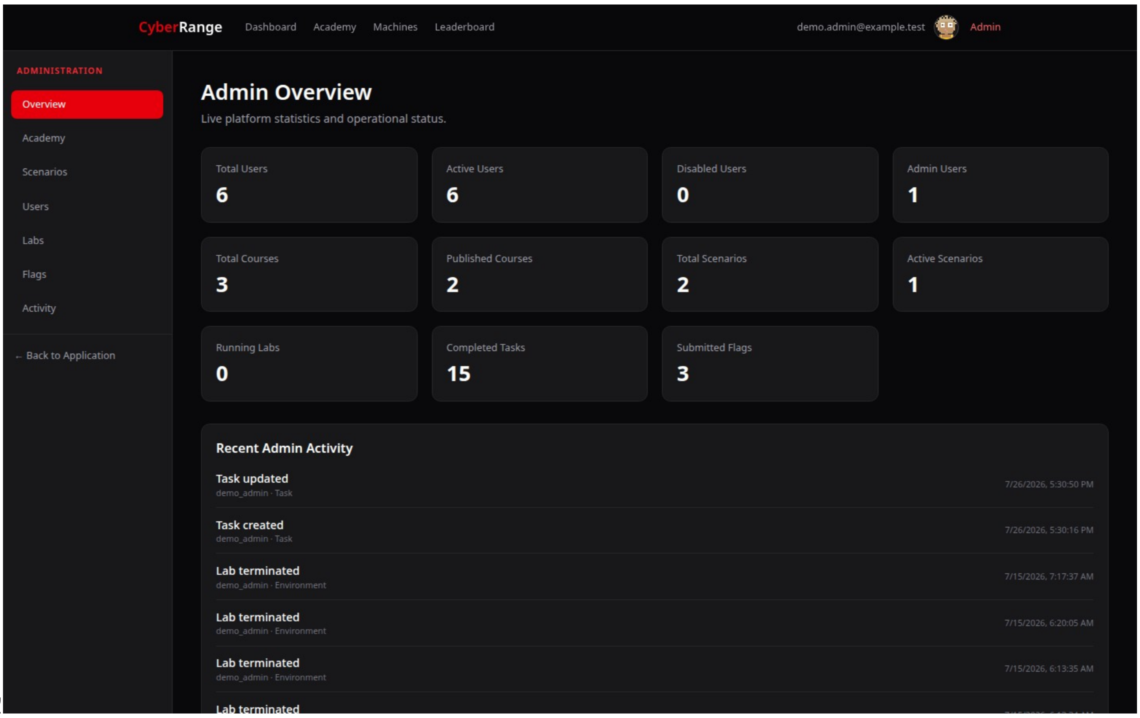
צילום 11 — Leaderboard placeholder Route: `/leaderboard` · הרשאה: User. זהו route עצמאי קיים שמציג כרגע הודעת Coming Soon; אין בו עדיין דירוג פעיל או קריאת API ייעודית.



צילום 11 — Leaderboard במצב הנוכחי

כיתוב: מסך placeholder שממחיש את מצב המימוש בפועל.

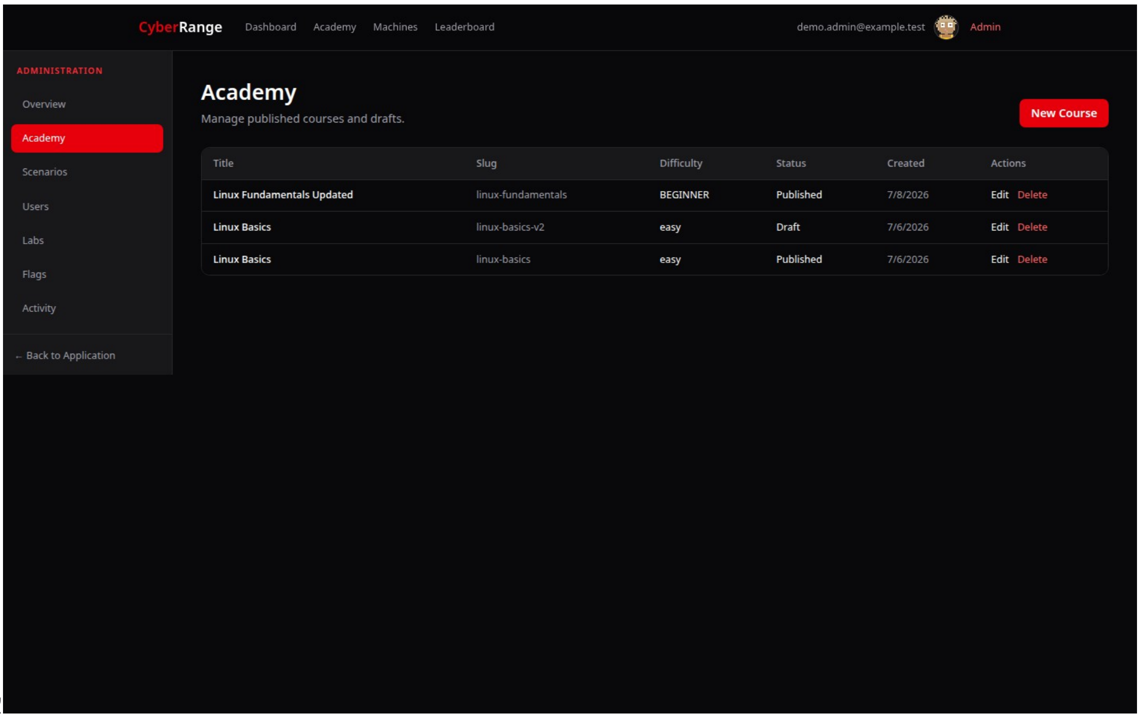
צילום 12 — Admin Overview Route: `/admin` · הרשאה: Admin. המסך מרכז ספירות משתמשים, קורסים, scenarios, labs, tasks ו־flags, יחד עם activity אחרון. הנתונים מגיעים מ־Admin overview endpoints מוגנים.



צילום 12 — Admin Overview

כיתוב: Dashboard תפעולי של מנהל עם ניווט לכל מודולי Admin.

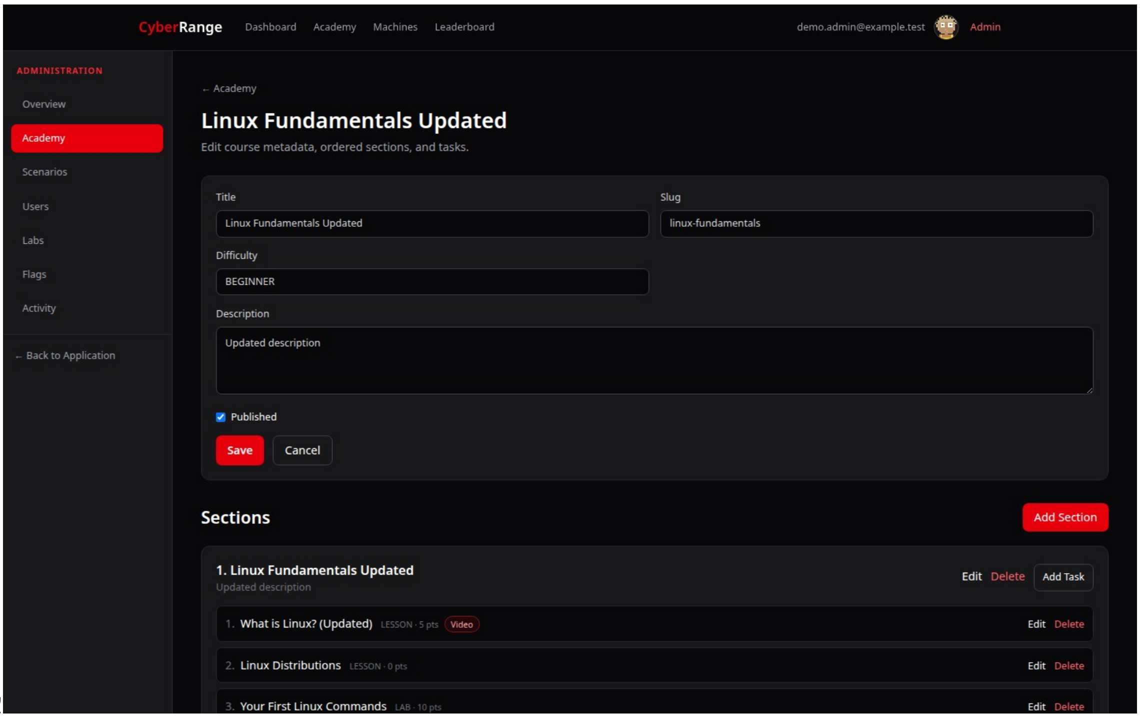
צילום 13 — ניהול קורסים Route: `/admin/academy` · הרשאה: Admin. הטבלה מציגה קורסים מפורסמים וטיוטות ומציעה New Course, Edit, Delete וקישור לעורך המלא.



צילום 13 — Admin Academy

כיתוב: רשימת קורסים לניהול metadata, פרסום וכניסה לעריכה.

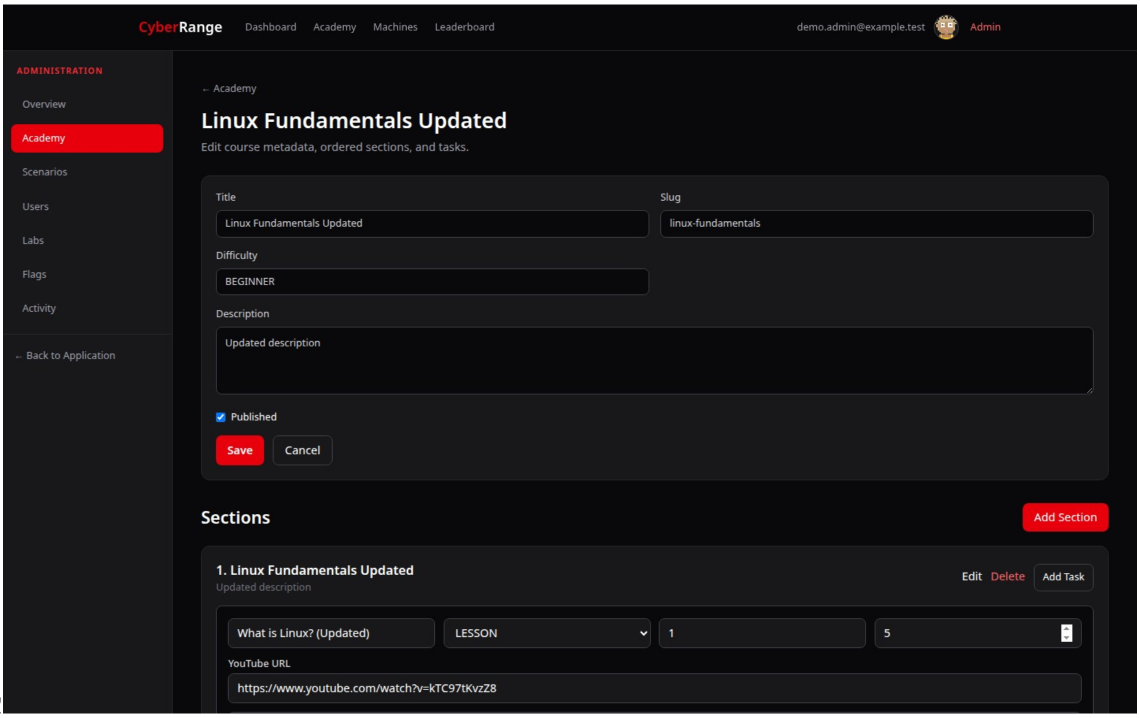
צילום 14 — עריכת קורס, Sections ו־Tasks Route: `/admin/academy/courses/:courseId` · הרשאה: Admin. העורך כולל metadata של הקורס, רשימת Sections מסודרים, ופעולות Tasks הוספה/עריכה/מחיקה. צילום זה מציג את המבנה בלבד; לא נשמר שינוי.



צילום 14 — עורך קורס Admin

כיתוב: Course editor עם טופס הקורס, Section ומשימות מסוג LESSON ו־LAB.

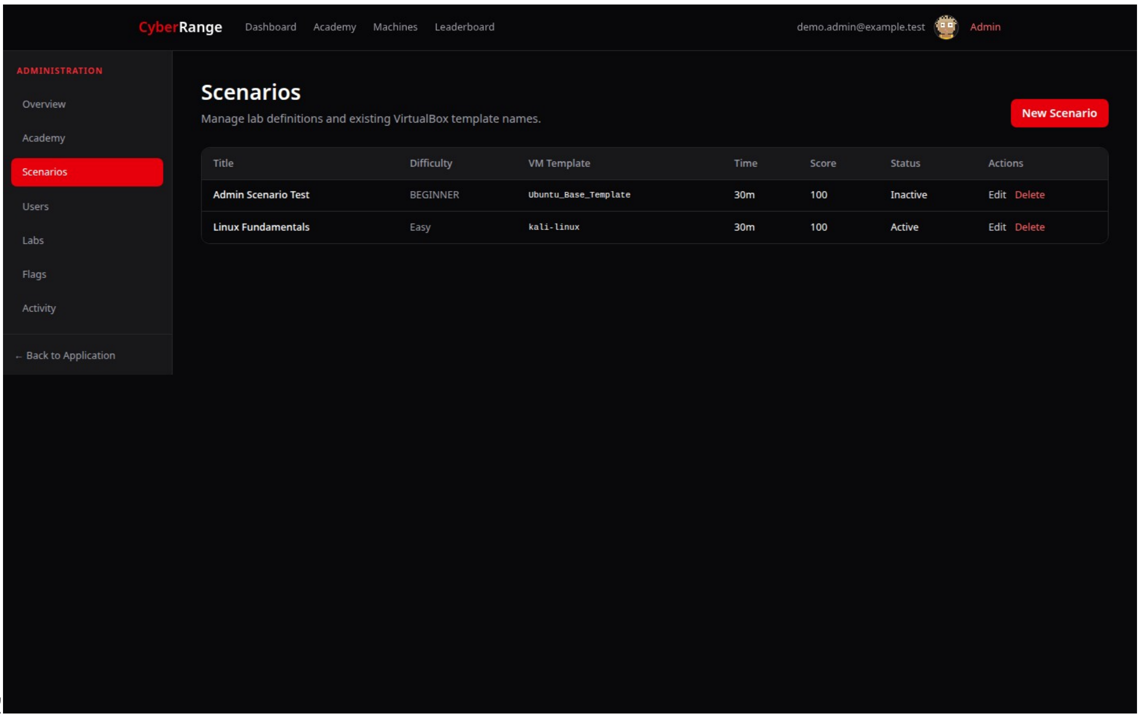
צילום 15 — טופס Task עם YouTube Route: `/admin/academy/courses/:courseId` · הרשאה: Admin. פתיחת Edit מקומית למשימת וידאו מציגה את השדה `YouTube URL`. הטופס מאמת כתובות HTTPS נתמכות ומעביר `video_url` דרך שירותי Academy רק אם המשתמש לוחץ Save Task; בצילום לא בוצעה שמירה.



צילום 15 — טופס משימת YouTube

כיתוב: עריכת LESSON עם שדה YouTube וה־ID הקיים של הווידאו.

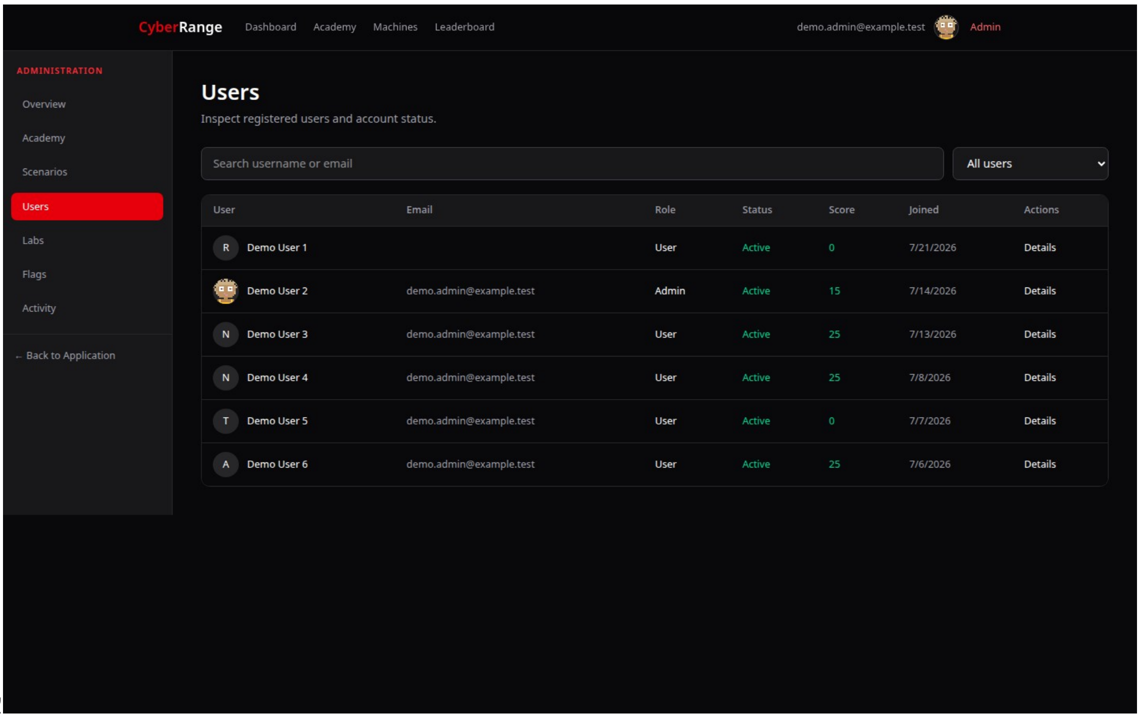
צילום 16 — ניהול Scenarios Route: `/admin/scenarios` · הרשאה: Admin. הטבלה מציגה שם, difficulty, VM template, זמן, score וסטטוס, ומספקת פעולות יצירה ועריכה. לא הופעלה מכונה ולא שונה scenario.



צילום 16 — Admin Scenarios

כיתוב: הגדרות תרחישי Lab והקשר ל-VirtualBox templates.

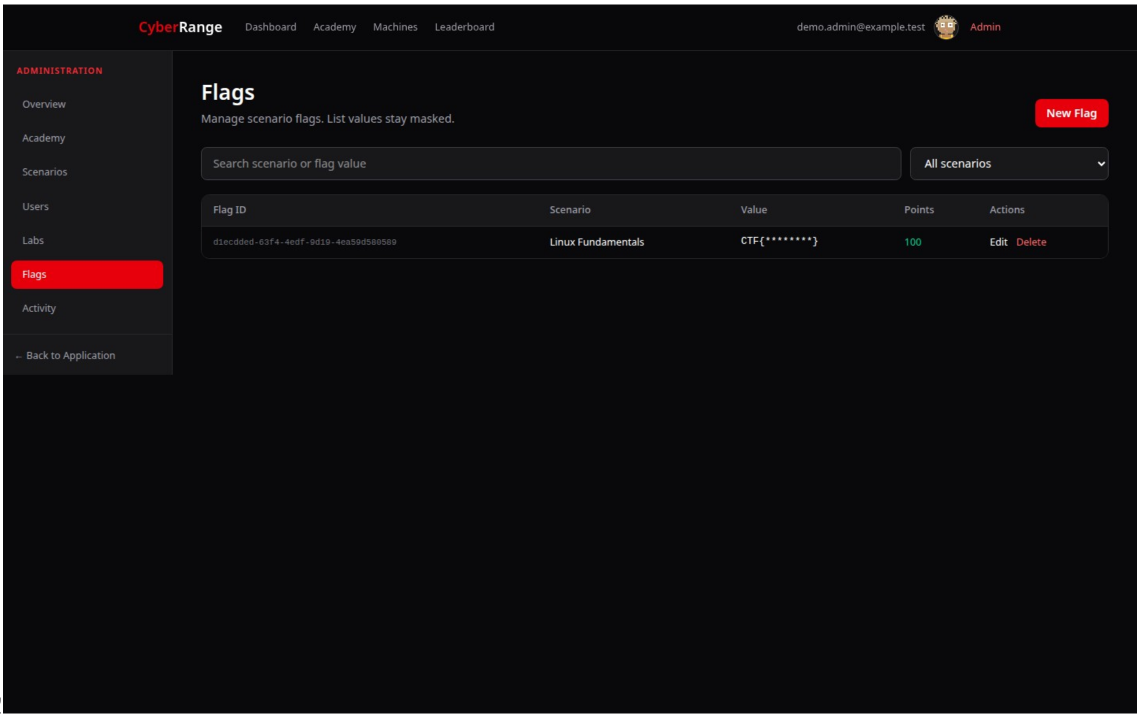
צילום 17 — ניהול משתמשים Route: `/admin/users` · הרשאה: Admin. המסך מאפשר חיפוש וסינון ומציג role, status, score ותאריך הצטרפות. פעולת Details מובילה ל-`/admin/users/:userId`; פרטי הזיהוי בצילום הוחלפו בערכי Demo.



צילום 17 — Admin Users

כיתוב: טבלת משתמשים מאובטחת עם pagination וקישור לפרטי חשבון.

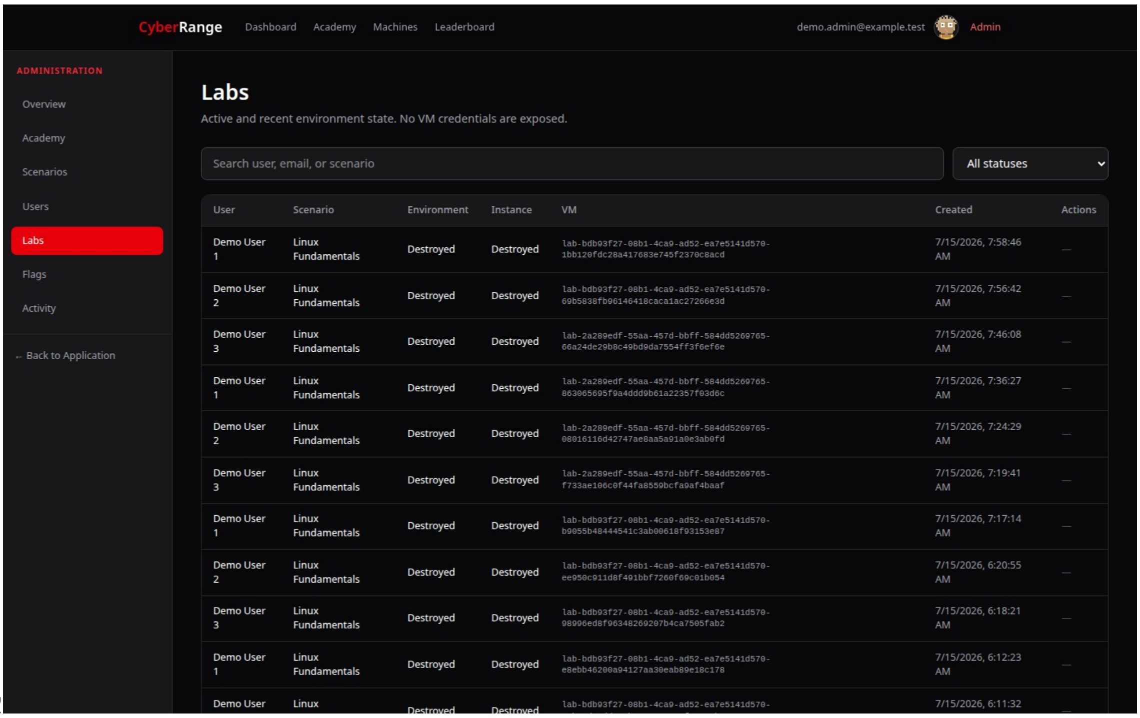
צילום 18 — ניהול Flags Route: `/admin/flags` · הרשאה: Admin. הטבלה מציגה scenario, ממוסך, ומאפשרת יצירה ועריכה בלי לחשוף את הערך המלא ברשימה flag וערך points.



צילום 18 — Admin Flags

כיתוב: ניהול flags כאשר הערך הרגיש נשאר masked.

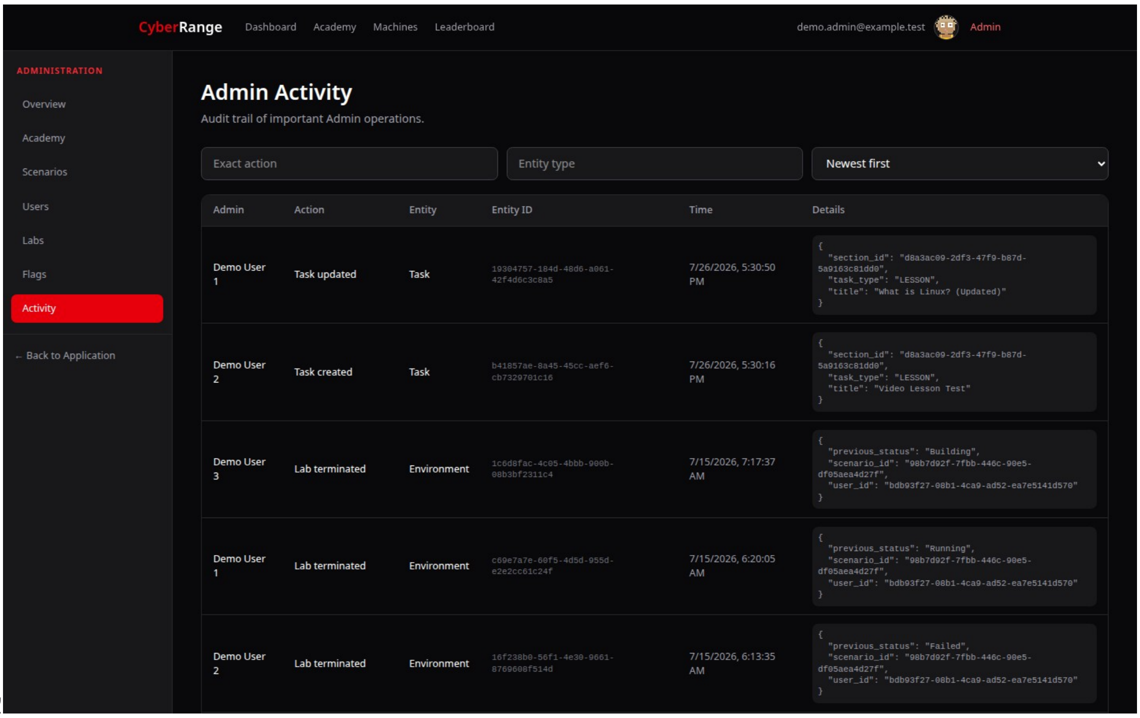
צילום 19 — ניהול Labs Route: `/admin/labs` · הרשאה: Admin. המסך מציג environments זמינה רק לפי המצב; terminate וזמן יצירה. פעולת VM מזהי ,Environment/Instance היסטוריים, מצבי לא הופעלה בצילום.



צילום 19 — Admin Labs

כיתוב: תצוגת lifecycle תפעולית של Labs ללא חשיפת credentials וללא mutation.

צילום 20 — Activity Logs Route: `/admin/activity` · הרשאה: Admin. Audit trail מציג actor, action, entity, זמן ו־JSON details, עם מסננים וסדר כרונולוגי. פרטי המשתמש הוחלפו בערכי Demo.



צילום 20 — Admin Activity Logs

כיתוב: תיעוד פעולות Admin כגון יצירת Task, עדכון Task וסיום Lab.

מצבי UI: Dashboard כולל skeleton/loading, error וקטגוריות continue/available/completed; Academy מציגה locked/available/completed על בסיס `taskUiState`; Labs מציגים provisioning/status/errors ו־terminal; Admin כולל empty state, confirmation modal ו־pagination. route catch-all/404 הוא טקסט פשוט, ולכן האחידות חלקית. אין Loading ייעודי בחלק מהמסכים.

21. קטעי קוד מוסברים

21.1 הרכבת Router

מקור: `backend/src/routes/mod.rs`, `create_api_router`.

```
pub fn create_api_router() -> Router<PgPool> {
    Router::new()
        .nest("/admin", admin_routes::router())
        .nest("/auth", auth::router())
        .nest("/users", user_routes::router())
        .nest("/lab", lab_routes::router())
```

```
        .nest("/academy", academy_routes::academy_routes())
        .nest("/task-progress", task_progress_routes::router())
        .nest("/dashboard", dashboard_routes::router())
        .nest("/profile", profile_routes::router())
}
```

הוא חשוב כי רק נתיבים המורכבים כאן זמינים בפועל. `main.rs` מוסיף `/api`, state ו־CORS. זהו חוזה ה־API העליון.

21.2 JWT ומצב חשבון עדכני

מקור: `backend/src/middleware/auth_middleware.rs`, `auth_middleware`.

```
let mut claims = auth_service::verify_token(token)
    .map_err(|_| AppError::Unauthorized)?;
let user_id: Uuid = claims.sub.parse()
    .map_err(|_| AppError::Unauthorized)?;
let current_user = sqlx::query_as::<_, AuthUserRow>(
    "SELECT role, is_active FROM users WHERE id = $1",
)
.bind(user_id)
.fetch_optional(&pool)
.await?;
let current_user = current_user.ok_or(AppError::Unauthorized)?;
if !current_user.is_active { return Err(AppError::ForbiddenMessage(...)); }
claims.role = if current_user.role == "admin" { Role::Admin } else { Role::User };
req.extensions_mut().insert(claims);
```

ה־JWT מזהה, אך הרשאה עדכנית מגיעה מה־DB. כך token ישן אינו משמר Admin לאחר הורדת role.

21.3 נעילת משימות

מקור: `backend/src/services/task_progress_service.rs`, `apply_access_status`.

```
let mut found_first_incomplete = false;
for task in tasks {
    match task.progress_status {
        TaskProgressStatus::Completed =>
            task.access_status = TaskAccessStatus::Available,
        TaskProgressStatus::NotStarted | TaskProgressStatus::InProgress => {
            if found_first_incomplete {
                task.access_status = TaskAccessStatus::Locked;
            } else {
                task.access_status = TaskAccessStatus::Available;
                found_first_incomplete = true;
```

```
            }
        }
    }
}
```

האלגוריתם נגזר בכל קריאה ואינו שומר LOCKED. הסדר מגיע מה־repository לפי section/task order.

21.4 יצירת Lab

מקור: `backend/src/services/instance_service.rs`, `create_user_lab`.

```
let environment = environment_repo::create_environment(
    pool, user_id, scenario_id, next_expiration()
).await?;
let host_ssh_port = network::get_available_port();
let instance = instance_repo::create_instance(
    pool, environment.id, &vm_name, i32::from(host_ssh_port), true
).await?;
let virtual_machine_result = task::spawn_blocking(move || {
    virtualbox_manager::clone_vm(&template_name, &vm_name_for_virtualbox)?;
    virtualbox_manager::start_vm(&vm_name_for_virtualbox, host_ssh_port)?;
    virtualbox_manager::wait_for_ssh(host_ssh_port, Duration::from_secs(120))
}).await?;
```

קוד blocking אינו חוסם async worker. לעומת זאת request עדיין ממתין לכל provisioning.

21.5 VirtualBox

מקור: `backend/src/utils/virtualbox_manager.rs`, `clone_vm`/`start_vm`.

```
let output = run_vbox_command(&[
    "clonevm", base_vm,
    "--mode", "all",
    "--name", new_vm_name,
    "--register",
])?;
configure_ssh_port_forwarding(vm_name, host_ssh_port)?;
let output = run_vbox_command(&[
    "startvm", vm_name,
    "--type", "headless",
])?;
```

שם template ו־VM מועברים כארגומנטים ל־`Command`, לא דרך shell, ולכן אין shell injection ישיר.

21.6 Bridge WebSocket–SSH

מקור: `terminal_handler.rs:180-209` ו־`ssh_manager.rs:17-31`.

```
let (input_sender, input_receiver) = mpsc::channel::<String>(100);
let (output_sender, mut output_receiver) = mpsc::channel::<String>(100);
let ssh_task = tokio::task::spawn_blocking(move || {
    ssh_manager::connect_and_bridge(
        ssh_port, SSH_USERNAME, SSH_PASSWORD,
        input_receiver, output_sender,
    )
});
```

הערוצים bounded מספקים backpressure בסיסי. credentials הוחלפו בתיאור `<REDACTED>` כאן בספר ואינם מועתקים.

21.7 טרנזקציית דגל

מקור: `backend/src/repositories/ctf_repo.rs`, `submit_flag_and_complete_task`.

```
INSERT INTO user_flags (user_id, flag_id)
VALUES ($1, $2)
ON CONFLICT (user_id, flag_id) DO NOTHING;

INSERT INTO user_task_progress (..., status, completed_at, earned_points)
SELECT ..., 'COMPLETED', now(), t.points FROM tasks t WHERE t.id = $2
ON CONFLICT (user_id, task_id) DO UPDATE SET
  status = 'COMPLETED',
  earned_points = CASE WHEN user_task_progress.status = 'COMPLETED'
    THEN user_task_progress.earned_points ELSE EXCLUDED.earned_points END;
```

שתי הפעולות מתבצעות בטרנזקציה אחת, והניקוד אינו מוכפל בהשלמה חוזרת.

21.8 AuthContext

מקור: `frontend/src/context/AuthContext.jsx`.

```
const [initialSession] = useState(readStoredSession);
const [user, setUser] = useState(initialSession.user);
const [token, setToken] = useState(initialSession.token);
const login = async (email, password) => {
  const data = await loginRequest(email, password);
  setUser(data.user);
  setToken(data.token);
```

```
  localStorage.setItem("token", data.token);
  localStorage.setItem("user", JSON.stringify(data.user));
};
```

ה־Context הוא מקור state נוח ל־UI; localStorage הוא persistence אך גם סיכון XSS.

21.9 API service

מקור: `frontend/src/features/labs/services/labService.js`.

```
export async function submitLabFlag(environmentId, taskId, flagValue) {
  const response = await fetch(`${BASE_URL}/submit`, {
    method: "POST",
    headers: getAuthHeaders(),
    body: JSON.stringify({
      env_id: environmentId,
      task_id: taskId,
      flag: flagValue.trim(),
    }),
  });
  return parseResponse(response);
}
```

ה־service מתרגם naming של UI ל־DTO המדויק ושומר את רכיב התצוגה נקי מפרטי transport.

21.10 component מרכזי

`TaskRenderer.jsx` בוחר layout לפי `task_type`; `TerminalWrapper.jsx` מנהל lifecycle של xterm, fit, WebSocket ו־cleanup ב־effect. הפרדה היא בין content model חשיבותם לבין interaction widgets. קטעים מלאים לא הועתקו כדי להישאר בגבול 10–30 שורות לקטע.

22. התקנה והרצה על Ubuntu

22.1 דרישות מוקדמות

- Rust toolchain התומך edition 2024 ו־Cargo.
- Node.js התואם Vite 8; לא צוין מספר Node מדויק בקבצים ולכן **לא ניתן לאמת** גרסה מינימלית מעבר לדרישת Vite.
- PostgreSQL נגיש, עם database ומשתמש ייעודיים.

- VirtualBox ופקודת `VBoxManage` ב־PATH.
- VM template ששמו תואם `scenarios.vm_template_name`, עם SSH על guest port 22 ו־credentials שה־Backend מצפה להם.
- רשימה מדויקת של ספריות build מערכתיות הנדרשות ל־`ssh2`/OpenSSL בהתאם להפצת Ubuntu; אינה מתועדת בריפו.

22.2 משתני סביבה

צור `backend/.env` בלי להכניס ערכים למסמכים:

```
DATABASE_URL=<REDACTED>
JWT_SECRET=<REDACTED>
LAB_IDLE_TIMEOUT_MINUTES=20
```

`DATABASE_URL` ו־`JWT_SECRET` חובה; timeout אופציונלי. ב־Frontend ניתן להגדיר `VITE_API_URL`; ברירת המחדל היא `http://localhost:3000`.

22.3 הכנת DB והרצה

של המפעיל, יש להריץ PostgreSQL לאחר יצירתם בכלי role/database. ליצירת script הריפו אינו מכיל את migrations פקודת. לפי הסדר שבתיקייה SQLx CLI מקבלת היא `sqlx migrate run`, אך ה־CLI בקובצי הפרויקט ולכן יש לראות בה דרישת כלי ולא בפקודה שהריפו מספק. חלופה dependency אינו קובצי את להחיל היא בטוחה SQL לפי שמם בעזרת כלי ניהול PostgreSQL.

פקודות שקיימות מהמבנה בפועל:

```
cd backend
cargo run

cd frontend
npm install
npm run dev
```

Backend מאזין קשיח על `0.0.0.0:3000`. Vite בוחר ברירת מחדל מקומית ומעביר `/api` ל־`http://localhost:3000`. build הוא `npm run build`; preview הוא `npm run preview`.

22.4 בדיקות smoke מוצעות לאחר התקנה

פתח `/`, `/login` ו־`/register`; רשום משתמש בדיקה; ודא login ו־Dashboard; צור course/scenario/template רק בסביבת test; פתח LAB ובדוק שה־DB עובר Building→Running, ש־SSH זמין ושה־terminal מתחבר; מחק ובדוק `Destroyed` והיעלמות ה־VM. אין לבצע smoke Labs ללא גיבוי ובידוד מול נתוני production.

22.5 תקלות נפוצות

- `DATABASE_URL missing` או DB connection fail: בדוק `.env` ו־PostgreSQL.
- compile errors של SQLx: ה־DB חייב להיות נגיש ובעל schema תואם בזמן build; אין `.sqlx` offline metadata.
- `template does not exist`: `vm_template_name` אינו VM רשום ב־VirtualBox.
- SSH timeout: guest לא עלה, NAT rule/SSH service/credentials אינם תואמים.
- “already have active lab”: סיים את environment הפעיל או נקה סטייה בזהירות.
- 409: environment/instance אינם Running או אין ssh_port.
- CORS/URL: בדוק `VITE_API_URL`; ב־production יש לצמצם CORS ולספק WSS.

23. בדיקות ואימות

הבדיקות הורצו ב־5 באוגוסט 2026 על `main@dc4f66b`. לא הופעלו migrations, לא שונו נתוני DB ולא הופעלה VirtualBox.

פקודה	תוצאה	הערות
`cargo fmt --check`	PASS	ללא output וללא שינוי קבצים
`cargo test`	PASS, 12/12	נדרשה גישת read למסכת PostgreSQL עבור SQLx compile-time; 8 warnings על dead code/unused
`npm run lint`	PASS	ESLint ללא ממצאים

פקודה	תוצאה	הערות
`npm run build -- --outDir /tmp/home-lab-manager-doc-build`	PASS	108 modules; warning: JS chunk 618.80KB, gzip 163.77KB
Vite UI smoke	PASS חלקי	landing/login/register נפתחו וצולמו; routes מוגנים לא נבדקו ללא session
migrations	לא הורצו	מטעמי בטיחות; נותחו סטטית בלבד
VirtualBox/SSH/Lab E2E	לא הורץ	דורש mutation של VM ונתונים ואישור מפורש

בדיקות Backend הן unit tests בלבד עבור YouTube parsing, task type normalization ו-tri-state video patch. לא נמצאו `#[tokio::test]`, integration test directory או mocks לתשתית. לא נמצאו בדיקות Frontend. `cargo test` אינו מוכיח provisioning, REST status mapping או security behavior ב-runtime.

24. אבטחה, מגבלות וחוב טכני

חומרה	ממצא	ראיה/השפעה	המלצה
קריטי	לא נמצא ממצא מאומת	—	להמשיך secret scanning ו-threat modeling
גבוה	credentials של SSH קשיחים בקוד	`terminal_handler.rs:192-205`; compromise של repo/logical secret	secret store או credentials פר-scenario, rotation, keys
גבוה	token ב-localStorage וב-WebSocket URL	`AuthContext.jsx`, `labService.js:126-134`; XSS/log leakage	HttpOnly cookie או in-memory + refresh; one-time WS ticket
גבוה	אין host-key verification ב-SSH	`ssh_manager.rs`; חשיפה ל-MITM מקומי/endpoint שגוי	known_hosts/fingerprint pinning
בינ	CORS `Any`	`main.rs:42-45`	allowlist מהגדרה סביבתית

חומרה	ממצא	ראיה/השפעה	המלצה
וני	origins/headers		
בינוני	ללא registration password policy ואימות email	auth_service.rs:11	rate limit, אחיד validation verification
בינוני	flag values plaintext והשוואה רגילה	migrations + ctf_repo	hash/HMAC flags; constant-time verify; scoped task relation
בינוני	SSH input מודפס ללוג	ssh_manager.rs:61	להסיר/למסך input logs
בינוני	provisioning סינכרוני וללא reconciliation	create_user_lab, cleanup loop	job state machine, retries, startup reconciliation
בינוני	race בהקצאת port	release ואז bind ephemeral	allocator transaction/port range reservation, retry על VBox conflict
בינוני	CTF flag נקשר scenario ולא task	schema/service	join table או task_id ־flags ל
בינוני	אין rate limiting	auth/flag/WS routes	tower layer / reverse proxy limits
נמוך	שגיאות Lab/Auth ממופות ממחרוזות ולעיתים נחשפות	handlers	client-safe־ו typed errors messages
נמוך	user_flags FKs nullable	migration	NOT NULL־ל migration לאחר audit
נמוך	Stopped מוגדר אך לא נבנה	status enums + warnings	להסיר או לממש lifecycle ברור
שיפור	אין health/readiness/metrics	routes/main	endpoints, tracing subscriber init, structured logs/metrics
שיפור	bundle גדול	Vite build warning	route-level lazy loading/code splitting

חומרה	ממצא	ראיה/השפעה	המלצה

SQL injection: queries משתמשים ב־SQLx parameters, כולל Admin filters, ולא נמצאה הרכבה SQL ישירה מערכי משתמש. Command injection: VBox arguments מועברים ל־`Command::new` כ־args. עם זאת, template names צריכים עדיין allowlist כדי למנוע שימוש תפעולי בשמות לא צפויים. Password hashes אינם נשלחים ב־DTOs המאומתים.

25. מצב המימוש הנוכחי

מודול	מצב	מה קיים	מה חסר	ראיות
Auth/JWT	ממומש ומאומת	register/login/bcrypt/JWT/current DB role	refresh/revoke/rate limit/policy	auth route/service/middleware
Roles	ממומש ומאומת	user/admin, immediate changes	granular permissions	Role enum/admin middleware
Profile	ממומש ומאומת	details/edit/password/avatar URL	upload/storage/re-auth policy	profile files
Academy	ממומש ומאומת	courses/sections/tasks/publish/order	enrollment/prerequisites מעבר לסדר	academy files/migrations
Video lessons	ממומש ומאומת	safe YouTube parsing/storage/widget	provider alternatives/captions	migration, youtube utils, VideoWidget
Progress	ממומש ומאומת	start/complete/lock/points/percentage	tests אינטגרטיביים	progress service/repo
Dashboard	ממומש ומאומת בקוד/	continue/available/completed/	runtime UI לא צולם	dashboard files

מודול	מצב	מה קיים	מה חסר	ראיות
		statistics		
Labs	ממומש ומאומת בקוד	create/delete/ status/expiry/one active	runtime VBox לא נבדק; reconciliation	instance service/migrations
Terminal	ממומש ומאומת בקוד	authenticated WS/SSH/xterm	resize, keys, host verify, reconnect	terminal/ssh/ TerminalWrapper
CTF	ממומש ומאומת	exact flag, transaction, task points	flag hashing/task-specific relation	ctf repo/service
Admin Academy	ממומש ומאומת	CRUD + logs	bulk/reorder UI	admin academy files
Admin users/labs/flags/activity	ממומש ומאומת	pagination/filters/ actions	export, security events	admin files
Leaderboard	מתוכנן בלבד	placeholder route	backend, model, real page	`App.jsx:63-69`
Deployment	לא ניתן לאמת	local commands בלבד	container/ service/TLS/ backups	אין deployment files

MVP שהושלם: auth, learning flow, single-VM Labs, terminal, flag completion, profile בסיסי. יכולות חלקיות: observability, cleanup robustness, test automation ו־Admin security hardening. התכנון ב־`docs/ROADMAP.md` כקיים משום Docker תואם ברובו את הפערים; אין להציג Dockerfile/Compose שאין בקוד.

26. תוכנית המשך

26.1 דחוף — אבטחה ושלמות

1. להוציא SSH credentials מהקוד, לעבור למפתחות/secret store ולאמת host key.

2. להסיר logging של terminal input ולהחליף token query WS בכרטיס חד־פעמי.
3. לצמצם CORS, להוסיף password policy, rate limiting ו־security headers.
4. להגדיר relation בין flag ל־task ולהגן/להצפין או לבצע HMAC לערכים.
5. להוסיף migration ל־NOT NULL ב־`user_flags` לאחר ניקוי נתונים.

26.2 אמינות ובדיקות

לבנות integration tests מול PostgreSQL זמני, contract tests לכל endpoint, Frontend tests ו־Playwright E2E. ליצור adapter traits ל־VirtualBox/SSH כדי לבדוק success/failure ללא VM. להוסיף retries, locking cleanup, startup reconciliation ו־outbox/job queue ל־provisioning.

26.3 מוצר ותפעול

לשפר Dashboard, lazy-load Admin/xterm, להוסיף audit למשתמש ול־login, metrics, health/readiness, retention ו־backup docs. Leaderboard יפותח רק לאחר הגדרת score. Deployment צריך לכלול least-privilege service account, TLS/WSS ו־VirtualBox host isolation. Containers/Docker הם הצעה בלבד, לא מצב קיים; יש לבדוק התאמה לגישה הנדרשת ל־VirtualBox host.

27. סיכום ומסקנות

הפרויקט מדגים חיבור לא שגרתי בין מערכת לימוד Web לבין תשתית VM מקומית. נקודות החוזקה הן הפרדת features, ownership שמופק מ־JWT, רענון role מה־DB, נעילת משימות בצד שרת, constraints למסד, transaction של CTF ו־cleanup policy. מנגנון YouTube מציג validation רב־שכבתי טוב.

האתגר המרכזי אינו CRUD אלא עקביות בין PostgreSQL, תהליך Backend ו־VirtualBox תחת כשל. הפרויקט הגיע ל־MVP משמעותי, אך production readiness דורש הסרת secrets, הקשחת token/SSH/CORS, observability ובדיקות אינטגרציה. לאחר אלה אפשר להרחיב ל־multi-instance scenarios, deployment ו־Leaderboard בלי לערבב אותם עם המצב הנוכחי.

28. נספחים

נספח א — מילון מונחים

מונח	הגדרה בפרויקט
Course	יחידת לימוד עליונה
Section	פרק מסודר בתוך קורס
Task	פעילות LESSON/PRACTICE/LAB
Scenario	הגדרת Lab הכוללת template ומטא־נתונים
Environment	הקצאת scenario למשתמש ומחזור החיים שלה
Instance	רשומת VM בתוך environment; כיום אחת
Flag	secret challenge value ברמת scenario
Progress	סטטוס פר־user/task ונקודות שנצברו
Claims	`sub`, `exp`, `role` ב־JWT
Entry point	instance שאליה מתחברים; הקוד יוצר אחת כ־true

נספח ב — משתני סביבה

שם	חובה	שימוש	ברירת מחדל
`DATABASE_URL`	כן	startup ו־SQLx compile	אין
`JWT_SECRET`	כן	חתימה ואימות JWT	אין
`LAB_IDLE_TIMEOUT_MINUTES`	לא	חיובי בדקות expiry	20
`VITE_API_URL`	לא	origin ל־REST/WS	`http://localhost:3000`

נספח ג — רשימת migrations

1. `20260517080221_create_users_table.sql`
2. `20260706012517_create_academy_tables.sql`

3. `20260706013954_create_lab_engine_tables.sql`
4. `20260706014523_add_role_to_users.sql`
5. `20260706014546_add_total_score_to_users.sql`
6. `20260714000000_add_profile_avatar.sql`
7. `20260715000000_admin_operations.sql`
8. `20260715005000_complete_runtime_schema.sql`
9. `20260715010000_score_and_lab_policy.sql`
10. `20260726000000_add_lesson_video.sql`

נספח ד — מפת קבצים מרכזיים

נושא	קבצים
bootstrap/routes	`backend/src/main.rs`, `backend/src/routes/*.rs`
auth/security	`models/user.rs`, `services/auth_service.rs`, `middleware/*.rs`
Academy	`academy_*`, `models/dto/{course,section,task}.rs`, frontend `features/academy`
Progress	`task_progress_*`, `features/academy/hooks/useCourseProgress.js`
Labs	`instance_service.rs`, environment/instance repos, VBox/network utils, `features/labs`
Terminal	`terminal_handler.rs`, `ssh_manager.rs`, `TerminalWrapper.jsx`
CTF	`ctf_repo.rs`, `lab_handler.rs`, `FlagWidget.jsx`, `useCTF.js`
Profile	profile handler/service/repo/DTO ו-`features/profile`
Admin	admin route/handler/service/repo/DTO ו-`features/admin`
DB	`backend/migrations/*.sql`

נספח ה — מגבלות האימות

לא נוצר משתמש; לא צולמו migrations ולא נוצרה/נמחקה מכונה; לא הורצו VirtualBox לא הופעלה קיים. לכן יכולות אלו מסווגות "ממומש template או VM של credentials מסכים מוגנים; לא אומתו

נקראו רק לצורך זיהוי `backend/.env` ומאומת בקוד” ולא “אומת בהפעלה מלאה”. פרטי הסוד שב־ שמות משתנים ולא הועתקו לתוצרים.

נספח ו — מקורות

מקור ראשי: commit `dc4f66b10bfcd43e2e47f47a8f7f83986c49ef92`. הראיות המפורטות נמצאות ב־`EVIDENCE_MATRIX.md`; תוצאות הבדיקות ב־`VALIDATION_REPORT.md`; שאלות שנותרו ב־`OPEN_QUESTIONS.md`. קובצי המקור לכל התרשימים הם `assets/*.dot`, ולכל תרשים קיימים SVG ו־PNG.

נספח ז — קטלוג API מלא

הקטלוג נגזר ישירות מקובצי `backend/src/routes/*.rs`. כל path להלן מקבל prefix של `/api`. הסימון “JWT” משמעו `auth_middleware`; “Admin” משמעו Auth ולאחריו `admin_middleware`; במסלול terminal האימות מתבצע ידנית באמצעות query token לפני upgrade. עמודת status מציגה חוזה מרכזי ולא כל `500` אפשרי. שמות repository מקוצרים אך מפנים למודול המדויק.

ז.1 Auth, Users, Dashboard ו־Profile

Method	Path	Auth/role	Request	Response	Statuses	Handler	Service	Repository
POST	`/auth/register`	ציבורי	`RegisterRequest`	`UserResponse`	201; 400	`auth_handler::register`	`auth_service::register`	`user_repo`
POST	`/auth/login`	ציבורי	`LoginRequest`	token + user	200; 401	`auth_handler::login`	`auth_service::login`	`user_repo`
GET	`/users/`	Admin	—	users array	200; 401/403; 500	`user_handler::get_users`	`user_service::fetch_all_users`	`user_repo`
GET	`/dashboard/`	JWT	Claims	dashboard DTO	200; 401; 404; 500	`dashboard_handler::get_dashboard_handler`	`dashboard_service::get_dashboard`	`dashboard_repo`, academy/progress repos
GET	`/profile/`	JWT	Claims	profile DTO	200; 401; 404	`profile_handler::get_profile_handler`	`profile_service::get_profile`	`profile_repo`

Method	Path	Auth/role	Request	Response	Statuses	Handler	Service	Repository
PUT	`/profile/`	JWT	name + email	profile DTO	200; 400; 409	`profile_handler::update_profile_handler`	`profile_service::update_profile`	`profile_repo`
PUT	`/profile/password`	JWT	current/new/confirm	message	200; 400; 401	`profile_handler::change_password_handler`	`profile_service::change_password`	`profile_repo`
PUT	`/profile/avatar`	JWT	data URL או null	profile DTO	200; 400; 413	`profile_handler::update_avatar_handler`	`profile_service::update_avatar`	`profile_repo`

2.ז Academy ו־Progress

Method	Path	Auth/role	Request	Response	Statuses	Handler	Service	Repository
GET	`/academy/courses/`	ציבורי	—	קורסים מפורסמים	200; 500	`academy_handler::get_courses_handler`	`academy_service::get_courses`	`academy_repo`
GET	`/academy/courses/:id/full`	ציבורי	course UUID	course tree	200; 404; 500	`academy_handler::get_course_full_handler`	`academy_service::get_course_full`	`academy_repo`
GET	`/task-progress/courses/:course_id`	JWT	course UUID	progress + access	200; 401; 404	`task_progress_handler::get_course_progress_handler`	`task_progress_service::get_course_progress`	`task_progress_repo`

Repository	Service	Handler	Statuses	Response	Request	Auth/role	Path	Method
`task_progress_repo`	`task_progress_service::start_task`	`task_progress_handler::start_task_handler`	200; 401; 403; 404	progress DTO	task UUID	JWT	`/task-progress/tasks/:task_id/start`	POST
`task_progress_repo`	`task_progress_service::complete_content_task`	`task_progress_handler::complete_content_task_handler`	200; 400; 403; 404	progress DTO	task UUID	JWT	`/task-progress/tasks/:task_id/complete`	POST

3.ו Labs, Terminal ו־CTF

Repository	Service	Handler	Statuses	Response	Request	Auth/role	Path	Method
scenario/ environment/ instance repos	`instance_service::create_user_lab`	`lab_handler::handle_create_lab`	201; 400; 500	env, port, expiry	`scenario_id`	JWT	`/lab/create`	POST
environment/ instance repos	`instance_service::delete_user_lab`	`lab_handler::handle_delete_lab`	200; 404; 500	message	`env_id`	JWT	`/lab/delete`	POST
environment/ task/ctf repos	`instance_service::verify_and_submit_flag`	`lab_handler::handle_submit_flag`	200; 400; 404	result message	env, task, flag	JWT	`/lab/submit`	POST
environment/	`instance_service::`	`lab_handler::hand`	200; 404;	status	environ	JWT	`/lab/`	GET

Repository	Service	Handler	Statuses	Response	Request	Auth/role	Path	Method
instance repos	`get_lab_status`	`le_get_lab_status`	500	DTO או null	ment UUID		`status/:environment_id`	
environment/ instance repos	`instance_service::get_any_active_lab`	`lab_handler::handle_get_any_active_lab`	500 ;200	active Lab או null	Claims	JWT	`/lab/active`	GET
environment/ instance repos	`instance_service::get_active_lab`	`lab_handler::handle_get_active_lab`	500 ;200	active Lab או null	scenario UUID	JWT	`/lab/active/:scenario_id`	GET
user/ environment/ instance repos	handler bridge + `ssh_manager`	`terminal_handler::ws_terminal_handler`	;401 ;101 ;404 ;403 500 ;409	+ 101 text frames	env UUID + token	query JWT + owner	`/lab/terminal/:environment_id?token`	GET →WS

4.ז Academy Admin

Repository	Service	Handler	Statuses	Response	Request	Auth/role	Path	Method
`academy_repo`	`academy_service::get_all_courses`	`get_courses_handler`	;200 401/403	כל הקורסים	—	Admin	`/academy/admin/courses`	GET
`academy_repo`	`academy_service::create_course`	`create_course_handler`	;400 ;201 409	course	create course DTO	Admin	`/academy/admin/courses`	POST

Method	Path	Auth/role	Request	Response	Statuses	Handler	Service	Repository
PUT	/academy/admin/courses/:id	Admin	update course DTO	course	200; 400; 404; 409	update_course_handler	academy_service::update_course	academy_repo
DELETE	/academy/admin/courses/:id	Admin	course UUID	empty	204; 404; 409	delete_course_handler	academy_service::delete_course	academy_repo
POST	/academy/admin/sections	Admin	create section DTO	section	201; 400; 409	create_section_handler	academy_service::create_section	academy_repo
GET	/academy/admin/sections/:id	Admin	section UUID	section	200; 404	get_section_handler	academy_service::get_section_by_id	academy_repo
PUT	/academy/admin/sections/:id	Admin	update section DTO	section	200; 400; 404; 409	update_section_handler	academy_service::update_section	academy_repo
DELETE	/academy/admin/sections/:id	Admin	section UUID	empty	204; 404; 409	delete_section_handler	academy_service::delete_section	academy_repo
GET	/academy/admin/courses/:course_id/sections	Admin	course UUID	sections ordered	200; 404	get_sections_by_course_handler	academy_service::get_sections_by_course	academy_repo
POST	/academy/admin/tasks	Admin	create task DTO	task	201; 400; 409	create_task_handler	academy_service::create_task	academy_repo
GET	/academy/admin/	Admi	task UUID	task	200; 404	get_task_handler	academy_service::get_	academ

Method	Path	Auth/role	Request	Response	Statuses	Handler	Service	Repository
	`tasks/:id`	n					`task_by_id`	`y_repo`
PUT	`/academy/admin/tasks/:id`	Admin	update task DTO	task	200; 400; 404; 409	`update_task_handler`	`academy_service::update_task`	`academy_repo`
DELETE	`/academy/admin/tasks/:id`	Admin	task UUID	empty	204; 404; 409	`delete_task_handler`	`academy_service::delete_task`	`academy_repo`
GET	`/academy/admin/sections/:section_id/tasks`	Admin	section UUID	tasks ordered	200; 404	`get_tasks_by_section_handler`	`academy_service::get_tasks_by_section`	`academy_repo`

מוצגים לפי שמות service Admin. באמצעות שכבת activity גם מנסה לרשום mutation כל; `academy_admin_handler` בטבלה זו נמצאים ב־ handlers כל
`academy_service.rs` ־וב `models/dto` נמצא ב־ validation ושל DTO אחריותם; החוזה המדויק של

5.ז Admin Operations

Method	Path	Auth/role	Request	Response	Statuses	Handler	Service	Repository
GET	`/admin/dashboard`	Admin	—	statistics + activity	200; 401/403; 500	`admin_handler::dashboard`	`admin_service::get_dashboard`	`admin_repo`
GET	`/admin/users`	Admin	page/search/status	paginated users	200; 400	`admin_handler::users`	`admin_service::get_users`	`admin_repo`

Method	Path	Auth/role	Request	Response	Statuses	Handler	Service	Repository
GET	`/admin/users/:user_id`	Admin	user UUID	user details	200; 404	`admin_handler::user_details`	`admin_service::get_user_details`	`admin_repo`
PUT	`/admin/users/:user_id/status`	Admin	`is_active`	updated user/message	200; 400; 404	`admin_handler::update_user_status`	`admin_service::update_user_status`	`admin_repo`
PUT	`/admin/users/:user_id/role`	Admin	`role`	updated user/message	200; 400; 404	`admin_handler::update_user_role`	`admin_service::update_user_role`	`admin_repo`
PUT	`/admin/users/:user_id/password`	Admin	new + confirm	message	200; 400; 404	`admin_handler::reset_user_password`	`admin_service::reset_user_password`	`admin_repo`
GET	`/admin/labs`	Admin	page/search/status/scenario	paginated labs	200; 400	`admin_handler::labs`	`admin_service::get_labs`	`admin_repo`
POST	`/admin/labs/:environment_id/terminate`	Admin	environment UUID	message	200; 404; 500	`admin_handler::terminate_lab`	`admin_service::terminate_lab` → instance service	admin/environment/instance repos
GET	`/admin/flags`	Admin	page/search/	masked paginated	200; 400	`admin_handler::flags`	`admin_service::get_flags`	`admin_repo`

Repository	Service	Handler	Statuses	Response	Request	Auth/role	Path	Method
				flags	scenario			
admin_repo	admin_service::create_flag	admin_handler::create_flag	;201 ;400 409	flag DTO	create flag DTO	Admin	/admin/flags	POST
admin_repo	admin_service::get_flag	admin_handler::flag	;200 404	full flag details	flag UUID	Admin	/admin/flags/:id	GET
admin_repo	admin_service::update_flag	admin_handler::update_flag	;200 ;400 404	flag DTO	update flag DTO	Admin	/admin/flags/:id	PUT
admin_repo	admin_service::delete_flag	admin_handler::delete_flag	;204 ;404 409	empty/ message	flag UUID	Admin	/admin/flags/:id	DELETE
admin_repo	admin_service::get_activity	admin_handler::activity	;200 400	paginated logs	filters/ page/order	Admin	/admin/activity	GET
admin_repo	admin_service::get_scenarios	admin_handler::scenarios	200	scenarios	filters	Admin	/admin/scenarios	GET
admin_repo	admin_service::create_scenario	admin_handler::create_scenario	;201 ;400 409	scenario	create scenario DTO	Admin	/admin/scenarios	POST
admin_repo	admin_service::updat	admin_handler::	;200	scenario	update	Adm	/admin/	PUT

Method	Path	Auth/role	Request	Response	Statuses	Handler	Service	Repository
	`scenarios/:id`	in	scenario DTO		;400 ;404 409	`update_scenario`	`e_scenario`	
DELETE	`/admin/scenarios/:id`	Admin	scenario UUID	empty/ message	;204 ;404 409	`admin_handler::delete_scenario`	`admin_service::delete_scenario`	`admin_repo`